

前言部分

- **标题:** 超越模式识别的神经网络——一种符号主义和联结主义结合的新范式
- **作者:** Chao Zheng | 独立研究者
- **摘要:** 本研究挑战了神经网络只能作为统计模式识别器的传统印象，提出并验证了一种通程序化理想数据来激发其精确规则学习能力的训练范式，并通过这种范式将标准神经网络从概率性的模仿者转变为确定性的规则执行器。本文将这类训练设置称为神经雕刻范式：任务由程序生成，输入输出空间预先固定，样本来自目标分布的随机采样，输出采用一次性结构化预测，训练目标是学习从输入到唯一确定输出的变换关系。通过符号规则、算法拟合、元胞自动机、结构化图像变换等一系列任务，表明标准神经网络可以在受控条件下学习接近精确的规则映射，有力的证明了这个范式的通用性。其中，语义洗牌与位置洗牌实验表明，模型的学习结果不能简单解释为对表层符号含义或固定位置模板的记忆。进一步的架构对比实验显示，这一能力并非 Transformer 所独有——MLP、CNN、RNN、UNet、Diffusion 等多种架构也在部分任务中表现出类似能力，这说明精确规则学习很可能不是某种特定架构的偶然性质，而是作为联结主义系统的神经网络的更一般的内在属性。进一步的元胞自动机插值实验和 MNIST+ 元胞自动机实验显示，模式识别与规则执行可以在同一模型中自然耦合，强烈暗示二者很可能不是彼此割裂的能力，而是都是神经网络的内生能力，甚至很可能是同一种能力在不同数据形式下的表现。基于这些结果，本文提出一种压缩即智能/MDL 风格的解释：神经网络的学习过程可以被理解为不断逼近最低 Kolmogorov 复杂度表示的过程，即从大量样本中逼近最短的底层生成规则。本文通过一系列受控实验，首次系统证明了标准神经网络精确执行算法的内在潜能，展现了神经网络在理想条件下能够显露出的本质能力，并为理解神经网络的本质性质——智能、压缩与规则学习之间的关系提供实验线索，为构建高可靠、可解释的通用人工智能系统开辟了一条全新的道路。代码已开源于：<https://github.com/ball-lightning6/neural-sculpting-paradigm>。

目录

1 第一幕：现代 AI 的推理鸿沟	5
2 第二幕：神经符号计算简史	5
3 第三幕：神经雕刻范式	6
3.1 模型架构：Transformer 只是起点	6
3.2 输出范式：一次性结构化预测	7
3.3 数据范式：从现实噪声到理想法则	7
3.4 范式解读：用梯度下降雕刻精确规则	7
3.5 模型描述	7
4 第四幕：跨越多领域的实证	8
4.1 基础能力：符号规则学习	8
4.1.1 元胞自动机	8
4.1.2 学习本质探究：N 进制加法与语义洗牌	9
4.2 高级能力：算法学习与规划	11
4.2.1 案例：接雨水问题	11
4.2.2 稠密迷宫寻路	13
4.3 从图像中推理和提取符号输出	15
4.3.1 视觉符号解码：表盘角度识别	15
4.4 图像推理能力	16
4.4.1 几何推理与构造：预测三角形内切圆	16
4.4.2 几何推理与构造：平面镶嵌	21
4.5 符号推理生成图像	21
4.5.1 元胞自动机	28
4.5.2 旋转立方体	28
4.6 物理模拟	37
4.6.1 悬链线	37
4.6.2 行星轨道模拟	42
4.7 arc-agi 任务相关	42
4.7.1 水流模拟	48
4.7.2 积木游戏	54
4.8 可解释性探索	59
4.8.1 编辑过程输出	59
5 第五幕：论可学习性与可解释性的边界	60
6 第六幕：超越架构的普适性	67
6.1 元胞自动机	67
6.1.1 cnn 模型	68
6.1.2 mlp 模型	68
6.1.3 rnn 模型	68
6.1.4 unet 模型	69
6.1.5 diffusion 模型	69
6.2 接雨水	69

6.2.1	mlp 模型	69
6.2.2	rnn 模型	69
6.2.3	cnn 模型	70
6.3	总结和分析	70
7	第七幕：跨越鸿沟：从理想规则到现实世界的连续谱	70
7.1	精确规则的扰动	70
7.1.1	对输出的随机翻转测验	70
7.1.2	对输入的随机翻转测验	71
7.1.3	总结讨论	71
7.2	内插和规则学习的一体性	71
7.2.1	实验描述	71
7.2.2	实验结果	72
7.2.3	总结讨论	72
7.3	模式识别和规则学习的一体性	72
7.3.1	MNIST+ 元胞自动机：模式识别到规则执行	72
7.3.2	MNIST+ 元胞自动机：规则控制与图像变换	74
8	第八幕：运用解耦手段辅助训练——重新思考符号主义和联结主义的关系	76
8.1	2 进制数 mod 3 实验	77
8.1.1	实验部分	77
8.1.2	实验结果讨论	77
8.2	接雨水实验的再一次解耦	78
8.2.1	实验部分	78
8.2.2	实验结果讨论	79
8.3	哲学意义相关讨论	79
9	第九幕：对已经收敛的神经网络的研究	80
9.1	通过接雨水实验研究最后一个隐藏层	81
9.1.1	实验部分	81
9.1.2	实验结果讨论	81
9.2	用元胞自动机任务探测一个收敛后的 mlp 的隐藏层	81
9.2.1	实验部分	81
9.2.2	实验结果讨论	82
9.3	从最后一个隐藏层解码元胞自动机的完整演化序列	82
9.3.1	实验部分	82
9.3.2	实验结果讨论	83
10	第十幕：一些更多的探索	83
10.1	训练可视化	83
10.2	自回归	83
11	第十一幕：理论解释	83
11.1	理论假设和证据	84
11.2	相关实验证据	86
11.2.1	[27] ntk 相关实验	86

11.2.2 规则偏好实验	86
11.2.3 极端过参数化实验	87
11.2.4 规则 ood 泛化实验	87
11.3 另外一些思考	87
11.4 相关工作	88
12 第十二幕：从炼金术到科学的深度学习	89
13 参考文献	90
14 附录	93
14.1 这里列出了一些有趣的实验图片	93
14.2 这里列出所有做过的实验	97

正文部分

1 第一幕：现代 AI 的推理鸿沟

深度神经网络在图像识别、语音识别、自然语言处理和生成式 AI 等领域取得了巨大成功，但围绕其本质能力的争议从未消失。一方面，神经网络常被理解为高维统计拟合器、模式识别器或复杂函数逼近器：它们能够从海量数据中捕获相关性，却未必真正理解规则、算法或因果结构。大语言模型中的“幻觉”、逻辑不一致、数学和程序推理中的脆弱性，以及对抗样本现象，进一步强化了这种怀疑。在更尖锐的表述中，神经网络甚至被视为“随机鹦鹉”：它们看似推理，实则只是在复杂分布上进行概率性的模仿。

另一方面，神经网络的实际表现又不断超出这种简单图景。大模型的涌现能力、grokking 现象、隐式正则化，以及“压缩即智能”等观点，都提示神经网络可能并不只是逐样本记忆或表层模式拟合，而是在某些条件下倾向于发现更简单、更稳定、更可泛化的结构。问题在于，现实数据和开放任务往往同时混合了噪声、多义性、记忆、语义关联、分布偏差和评估不确定性，因此很难单纯根据大模型的成功或失败判断神经网络到底在做什么。

本文认为，真正需要回答的问题不是“神经网络是否已经可靠地解决现实推理”，而是一个更基础的问题：在足够清晰、可验证、可控的条件下，神经网络是否具有学习并执行精确规则的内生能力？如果这种能力存在，它是否只属于某些特殊架构，还是神经网络更一般的属性？它与通常意义上的模式识别是什么关系？它是否可以被理解为一种压缩过程？

因此，本文并不首先把神经雕刻范式视为一种直接改进现有 AI 系统的工程方法，而是将其作为一个探索神经网络本质性质的受控实验场。在这一实验场中，任务由程序生成，输入输出空间预先固定，样本来自目标分布的随机采样，输出采用一次性结构化预测，训练目标是学习从输入到唯一确定输出的变换关系。通过有意排除现实数据中的噪声、歧义和开放边界，本文考察标准神经网络在理想条件下是否能够学习并执行精确规则，以及这种能力与架构、模式识别和压缩之间的关系。

本文的核心发现是：在这一受控设置中，标准神经网络可以在多类确定性任务上学习接近精确的规则映射，并且这种能力很可能不是某个特定架构的偶然性质。进一步的元胞自动机插值实验和 MNIST+ 元胞自动机实验暗示，模式识别与规则执行可能并非彼此割裂的能力，而可能是同一底层能力在不同数据形式下的表现。最后，本文从压缩即智能/MDL 视角解释这些实验现象：在程序化理想数据中，神经网络的学习过程可以被理解为从大量样本中逼近更短、更稳定的底层生成规则。本文的目标不是声称已经解决现实 AI 系统中的所有推理问题，而是通过一系列可复现实验，重新审视神经网络作为联结主义系统所可能具备的本质能力。

2 第二幕：神经符号计算简史

将神经网络的学习能力与符号系统的精确推理能力结合起来，一直是人工智能研究中的长期目标。围绕这一目标，已有探索大致可以分为两类。

第一类是“架构的改造”。这类工作试图通过设计更适合符号计算或结构化推理的网络结构，来增强模型的推理能力。其代表性探索包括引入外部记忆模块的神经图灵机、可微神经计算机，以及后来的神经模块网络和图神经网络等。这些工作的共同特点，是试图通过更强的结构偏置，

使神经网络更适合处理组合关系、状态转移和算法过程。

第二类是“逻辑的嵌入”。这类工作试图将外部符号规则、逻辑约束或概率逻辑程序，以可微或可学习的形式引入神经网络训练过程。无论是通过模糊逻辑将规则转化为损失约束的神经定理证明器，还是将神经网络作为概率逻辑程序一部分的 DeepProbLog，其核心都是用某种外部符号系统来指导、约束或补充神经网络。

这些路线都非常重要，但它们通常隐含一个前提：神经网络的学习能力与符号推理能力是两种需要通过专门结构或外部规则连接起来的能力。本文关注的是另一种可能性：在不外接符号推理器、不预先嵌入人工逻辑规则、也不依赖专门推理架构的情况下，标准神经网络是否能够在理想数据条件下显露出精确规则学习能力？

从这个角度看，本文题目中“符号主义和联结主义的结合”，并不是指把一个外部符号系统机械地接到神经网络之后，也不是指用人工规则替代神经网络的学习过程。本文尝试展示的是一种更内生的结合方式：符号规则由程序化理想数据定义，而联结主义神经网络通过梯度优化，在大量确定性样本的约束下自然逼近这些规则。换句话说，符号主义并不是被外部嫁接到神经网络上，而是在理想数据和训练动力学中，从联结主义系统内部逐渐显露出来。

进一步地，本文所谓结合也不只意味着神经网络可以学习符号规则。传统叙事中，符号主义更接近规则、逻辑和算法，联结主义更接近感知、模式识别和连续表示。但后文的元胞自动机插值实验与 MNIST+ 元胞自动机实验显示，精确规则执行与模式识别可以在同一个神经网络、同一个任务和同一次训练过程中自然耦合。这暗示二者可能不是需要外部拼接的两套机制，而可能是联结主义系统在不同数据形式下显露出的同一种底层能力。

后文将通过神经雕刻范式和一系列实验集中考察这一问题，并在后续理论部分提出一个解释这些实验现象的统一假设，讨论该假设与既有实验观测和相关理论之间的关系。

3 第三幕：神经雕刻范式

本文提出的神经雕刻范式，其核心思想并非设计一种全新的模型架构，而是通过程序化生成的理想数据和合适的输入输出格式，观察并证明标准神经网络在受控条件下学习精确规则的能力。这类任务的问题形式不限于符号规则、算法问题、图像推理、物理模拟或可解释性标签。若用一句话概括，它们都可以被理解为一类有明确变换关系的问题：对应给定输入，存在一个机械可执行的生成或变换过程，产生唯一确定、可验证的输出。本文使用“符号任务”“规则学习”“图像推理”等术语，主要是为了方便讨论这些实验现象，而不是试图给出严格封闭的数学定义。

该范式大致建立在以下几个相互配合的原则之上。

3.1 模型架构：Transformer 只是起点

本文早期实验主要基于标准 Transformer 及其视觉变体 ViT、Swin Transformer，因为它们容易适配符号、图像和多模态任务。然而，后续实验表明，MLP、CNN、RNN、UNet、Diffusion 等架构同样可以在部分任务中表现出类似能力，有些任务甚至收敛更快。因此，Transformer 在本文中并没有特殊的本体地位——它只是探索初期的主要实验平台，需要说明的是，在探索后期，我已经完全采用一个朴素的 MLP 来做实验了。本文真正关注的是，在理想数据条件下，标

神经网络是否普遍具有学习精确规则的潜能。

3.2 输出范式：一次性结构化预测

本文的大多数实验采用一次性结构化输出，即模型在一次前向传播中直接预测完整结果。输出可以是固定长度向量、多标签二分类、多类别分类、结构化图像，或者附带中间过程标签的拼接结果。本文倾向于认为，因为不同前缀状态和错误累积等额外变量，自回归生成在学习精确规则时可能会增加难度，甚至可能使某些任务难以达到真正的精确执行，但自回归模型能否在专门训练和充分约束下稳定实现精确规则执行，仍然是一个开放问题。对于本文的研究目的来说，一次性结构化输出提供了更干净、更可验证的实验条件。

3.3 数据范式：从现实噪声到理想法则

神经雕刻范式最核心的变化发生在数据上。本文通过程序化脚本生成大量确定性、低噪声、输出可验证的理想数据。在这些数据中，输入和输出之间存在明确的生成关系，模型不需要从开放世界的噪声、多义性和偶然相关中猜测目标，而可以集中学习底层规则。

这并不意味着任意少量数据都足够。实验表明，数据量和模式覆盖存在阈值：样本覆盖足够时，训练损失和验证损失通常同步下降；样本量较少时，可能出现先记忆训练集、再转向泛化的 grokking 现象；样本量低于更小阈值时，模型只能过拟合训练集，验证集无法收敛。因此，程序化数据并不是少量提示，而是通过大量样本对可能函数空间施加约束。本文多数实验采用从目标输入空间随机采样的方式生成训练集和验证集，但这未必是唯一或最优的数据构造方式。

3.4 范式解读：用梯度下降雕刻精确规则

神经雕刻范式这个名称来自一个直观比喻：程序化理想数据像从不同角度给出的约束，不断排除与目标规则不一致的函数；梯度下降则在这些约束下调整神经网络，使其从高维连续参数空间中逐渐逼近一个离散、确定、可验证的规则结构。更简单地说，本文发现的范式，就是用联结主义的梯度下降去拟合一个精确规则。

这里的关键并不是神经网络“知道”输入和输出每一位的人类语义。模型面对的只是输入输出之间的约束和损失函数。即使它最终学会了加法、元胞自动机或某个算法任务，也不意味着它以人类可读的形式执行了对应步骤。它仍然只是在梯度下降中寻找一种能够压缩输入输出关系、降低损失并泛化到未见样本的参数配置。这一点也引出了后文关于压缩即智能和 MDL 的理论解释。

3.5 模型描述

本文根据输入输出形式选择不同标准神经网络。符号推理和算法推理主要使用小型 Transformer 或 MLP；图像到符号任务使用 ViT 或 Swin Transformer；图像到图像任务使用 Swin Transformer+UNet、UNet 或 Diffusion；部分符号到图像或多模态任务则将符号信息编码进图像通道，或使用 Transformer 后接图像生成模块。

对应代码包括：train_tiny_transformer.py、train_mlp.py、train_image2image.py、train_text2image.py、train_qwen2_text2image.py、train_swin_image2text.py 等。具体模型设置和数据生成脚本将在后续实验章节中分别说明。

4 第四幕：跨越多领域的实证

为了系统验证神经雕刻范式的通用性，本文在多类任务上进行了详尽的实验。除正文中详述的任务外，我还在超过 100 个任务上进行了测试。完整实验说明和数据集生成脚本详见 github 仓库<https://github.com/ball-lightning6/neural-sculpting-paradigm>

4.1 基础能力：符号规则学习

本节旨在从最根本的层面，验证本范式学习精确、复杂符号规则的核心能力。

训练集验证集划分比例：一般按照 1000step 左右的频率进行一次 eval，以便随时观察 eval loss 下降趋势，按照这个频率计算出不至于让 eval 时间过长的验证集比例，验证集占数据集比例一般在 0.01 以下，由于验证集和数据集和输入空间同分布，可以验证输出可靠的 eval loss。
关于模型训练超参数：几乎所有超参数都是不需要改变的，除了有时候提高显存使用会在一些比较简单的任务上提高 batchsize，以及一些涉及图像推理的任务训练有时候 loss 会变为 nan，这样情况会减半学习率。
模型结构变化：输出符号相关的一般最重要地是需要改变多标签二分类的标签个数。
输入空间估计：有些任务的输入空间难以直接计算，我采用类似蒙特卡洛法的方法估计，即产生任务相关的随机数（这些随机数足以确定一个样本）一个比较大的次数，根据重复样本数估算输入空间的量级。

注：下列实验很多都只列出了 eval loss，严格来说不够正式，这是由于我当时在做大量实验只记录了这个，后面对一些 case 的测试结果也进行了验证，证明测试集准确率和 eval loss 是一致的，eval loss 越低确实会对应正确率越高。我可能会在之后补充严格的实验结果。

4.1.1 元胞自动机

任务描述：模型学习预测和输出一个一维元胞自动机给定规则变换给定次数后的状态

输入格式：30 位的由“0”或者“1”组成的字符串，表示元胞自动机的初始状态

输出格式：30 个标签的多标签二分类格式，等同于 30 个 01 字符串，表示元胞自动机变换后的状态

loss：平均二元交叉熵损失

训练配置：4090 显卡

训练集生成脚本：generate_cellular_automata_1d.py

数据集：ca_rule110_layer15_30.jsonl

训练代码：train_tiny_transformer.py

训练集大小：500000

输入空间大小估计： 2 的 30 次方 $=1073741824$

训练集大小占输入空间大小的比例：0.0466%

任务相关设计思想和讨论：

1. 为了严格检验神经网络超越模式识别、学习并执行精确符号规则的能力，我选择了一维元胞自动机作为首要的实验平台。这一选择并非偶然，而是基于其独特的、能够剔除一切模糊性的理想特性。首先，CA 是一个完全由本地化、确定性规则驱动的计算系统，其行为

不依赖于任何源自现实世界的统计先验或语义信息，这使得它成为测试精确规则学习能力的理想受控环境。其次，其巨大的状态空间（对于一个宽度为 W 的二进制元胞自动机，存在 2^W 个可能的状态）使得简单记忆输入输出对难以解释验证集上的稳定收敛，从而促使我们关注模型是否学习到了规则本身。在此基础上，我特别选择了 Rule 110 作为核心测试规则。因为 Rule 110 不仅是一个展现复杂和看似混沌行为的非平凡规则，更关键的是，它已被证明是图灵完备的。这意味着，原则上，Rule 110 能够模拟任何计算过程。因此，让 Transformer 学习 Rule 110 的演化，就等同于在测试模型是否具备学习一种通用计算的潜力。成功地学习这一规则，将为“神经网络能够内化复杂算法”这一论点，提供最坚实、最令人信服的实验证据。

一些任务参数：

宽度：30

规则：110

层数：15

实验结果与分析：评估损失收敛至 0.000068。这个极低的评估损失，以及训练集大小远小于输入空间的事实，表明模型在该设置下已经稳定拟合了这一规则。

最终收敛结果：

— Step 477000 — Train Loss: 0.001766 Eval Loss: 0.000068

4.1.2 学习本质探究：N 进制加法与语义洗牌

任务描述：模型学习预测和输出两个 N 进制数的加法，同时对比语义洗牌和位置洗牌的训练结果

输入格式：16 位的字符串，前 8 位代表一个数，后 8 位代表一个数

输出格式：33 个标签的多标签二分类格式，等同于 33 个 01 字符串，用 33 位二进制数表示加法结果

loss：平均二元交叉熵损失

训练配置：4090 显卡

训练集生成脚本：generate_symbol_add_shuffle_dataset.py

数据集：

- adder_8bit_base16_train.jsonl——无语义洗牌无位置洗牌
- adder_8bit_base16_sem_shuffled_train.jsonl——有语义洗牌无位置洗牌
- adder_8bit_base16_pos_shuffled_train.jsonl——无语义洗牌有位置洗牌
- adder_8bit_base16_sem_shuffled_pos_shuffled_train.jsonl——有语义洗牌有位置洗牌

训练代码：train_tiny_transformer.py

训练集大小：500000

输入空间大小估计：16 的 16 次方 = 18446744073709551616 = 1.84e19

训练集大小占输入空间大小的比例：2.71e-14

任务相关设计思想和讨论：

1. 模型学习对两个 8 位 16 进制数进行加法运算。为了探究模型学习的本质，我设计了“语义洗牌”对照实验，包括两种洗牌，一种是用随机不同符号表示 16 进制数，一种是 16 进制数的位置排列随机选择，不再采用前 8 位后 8 位分别是两个 16 进制数的做法。这样设计实验的原因：- 语义洗牌：确定 transformer 学习的是相对于任务的符号之间的结构，和特定符号是什么无关。- 位置洗牌：确定 transformer 学习这个任务，以及绝大多数任务，不依赖于输入的符号排列方式。
2. 一些任务相关参数和细节：

进制数：16 进制

两个数的各自位数：8 无语义 shuffle 的情形下：0123456789abcde 作为 16 进制数符号表示有语义 shuffle 的情形下：任取不同的可见 ascii 字符作为 16 进制数符号表示无位置 shuffle 的情形下：输入字符串的前 8 位是一个 8 位 16 进制数，后 8 位是另一个有位置 shuffle 的情形下：在训练数据集脚本的开始，产生一个位置 shuffle 的映射，然后在整个数据集保持这个映射的一致，相应代码如下

```
if SHUFFLE_POSITIONS:
    position_map = list(range(INPUT_LEN))
    random.shuffle(position_map)
    POSITION_SHUFFLE_MAP = {
        from_idx: to_idx
        for from_idx, to_idx in enumerate(position_map)
    }
```

实验结果与分析：下表的结果有力支持了我的核心主张。在“无语义洗牌”和“有语义洗牌”两种情况下，模型最终都收敛到了极低的损失水平（<0.0001），且性能无显著差异。这表明，模型学习到的结果很难简单解释为表面的符号模式，而更接近于加法运算背后的抽象结构。

注：不同随机数据和训练的随机种子，产生的训练 loss 下降过程可能相差很多，所以不能通过单一一次对比分析训练难易程度。但是在多次尝试中，发现 4 种实验设置的收敛性是稳定存在的。

由于训练过程较短，在此列出所有 4 个实验的训练过程，以 eval loss 表示。

A：无语义 shuffle、无位置 shuffle。B：无语义 shuffle、有位置 shuffle。C：有语义 shuffle、无位置 shuffle。D：有语义 shuffle、有位置 shuffle。

step	A	B	C	D
1000	0.534105	0.505196	0.511109	0.564834
2000	0.438036	0.263462	0.446237	0.497787

step	A	B	C	D
3000	0.347195	0.001700	0.365586	0.461389
4000	0.324581	0.001392	0.179969	0.395561
5000	0.254971	0.000123	0.063128	0.328988
6000	0.129849	0.000042	0.001083	0.247512
7000	0.066946	0.000024	0.000203	0.159110
8000	0.044260		0.000123	0.149018
9000	0.032533		0.000051	0.139221
10000	0.001005			0.103275
11000	0.000325			0.080524
12000	0.000099			0.052849
13000				0.022913
14000				0.002443
15000				0.002063
16000				0.000327
17000				0.000472
18000				0.000272
19000				0.000293
20000				0.000409
21000				0.000173
22000				0.000164
23000				0.000391
24000				0.000243
25000				0.000095

4.2 高级能力：算法学习与规划

4.2.1 案例：接雨水问题

任务描述：源自 LeetCode 经典算法题，[42. 接雨水 - 力扣 \(LeetCode\)](#) [19]

42. 接雨水

困难 相关标签 相关企业 Aa

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1:



输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2:

输入: height = [4,2,0,3,2,5]

输出: 9

提示:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

通过次数 1,421,469/2.2M | 通过率 65.6%

输入格式: 30 位的 01 字符串，每连续 3 位 01 字符串，以 3 位二进制数格式代表一个柱子的高度（0-7），总共十根柱子

输出格式: 30 位的 01 字符串，每连续 3 位 01 字符串，以 3 位二进制数格式代表一个柱子的所接雨水量（0-7），总共十根柱子

loss: 平均二元交叉熵损失

训练配置: 4090 显卡

训练集生成脚本: generate_trapping_rain_water_decoupled.py

数据集: trapping_rain_water_decoupled_n10_b3_train.jsonl

训练代码: train_tiny_transformer.py

训练集大小: 300000

输入空间大小估计: 2 的 30 次方 = 1073741824 = 1.07e9

训练集大小占输入空间大小的比例: 0.0279%

任务相关设计思想和讨论:

- leetcode 是一个测试这个范式的算法能力的天然仓库。我从这里测试了很多算法题，很多题这个范式都是可以拟合的。关于接雨水这道题，它在 leetcode 上属于困难题。其解法也

不是任何简单的符号规则可以模拟的。至于拟合的原理，应该说是暂时未知的。但是我认为 transformer 并没有按人类通常的思维认为的那样，先解析格式，再执行一步步的算法步骤。神经网络不可能通过任何方法知道输入输出每一位 bit 的人类含义，也不可能知道这个任务在人类语境中的含义和结构；它面对的只是输入输出之间的约束，以及由 loss 定义的优化目标。因此，我更倾向于认为，模型在这里并不是在执行人类理解中的显式算法步骤，而是在通过梯度下降逼近一个从输入到输出的确定性映射。这也是我认为这个范式可以用——用联结主义的方式，梯度下降去逼近离散——描述的原因。

2. 关于为什么输出不是接雨水的总量，因为我之前直接把总量的二进制数作为目标训练，发现效果并不好，收敛困难。于是我采用了这种解耦的格式，得以极其迅速地收敛。在这里，我给 transformer 减少了一个求和的负担，这应该是模型迅速收敛的原因。另外，我做的另一个实验，也曾因为串行混合了较多的简单运算导致收敛困难，这是个很有趣的值得研究的现象，会在后面讨论。

一些任务参数：

柱子个数：10

表示柱子高度和接雨水两所用的二进制数位数：3

柱子高度范围/接雨水量范围：0-7

实验结果与分析：经过 16,000 步训练，模型评估损失收敛至 0.000059。这一结果表明，本范式能够有效学习需要理解全局信息（如左右最高墙）的复杂算法，并通过输出格式的解耦，实现了对问题结构的精确建模。

最终收敛结果：

— Step 16000 — Train Loss: 0.000570 Eval Loss: 0.000059

4.2.2 稠密迷宫寻路

任务描述：在一个复杂的、为人类玩家设计的稠密迷宫中，模型学习预测从任意位置出发，通往终点的最优路径的“下一步”方向（上/下/左/右）。

输入格式：169 位的由“0”或者“1”或者“s”或者“t”组成的字符串，表示迷宫的初始状态，每连续的 13 个字符表示迷宫的一行，0 表示空地，1 表示墙，s 表示起始点，t 表示目标点，s 和 t 都有且仅有一个，在 13*13 迷宫的外围默认有一圈墙。

输出格式：4 分类

loss：四分类交叉熵损失

训练配置：4090 显卡

训练集生成脚本：generate_maze_dense.py

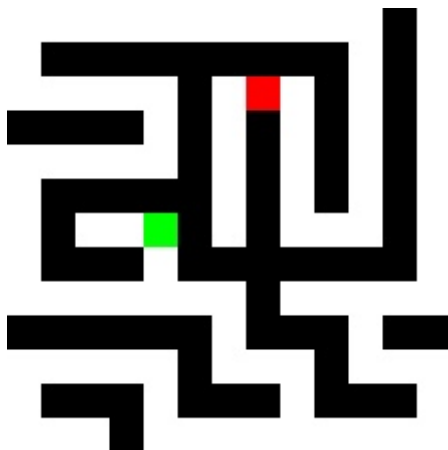
数据集：maze_optimized_13_13_dataset.jsonl

训练代码：train_tiny_transformer.py

训练集大小：2000000

输入空间大小估计：采用蒙特卡洛法估计 2000000 次无重复样本，暂且粗糙地估计为 2 的 169 次方 = 7.48e50

训练集大小占输入空间大小的比例：约 $2.673e-45$



以下图表示一个典型的迷宫，用白色代表空地，用黑色代表墙壁，用绿色代表起点，用红色代表终点。这是一个典型的适合人类玩的迷宫游戏，以区别于随机产生独立墙壁点的迷宫生成算法。任务相关设计思想和讨论：

1. 基于对于这个范式的算法拟合能力的信心，这个迷宫实验的设计一开始的算法其实相当于随机产生墙壁点。但是实践证明，这样的迷宫，对于人类来说可能非常容易，可以一眼看出最短路径（再次也是可行路径），但对于模型来说，分支数和复杂度都远超目前的迷宫实验设置。
2. 之所以选择最短路径的下一步方向，是因为这种格式设计起来比较简单，直接输出最短路径的话，无法提前知道最短路径的长度，即使采用其他措施，也使得这个任务设计变得不太优雅。另外一个重要原因是，这样的设计意味着另一种可能，迷宫任务可以被看成是在某个 state 选择最佳 action 的任务，这个可以和强化学习联系起来，这样训练 state-action 对，可以试这个范式有完成强化学习任务的潜力，见后面对 arc-agi-3 的讨论。训练也有直接监督学习最佳 action 和强化学习探索最佳 action 两种方法，但限于时间精力我并没有完成直接用强化学习探索最佳 action 的实验。

一些任务参数：

宽度：13

高度：13

实验结果与分析：评估损失收敛至 0.000014，表明模型在该固定迷宫分布下能够稳定预测最优路径的下一步方向。这说明传统搜索问题中的一部分输入到最优动作映射，可以在受控数据条件下被推销进一个前馈网络的权重中。这个极低的评估损失意味着模型已经学会走类似的迷宫，而极低的评估损失也意味着在这个最佳路径的长度尺度上，整个路径准确率也会非常高。

最终收敛结果：

— Step 279000 — Train Loss: 0.000083 Eval Loss: 0.000014

4.3 从图像中推理和提取符号输出

本节将展示该范式在处理多模态任务时的强大潜力，包括从视觉信息中解码符号

4.3.1 视觉符号解码：表盘角度识别

任务描述：图像推理，通过输入图像类似表盘的东西，输出这些不同颜色和粗细的线段对应的角度占用所有角度区间的情况

输入格式：224*224 图像

输出格式：36 个标签的多标签二分类格式，等同于 36 个 01 字符串，表示各个区间是否有被线段占用，36 个区间平分 360 度角，每个区间 10 度角

loss：平均二元交叉熵损失

训练配置：4090 显卡

训练集生成脚本：generate_line_angle_to_vector.py

数据集：line_angle 文件夹

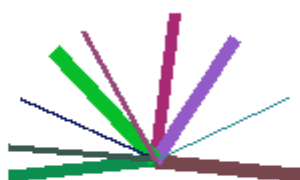
训练代码：train_swin_image2text.py

使用模型：Swin Transformer

训练集大小：200000

输入空间大小估计：389329651

训练集大小占输入空间大小的比例：0.0514%



任务相关设计思想和讨论：

1. 算法拟合能力得到证明后，我联想到 transformer 在图像中的应用，于是想测试图像 transformer 有没有类似的推理能力。于是用 ViT 测试了棋盘格相关的任务，效果很好。再之后就设计了这个任务，初衷是想测试纯粹的图像理解，而非简单识别图像中的符号再应用 transformer 已经被证明的符号规则学习能力和算法拟合能力。
2. 这个任务一开始采用了 ViT 模型，效果不好，于是后面换成了现在所使用的 Swin Transformer。

3. 为了增加多样性，线段的颜色和粗细加入了随机性

一些任务参数：

图像分辨率：224*224

区间个数：36

每个区间角度：10

实验结果与分析：评估损失收敛至 0.019183。这个 loss 并不算完美收敛，但表明模型已经能够从原始像素中提取相当稳定的抽象的、符号化的角度区间信息。

最终收敛结果：

— Step 24000 — Train Loss: 0.016392 Eval Loss: 0.012753

4.4 图像推理能力

本节将展示该范式纯粹的图像推理能力，以和符号规则学习能力和算法拟合能力区分。

4.4.1 几何推理与构造：预测三角形内切圆

任务描述：模型的输入是一张随机生成的绿色三角形图像，任务是生成其在数学上唯一的、正确的红色内切圆图像。

输入格式：224*224 图像

输出格式：224*224 图像

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_triangle_to_incircle.py

数据集：incircle_dataset 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：150000

输入空间大小估计：采用蒙特卡洛法估计约 $2.8e+12$

训练集大小占输入空间大小的比例：约 $5.4e-8$

任务相关设计思想和讨论：

1. 为了进一步检验 transformer 纯粹的图像推理能力，把它和符号规则学习能力和算法拟合能力分开。我设计了这个内切圆实验。理论上，人类可以把这个任务拆解为：先识别三个点的位置，再用几何公式计算内切圆，最后画出内切圆。但神经网络不可能知道这些人类定义的几何步骤，也不可能被自然地解释为在按这种显式算法执行。因此，这个任务真正检验的是：图像模型能否在没有显式几何程序的情况下，从大量样本中直接学会从三角形图像到内切圆图像的确定性映射。鉴于图像的模拟性质，这种推理能力很有向更大的领域扩展的潜能，比如纯粹的实验观测数据之类，但是很容易想到的是，鉴于真实的实验观测数据一定是有噪声的，又会退化到之前的模式识别深度学习范式，这个问题会在后面讨论。

2. 这个任务和后面相关的系列任务最有意思的地方，是训练过程以非常直观的形式展示了模型输出如何逐步逼近目标规则。某些阶段性的输出变化，很容易让人联想到一种“机器心智”正在形成的过程：它不是一步到位地学会规则，而是在训练中逐渐排除错误结构、保留更接近目标的结构。训练过程的可视化以 eval image 形式展现。

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该设置下已经能够稳定生成接近目标的内切圆图像，这说明图像模型确实可以通过训练掌握这类结构化几何映射。在这个例子中，单独看结果不能完全排除内插解释，但后面更复杂的图像规则任务会进一步削弱这种解释。同时，训练过程提供了观察模型如何逐步形成几何输出结构的直观窗口。

实验过程展示：

左边是输入图像，右边是 ground truth，中间是生成图像。

200 step

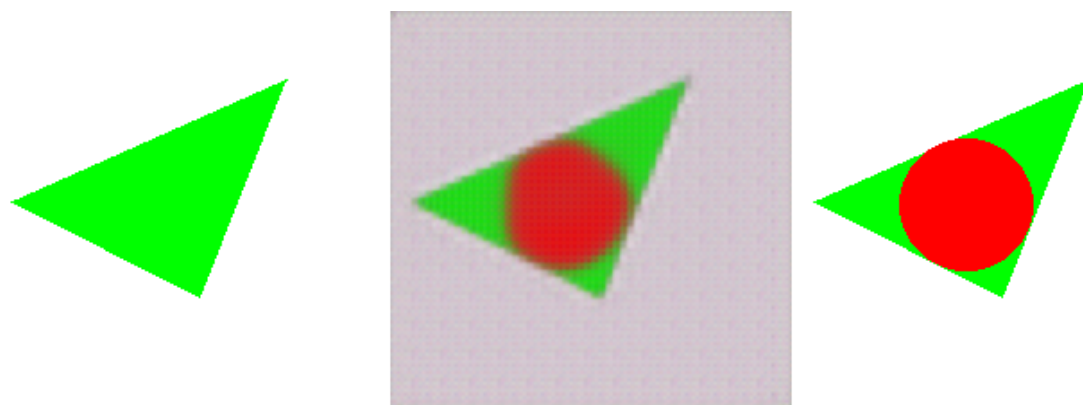


图 1: 三角形内切圆实验：200step 训练过程可视化，左为输入图像，中为模型生成图像，右为 ground truth

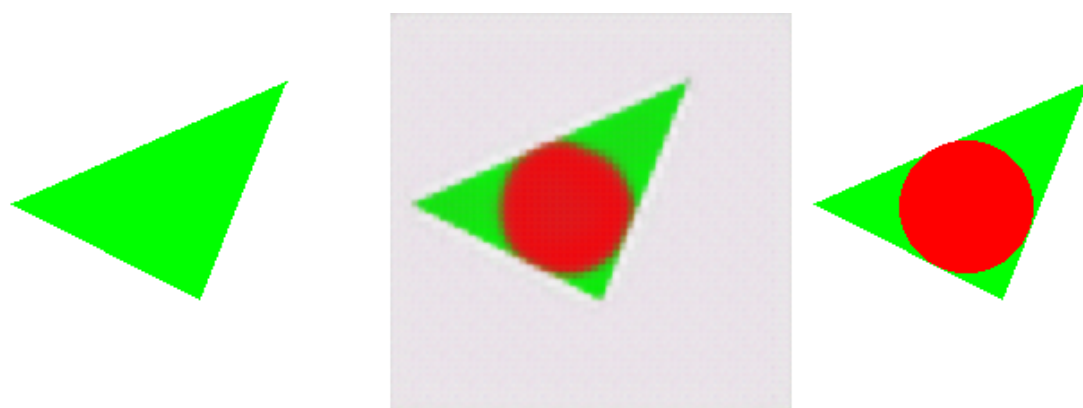


图 2: 400step

后面由于变化速度减缓，直接展示最后训练结果

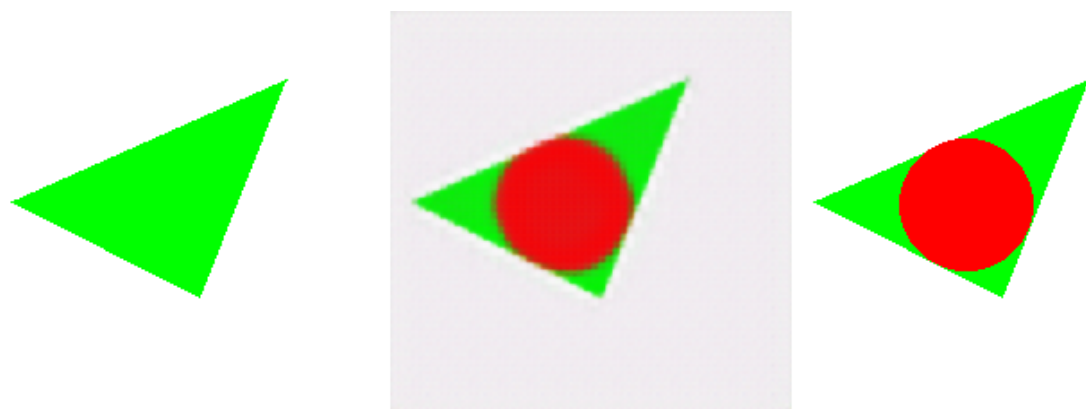


图 3: 600step

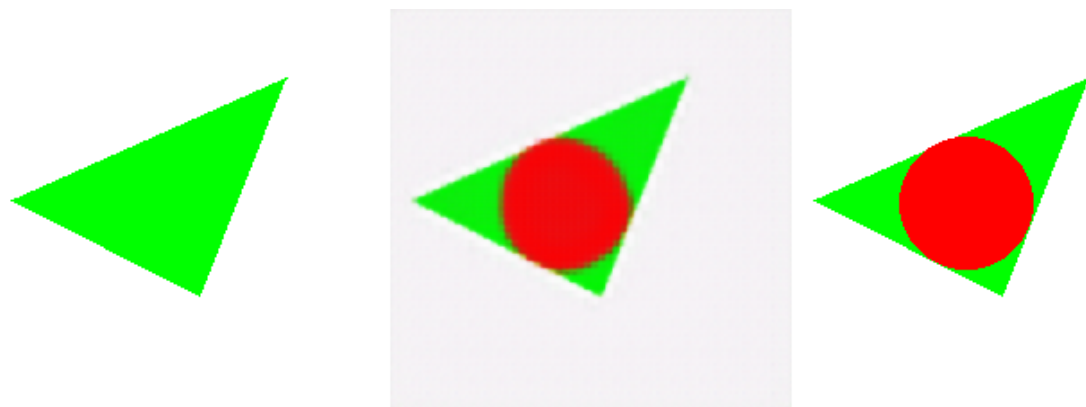


图 4: 800step

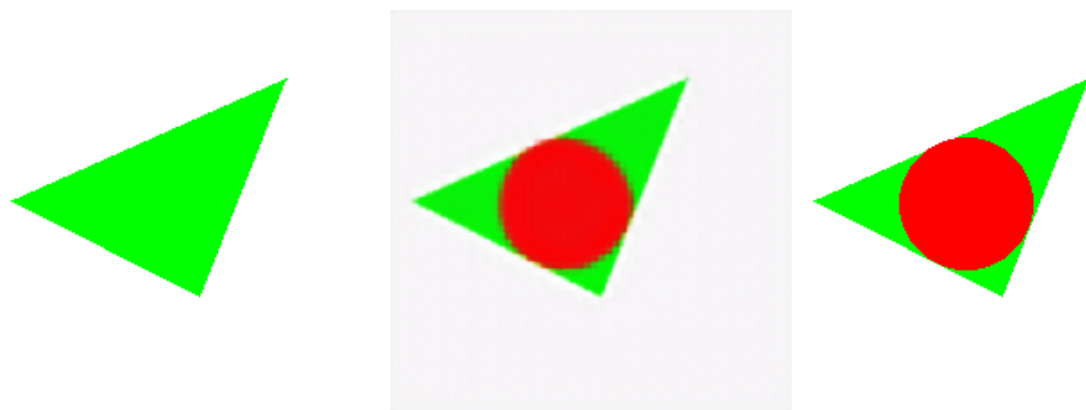


图 5: 1000step

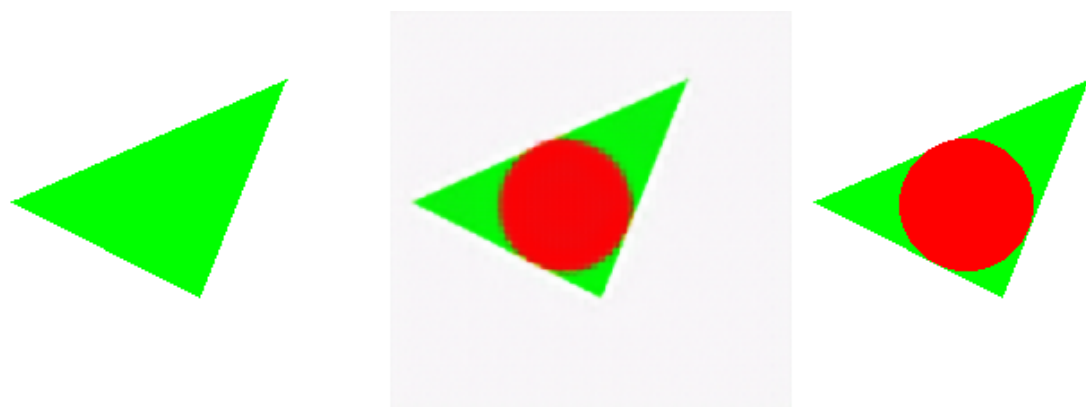


图 6: 1200step

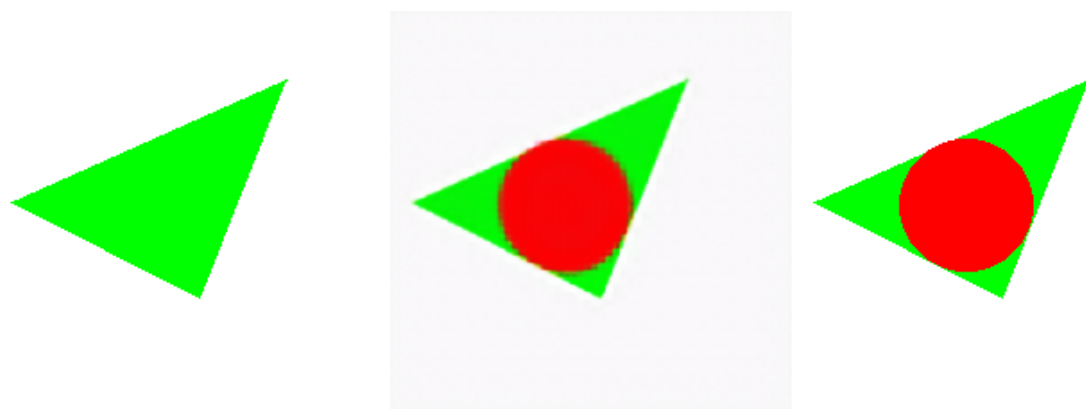


图 7: 1400step

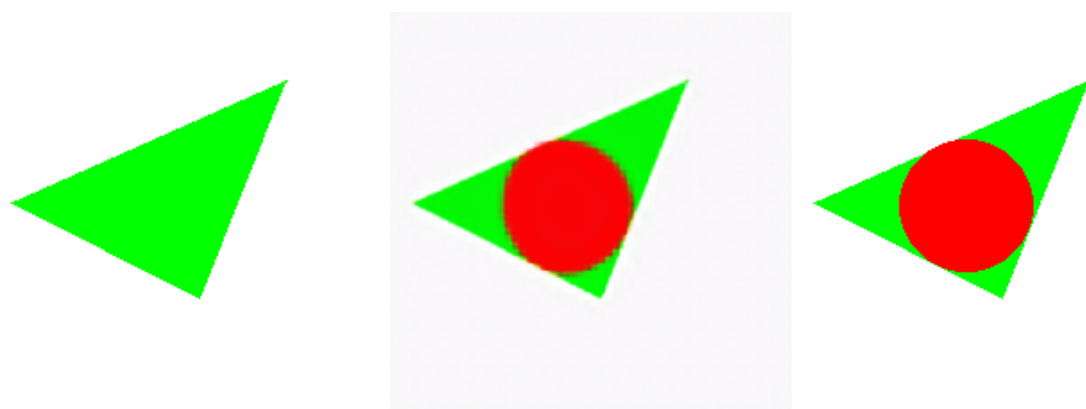


图 8: 1600step

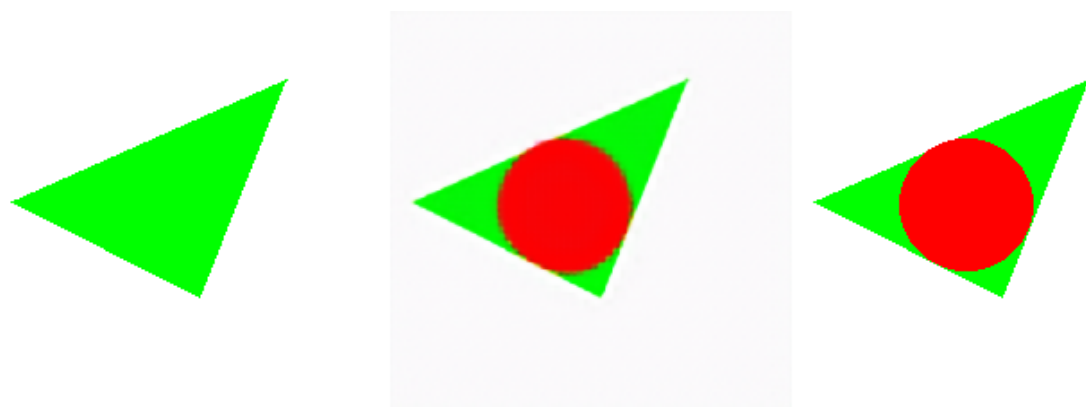


图 9: 1800step

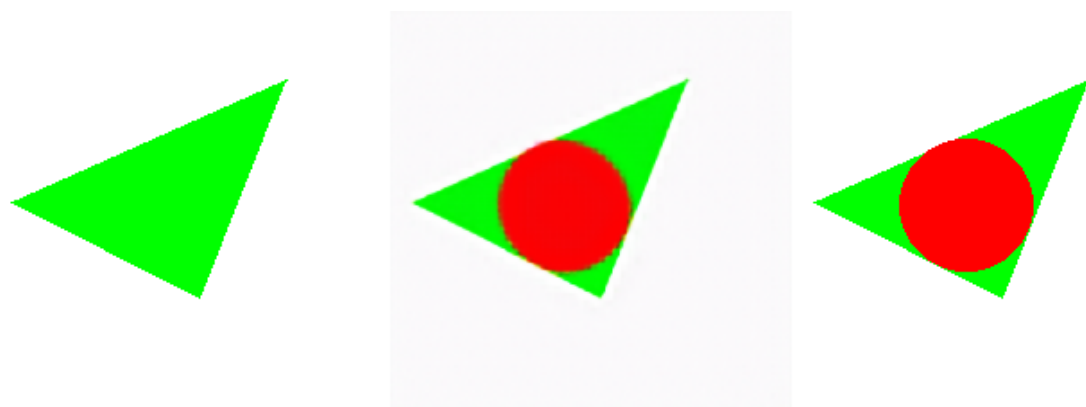


图 10: 2000step

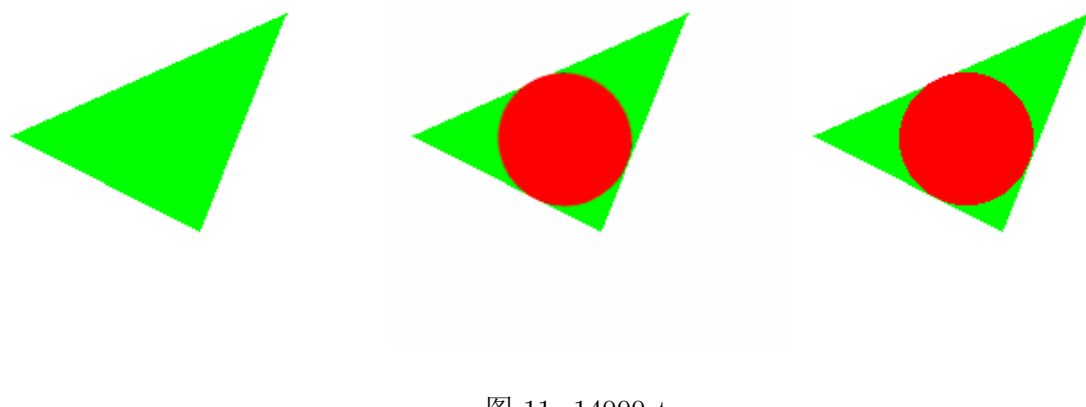


图 11: 14000step

4.4.2 几何推理与构造：平面镶嵌

任务描述：模型的输入是一张随机生成的绿色三角形图像，输出图像是绿色三角形的平面镶嵌，绿色三角形和红色三角形边相邻接和交替

输入格式：224*224 图像，包含一张随机生成的绿色三角形图像

输出格式：224*224 图像，为输入图像的平面镶嵌结果，绿色三角形和红色三角形边相邻接和交替

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_triangle_to_tessellation.py

数据集：tessellation_dataset_224 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：150000

输入空间大小估计：采用蒙特卡洛法估计约 $2.71e10$

训练集大小占输入空间大小的比例：约 $5.5e-6$

任务相关设计思想和讨论：

1. 这个任务很难用朴素内插解释。原因是平面镶嵌要求模型从输入三角形出发，在远离原始三角形的位置持续生成全局一致的重复结构；如果模型只是对训练样本做局部插值，误差会随着距离扩展迅速积累，难以维持远处图案的规则一致性。
2. 为了确保任务难度适中，对输入绿色三角形的形状和大小进行了限制，详见训练集生成脚本代码

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该设置下已经学会生成符合规则的平面镶嵌图像。这说明图像模型不仅可以拟合局部几何关系，也可以学习需要长程结构一致性的图像变换规则。这个实验也进一步削弱了“模型只是做朴素内插”的解释。训练过程显示，模型输出先在输入三角形附近形成正确结构，随后逐步向外扩展，这提供了一个非常直观的学习过程观察窗口。

实验过程展示：

左边是输入图像，右边是 ground truth，中间是生成图像。

后面由于变化速度减缓，直接展示最后训练结果

4.5 符号推理生成图像

本节将展示该范式在处理符号输入，学习对应规则，生成对应图像的能力。

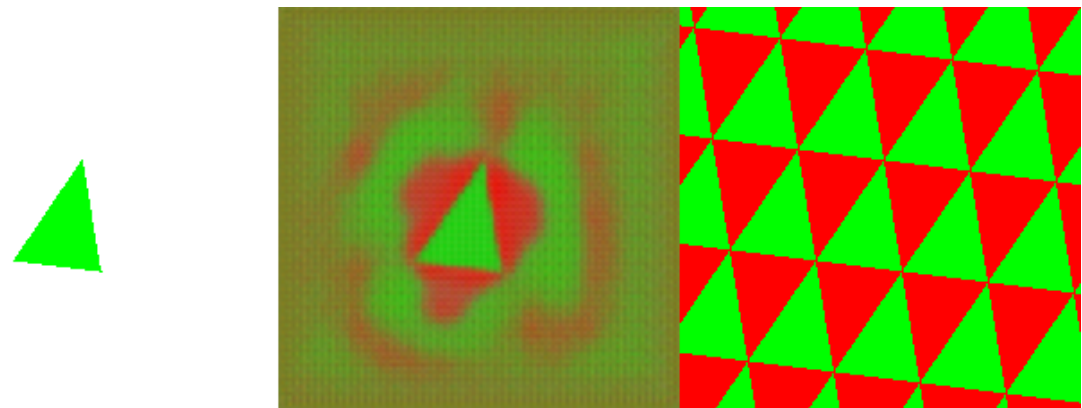


图 12: 200step

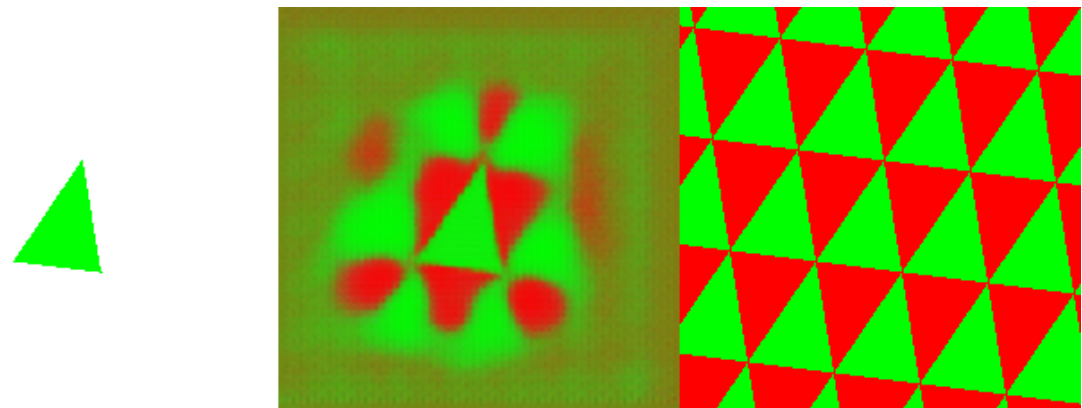


图 13: 400step

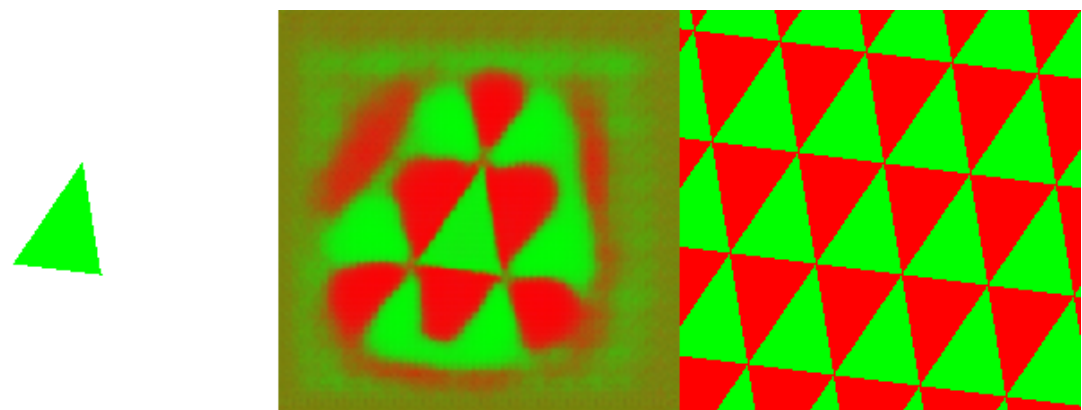


图 14: 600step

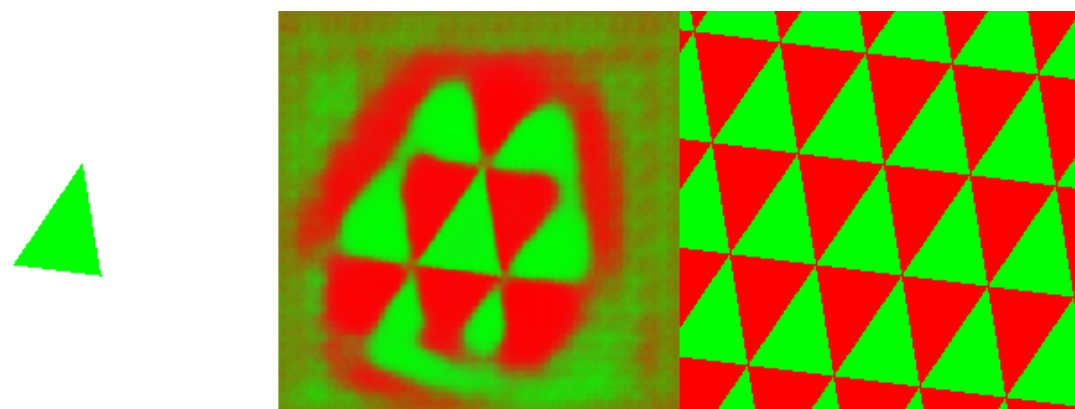


图 15: 800step

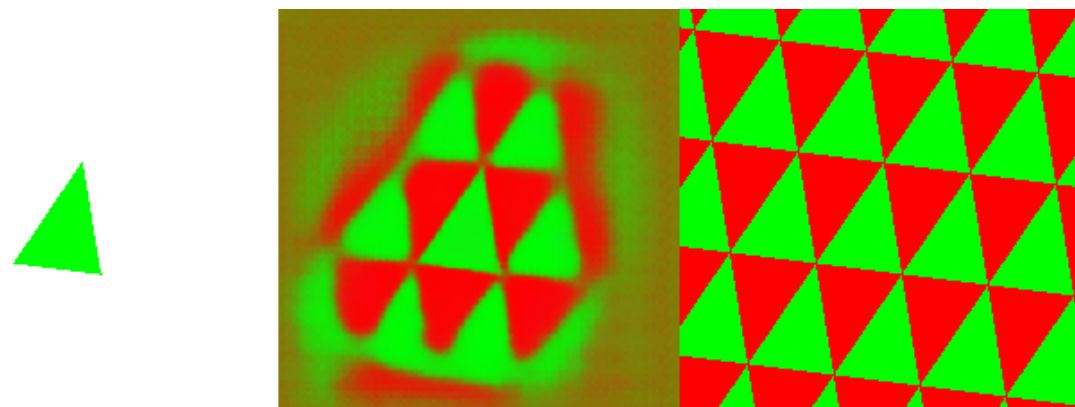


图 16: 1000step

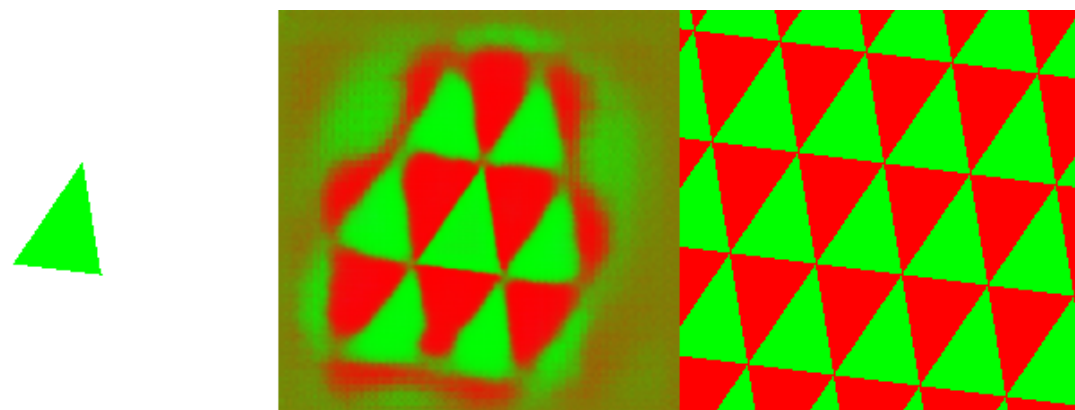


图 17: 1200step

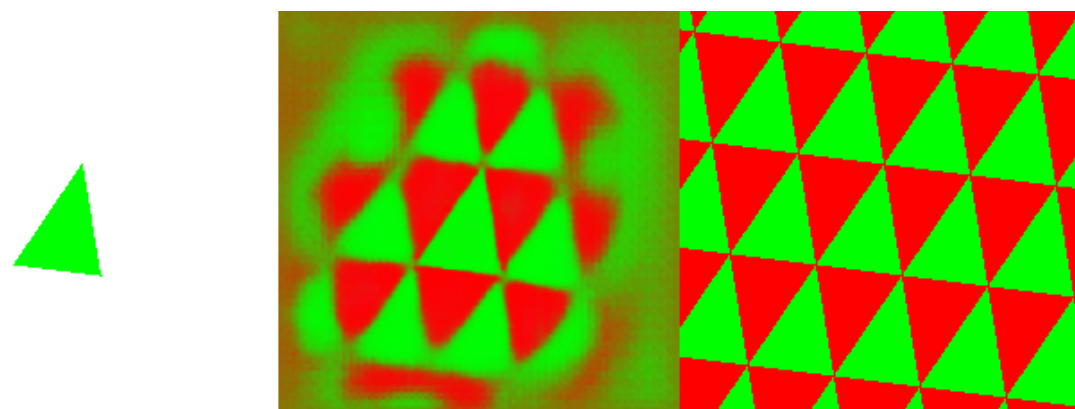


图 18: 1400step

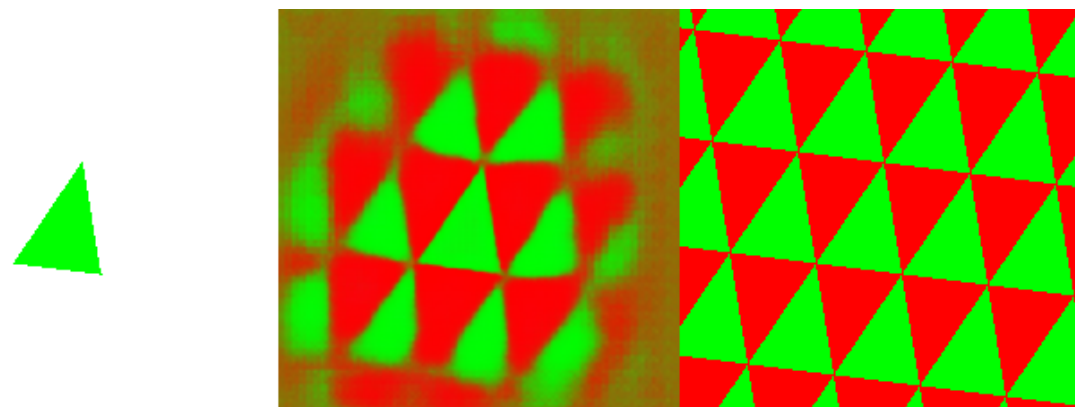


图 19: 1600step

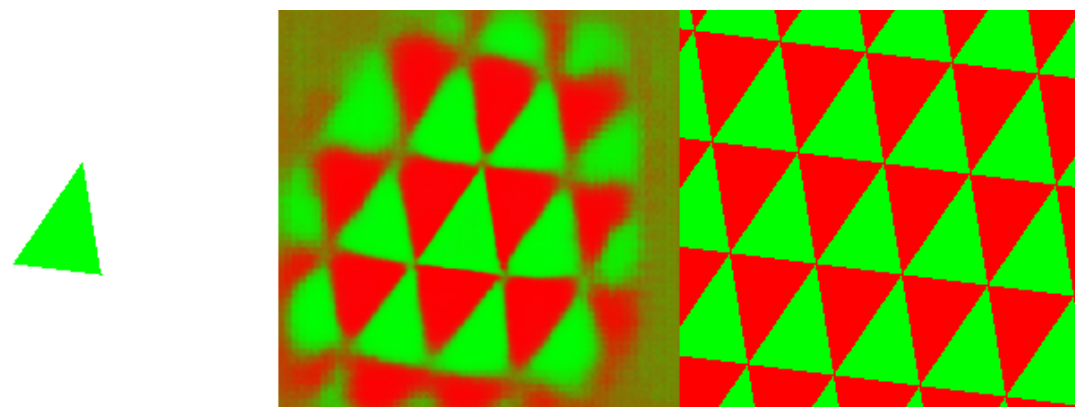


图 20: 1800step

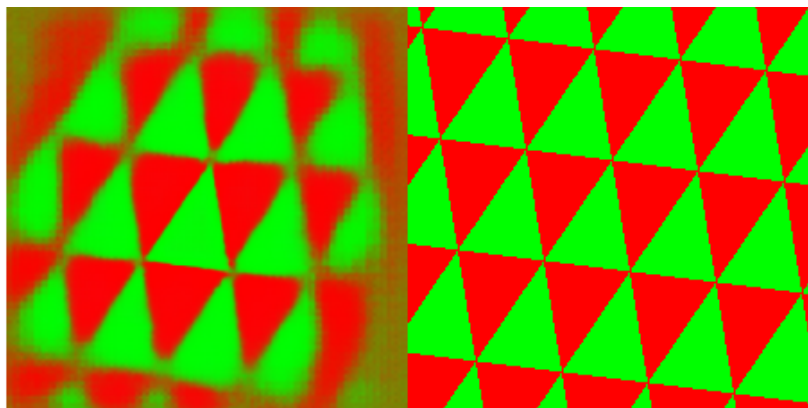


图 21: 2000step

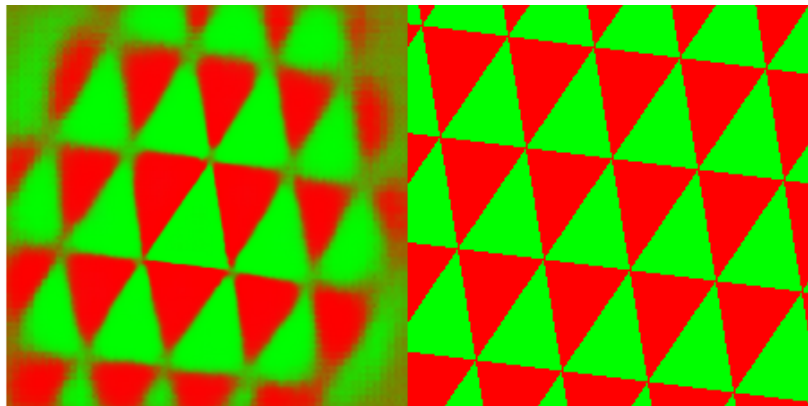


图 22: 3000step

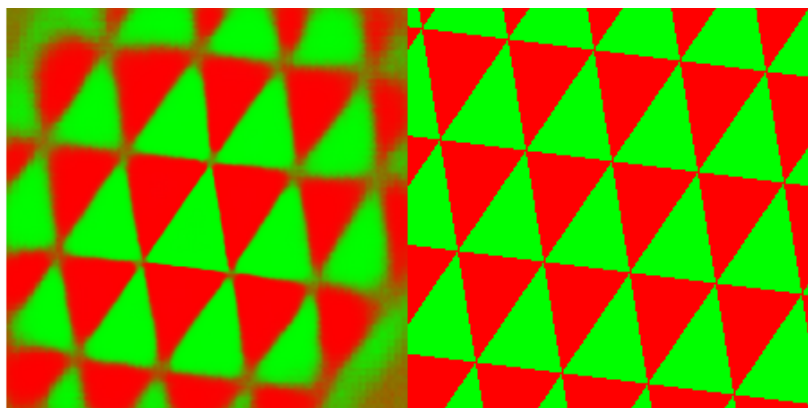


图 23: 4000step

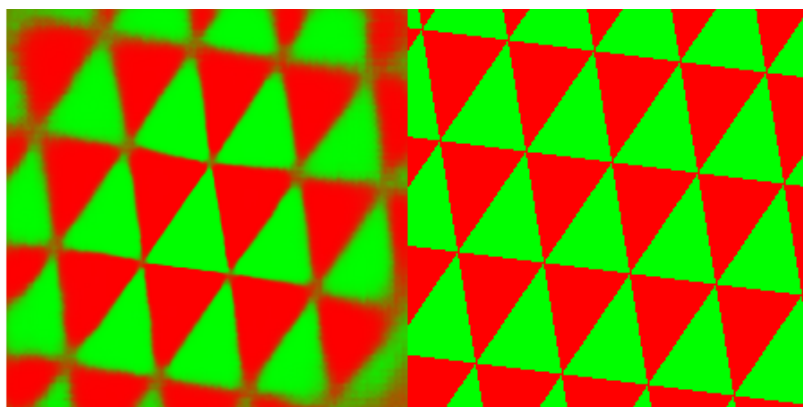


图 24: 5000step

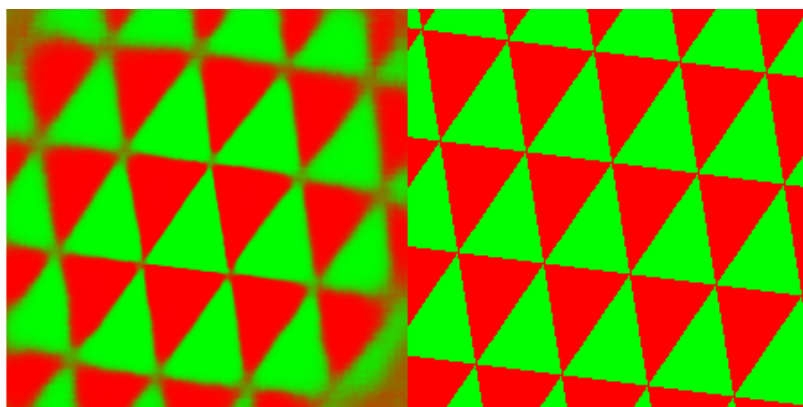


图 25: 6000step

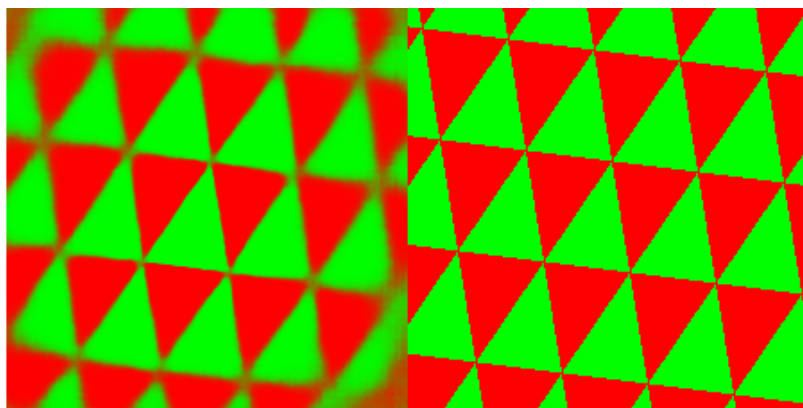


图 26: 7000step

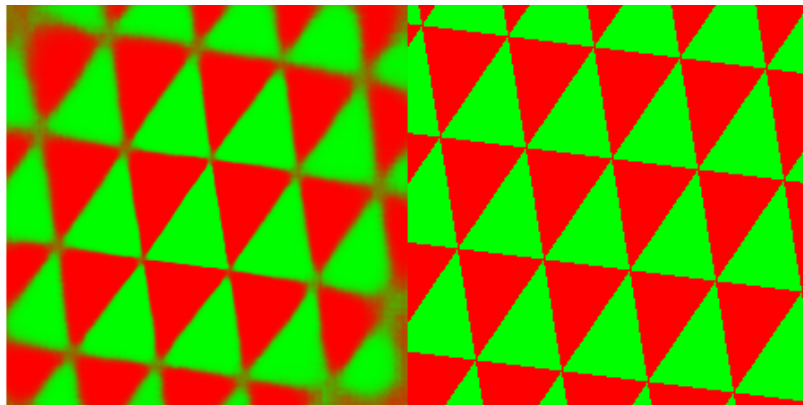


图 27: 8000step

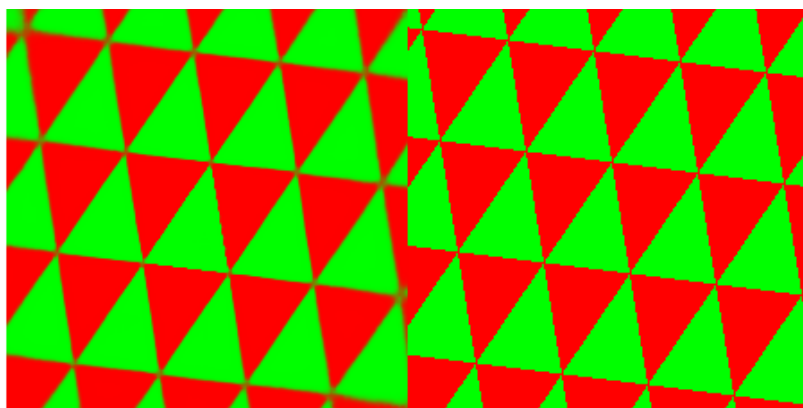


图 28: 150000step

4.5.1 元胞自动机

任务描述：模型学习预测和输出一个一维元胞自动机给定规则变换给定次数后的状态

输入格式：36 位的由“0”或者“1”组成的字符串，表示元胞自动机的初始状态

输出格式：240*240 图像

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_cellular_automata_1d_to_grid_image.py

数据集：ca_render_dataset_240 文件夹

训练代码：train_qwen2_text2image.py

使用模型：transformer+unet

训练集大小：300000

输入空间大小估计：2 的 36 次方 =68719476736

训练集大小占输入空间大小的比例：4.4e-6

任务相关设计思想和讨论：

1. 这个任务是为了检验 transformer 符号生成图像的推理能力，这个模式并非传统意义上的文生图，而是重点检验它的推理能力。
2. 输出图像中，把一维 36 位的元胞自动机状态以行优先排列成 6*6 的棋盘格图像，黑色代表 1，白色代表 0

一些任务参数：

格点维度：6*6

格点大小：40*40 像素

图像分辨率：240*240

元胞自动机规则：rule 110

规则演化层数：3

实验结果与分析：最终的训练结果表明，模型在图像输出形式中仍然能够稳定拟合元胞自动机规律。这说明，当输出从符号序列变为结构化图像时，模型仍然可以学习背后的离散规则。更重要的是，模型的学习速度非常快，训练过程也提供了一个直接观察“机器心智”形成过程的窗口：它并不是按照人类想象中的显式逻辑步骤，一步步构造最终图像，而是在梯度下降中以联结主义的方式逐渐逼近离散规则。这正是本文反复强调的核心现象之一。

左边是 ground truth，右边是模型生成图像。

4.5.2 旋转立方体

任务描述：模型学习预测和输出根据旋转角，生成一个彩色六方体旋转后的图像

输入格式：24 位的由“0”或者“1”组成的字符串，表示 3 个旋转角，连续 8 位二进制数表示一个旋转角

输出格式：256*256 图像



图 29: 100 step 符号输入: 111110011010001000111010000110



图 30: 200 step 符号输入: 111110011010001000111010000110

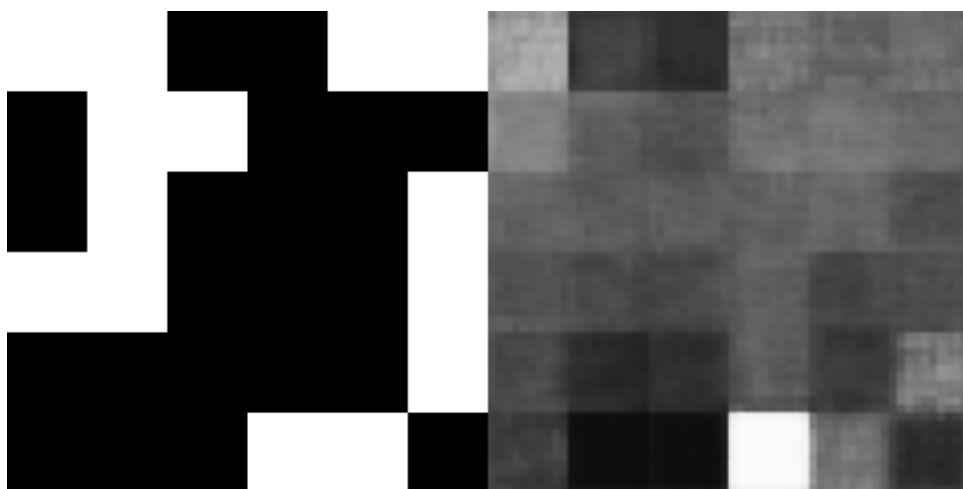


图 31: 300 step 符号输入: 111110011010001000111010000110



图 32: 400 step 符号输入: 111110011010001000111010000110



图 33: 500 step 符号输入: 111110011010001000111010000110



图 34: 600 step 符号输入: 111110011010001000111010000110



图 35: 700 step 符号输入: 111110011010001000111010000110



图 36: 800 step 符号输入: 111110011010001000111010000110

loss: MSELoss

训练配置: 4090 显卡

训练集生成脚本: generate_cube_rotation_pillow_with_anchor.py

数据集: cube_dataset_final_highlight 文件夹

训练代码: train_qwen2_text2image.py

使用模型: transformer+unet

训练集大小: 300000

输入空间大小估计: 16777216

训练集大小占输入空间大小的比例: 1.79%

任务相关设计思想和讨论:

1. 在这个任务中遇到了一个特殊的困难, 即模型很难把旋转角对应的顶点和符号输入联系起来, 导致无法收敛。解决方法是, 永远在生成训练集图片的最后步骤高亮一个特殊的顶点, 不考虑遮挡关系。采取这样的解决方法后, 模型最终得以在这个任务上最终收敛。在另外一个任务上, 我用符号指示三角形的位置, 输出平面镶嵌结果, 模型遇到了同样的困难, 它无法了解该把符号对应到输出图像的哪一个绿色三角形。解决方法类似, 我把符号输入指示的三角形在输出图像中的颜色改为了黑色, 一切就正常了。这应该也是比较典型的问题, 即模型很难确定如何把符号输入对应输出图像的哪一个特征, 解决方案应该也是类似的。
2. 关于每个面上的白色十字线段, 只是为了美观, 可能也有一些辅助训练的作用。
3. 符号输入的 3 个旋转角, 即 pitch, yaw, roll, 具体对应请参考训练集生成代码。
4. 其实我有另一个版本的模型, 采用的不是 lora 微调 qwen2-0.5b 的方法, 而是以一个比 qwen2-0.5b 的总参数更少, 但比 qwen2-0.5b 的 lora 参数更多的一个 transformer, 实践证明效果不如 qwen2-0.5b 的学习效果, 本文认为这是一个有意思的现象。

一些任务参数:

图像分辨率: 256*256

实验结果与分析: 最终的训练结果表明, 模型在该设置下能够稳定完成这个符号到图像的生成任务。这说明, 在合适的数据构造下, 符号条件和图像生成也可以被耦合到同一训练任务中。鉴于该实验使用的是 qwen2-0.5b 的 LoRA 微调, 这也说明类似能力不局限于 Swin Transformer, ViT 等图像 transformer。模型的学习速度并不是很快, 可能也是因为任务相对较难, 提供了直接观察机器心智的形成过程, 在这个任务上可以看到, 一开始模型还没有理解旋转角, 只是简单学会了生成圆形和较为平均的填充颜色, 在后面模型慢慢理解了旋转立方体的概念, 再后面模型慢慢填充精确的细节。

左边是 ground truth, 右边是模型生成图像。

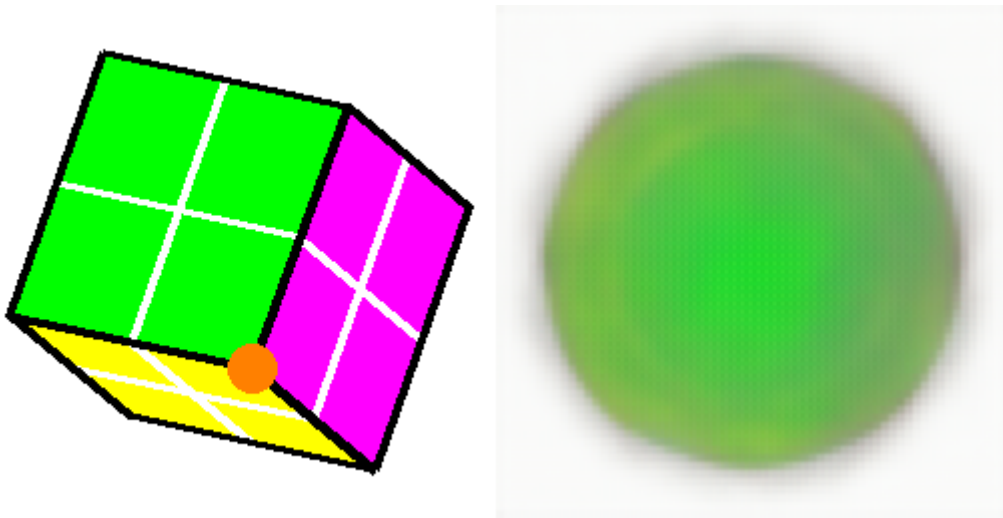


图 37: step 500 符号输入: 100101101000100100110010

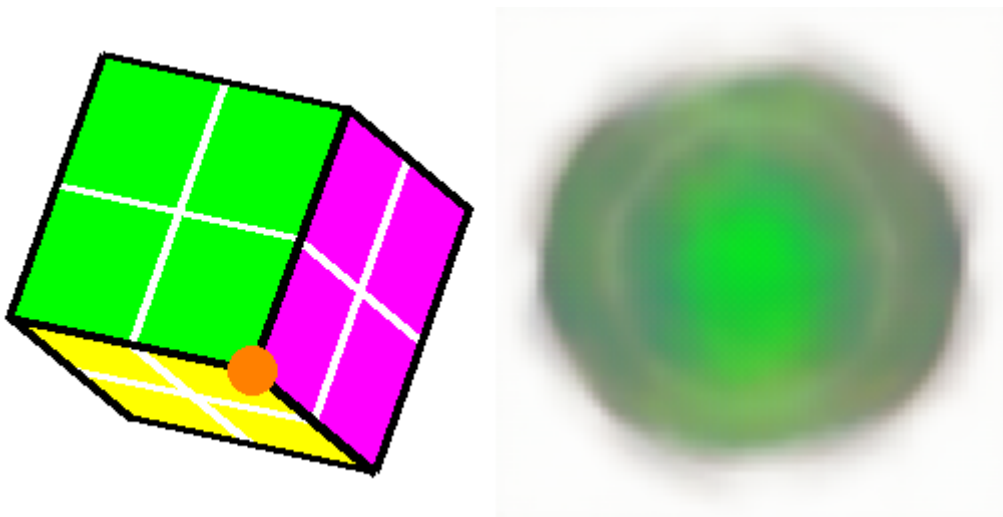
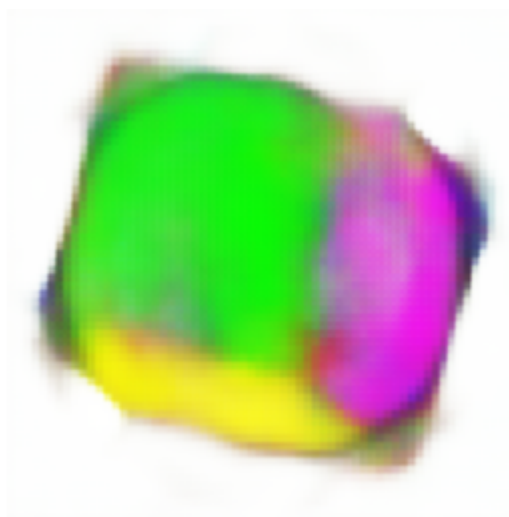
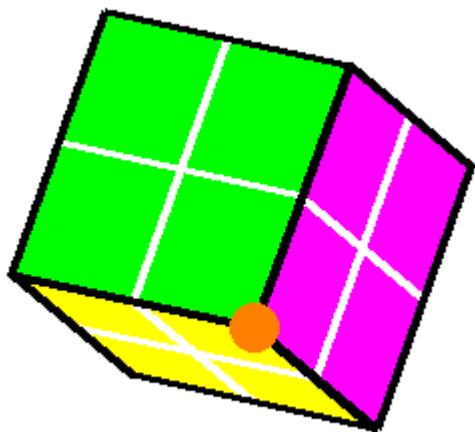
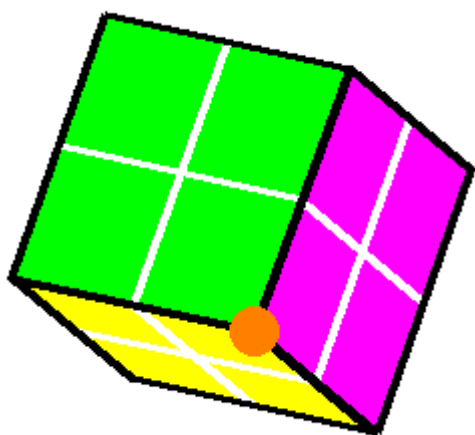


图 38: step 1000 符号输入: 100101101000100100110010

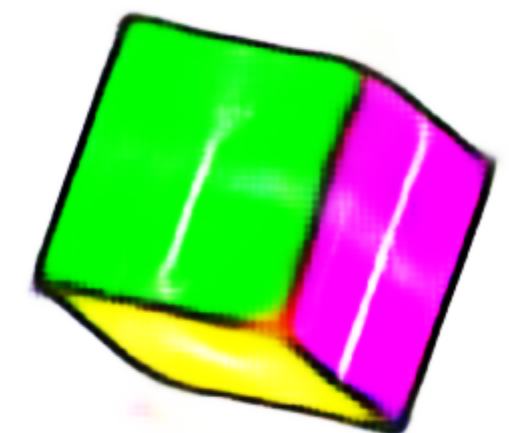
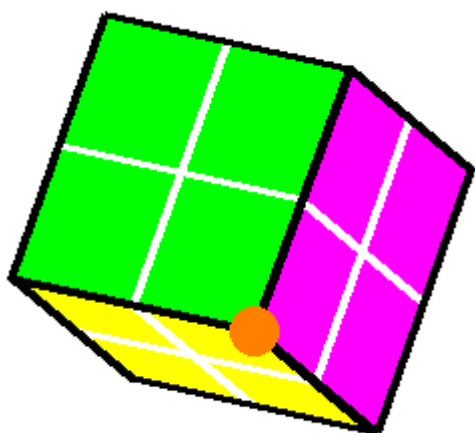


step 1500 符号输入: 100101101000100100110010 step 2000



符号输入: 100101101000100100110010 step 2500

符号输入: 100101101000100100110010



step 8000 符号输入: 100101101000100100110010 后面由于变化速度减缓, 直接展示最后训练结果

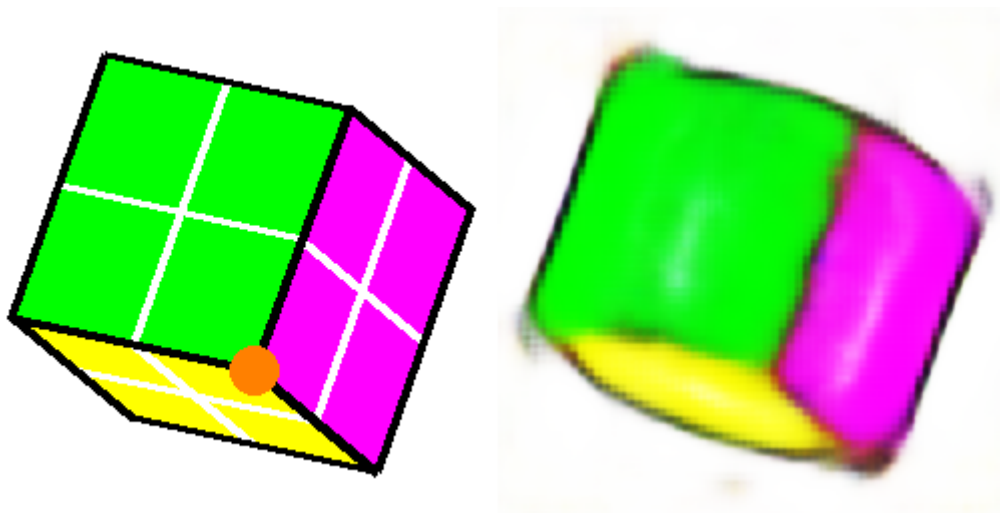


图 39: step 2500 符号输入: 100101101000100100110010

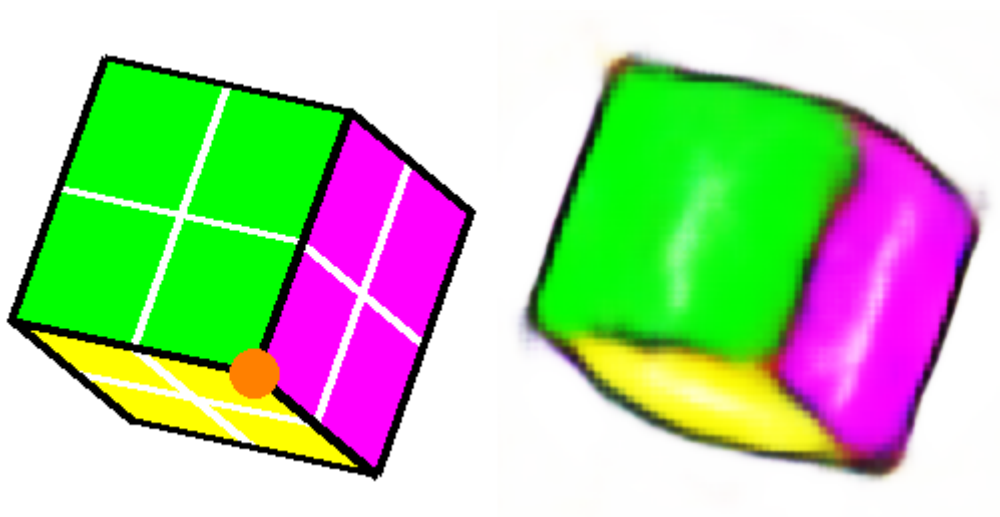


图 40: step 3000 符号输入: 100101101000100100110010

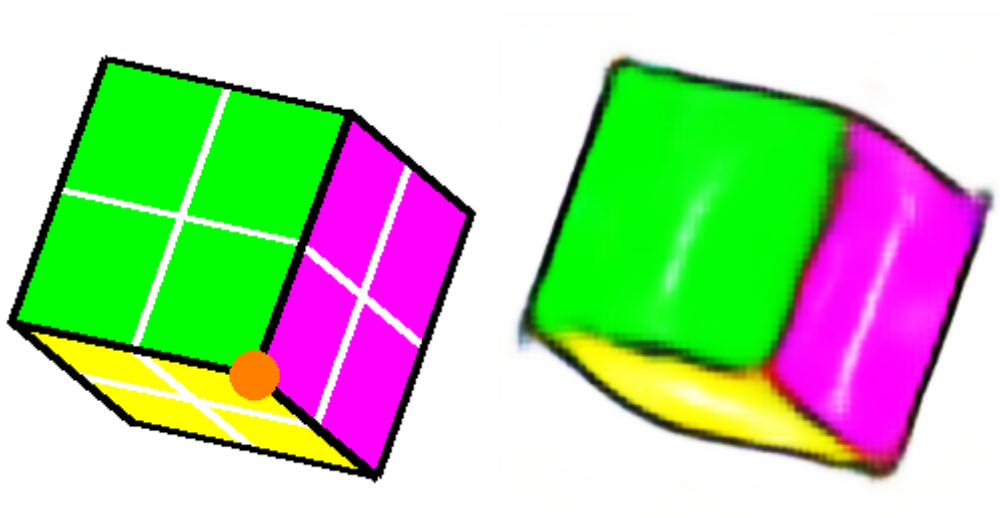


图 41: step 4000 符号输入: 100101101000100100110010

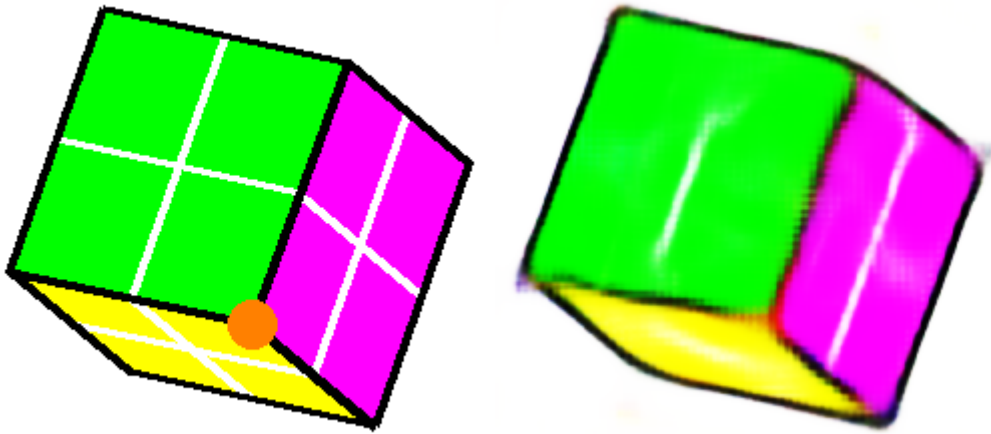


图 42: step 6000 符号输入: 100101101000100100110010

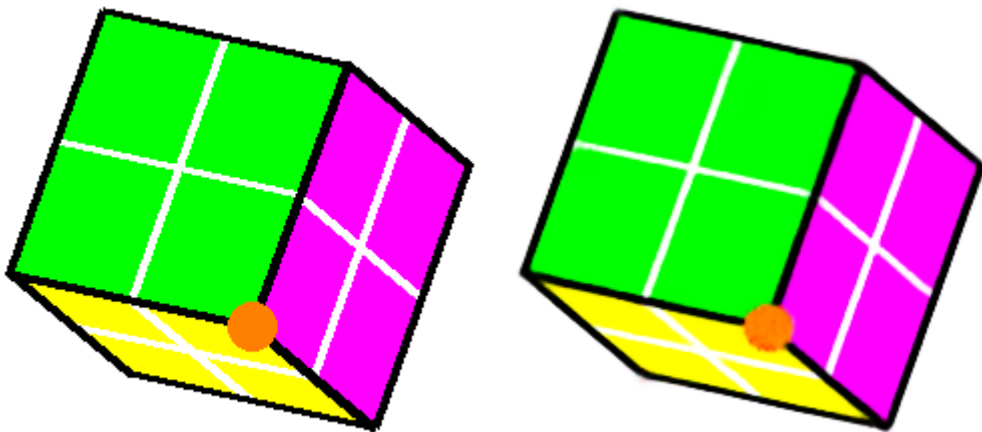


图 43: step 133600 符号输入: 100101101000100100110010

4.6 物理模拟

本节将展示该范式在模拟物理过程中的能力。

4.6.1 悬链线

任务描述：给定悬链线两端的端点，和悬链线上的某一个给定的经过点，模型学习绘制出由物理定律（最小势能原理）所唯一确定的悬链线轨迹。

输入格式：224*224 图像

输出格式：224*224 图像

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_catenary_curve_from_points.py

数据集：catenary_dataset_v5_CONSTRUCTIVE 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：150000

输入空间大小估计：采用蒙特卡洛法估计约 $9.3e9$

训练集大小占输入空间大小的比例：约 $1.61e-5$

任务相关设计思想和讨论：

1. 这个任务是为了检验这个范式的物理相关的模拟能力，通过图像推理模型实现。
2. 虽然这个问题存在通过内插近似解决的可能性，但同一类模型和超参数配置也能够处理一些难以用朴素内插解释的任务，因此这里更倾向于把它视为对程序化物理规则映射的拟合。
3. 之所以加入悬链线经过点这个限制，主要是因为设置悬链线为固定长度或者通过某种方式表示悬链线长度的方式感觉都不如这种设置更好。然后加入悬链线经过点这个限制，也增加了输入空间，进一步排除记忆或内插的可能性。

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该设置下能够生成接近目标的悬链线图像。这说明该实验设置可以从离散规则任务扩展到程序化物理规律生成的图像任务，但这种模拟的精度和泛化边界仍需要进一步探究。训练过程也显示，模型并不是简单地逐点复制训练图像，而是在逐步形成符合悬链线结构的输出。同时注意到我产生的训练集是有瑕疵的，输出悬链线图像有毛边，而模型收敛后的输出却是相对平滑的，这个现象值得进一步研究。

左边是输入图像，中间是模型生成图像，右边是 ground truth。

后面由于变化速度减缓，直接展示最后训练结果



图 44: 200step



图 45: 400step

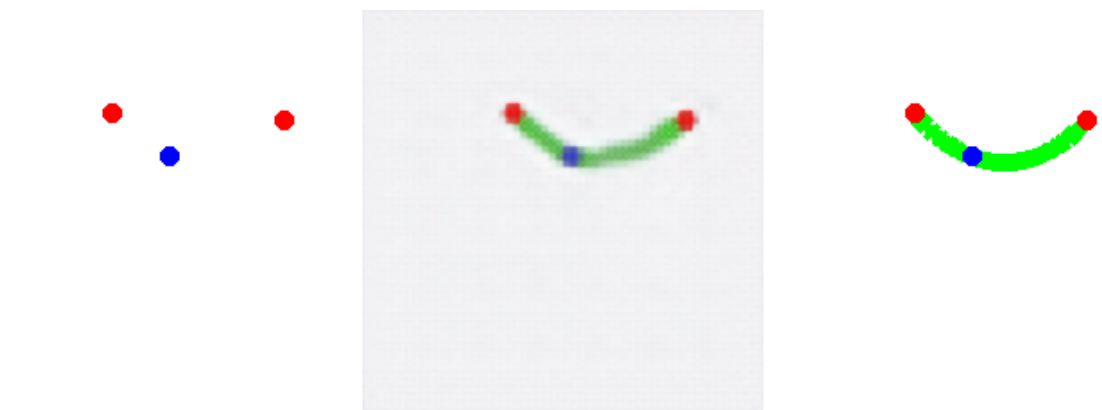


图 46: 600step

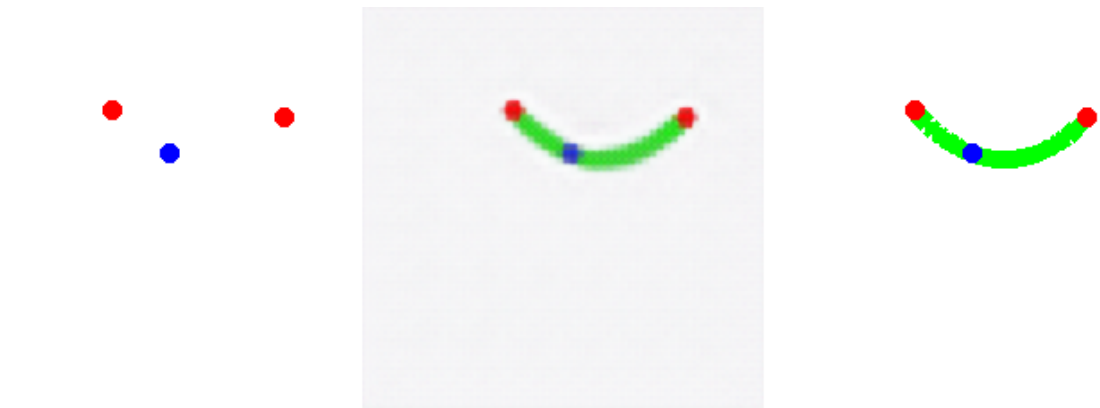


图 47: 800step

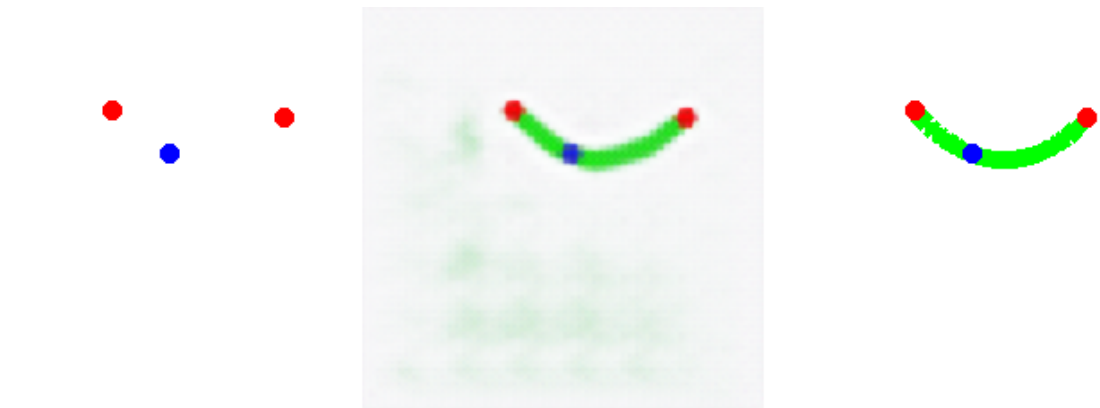


图 48: 1000step

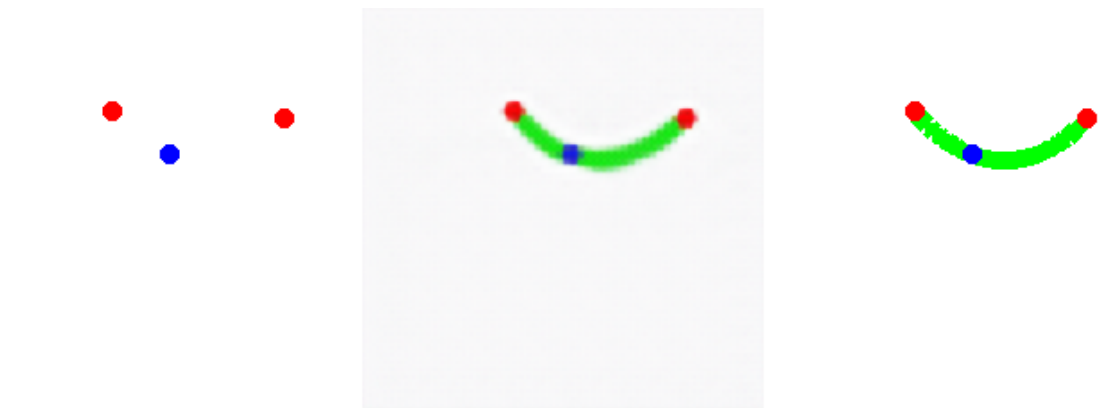


图 49: 1200step

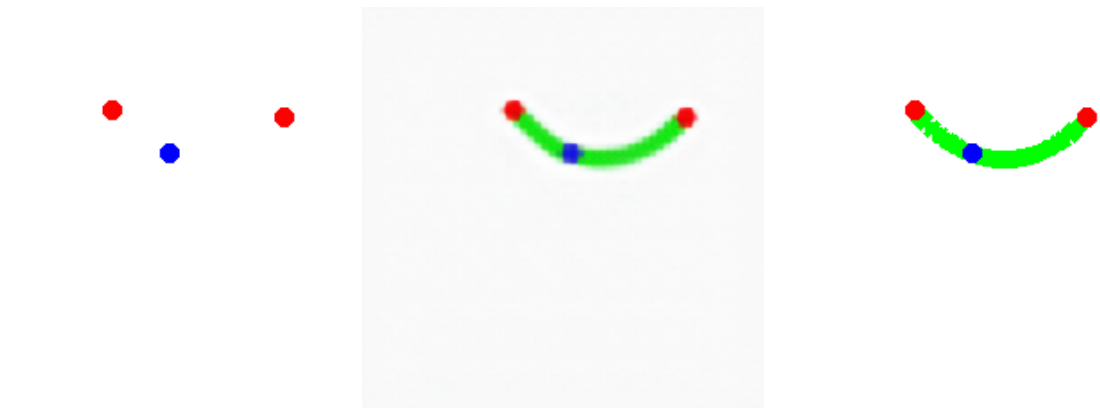


图 50: 1400step

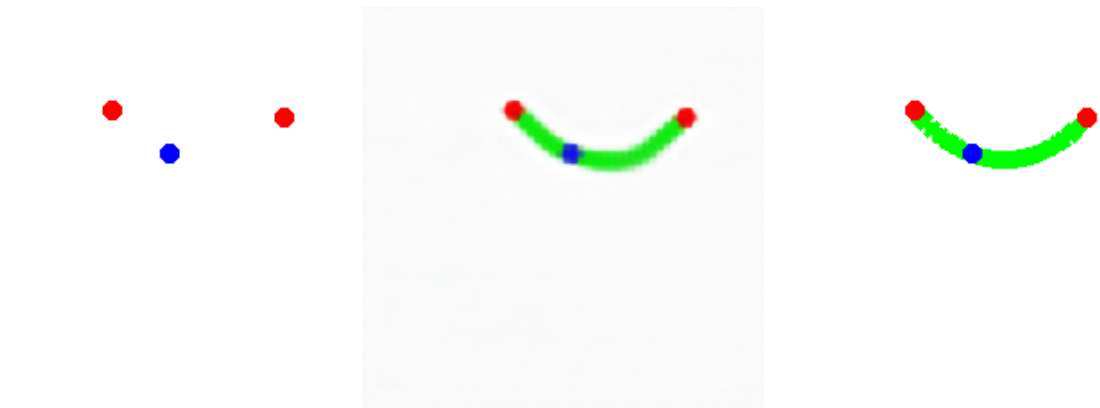


图 51: 1600step

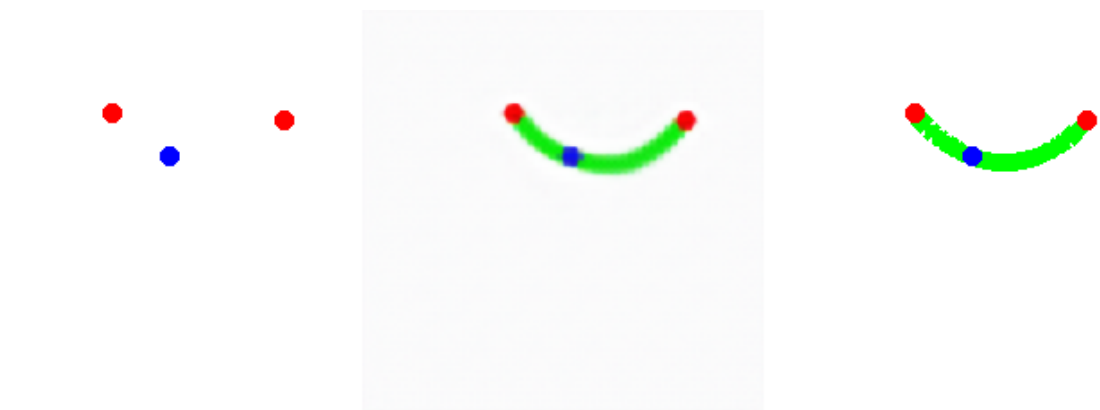


图 52: 1800step



图 53: 2000step

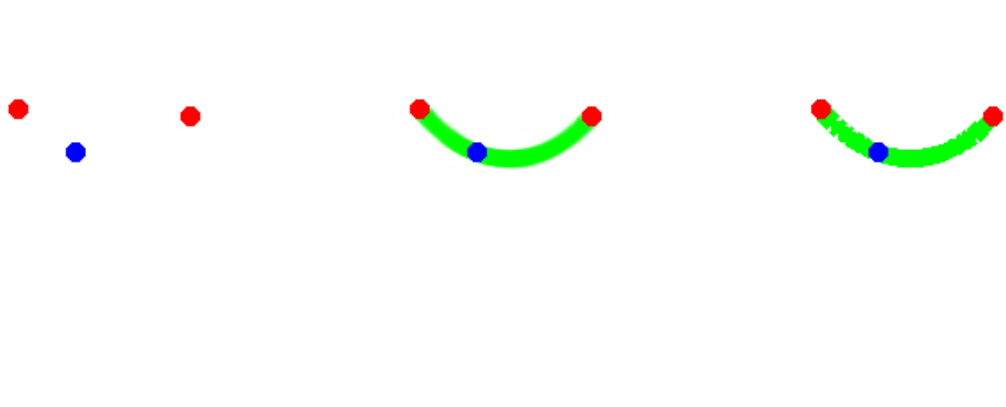


图 54: 19000step

4.6.2 行星轨道模拟

任务描述：给定恒星的固定位置，行星初始位置，初始速度大小，初始速度方向，预测行星轨道

输入格式：224*224 图像

输出格式：224*224 图像

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_orbital_path_from_initial_state.py

数据集：orbital_dataset_256_separated_v3 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：150000

输入空间大小估计：采用蒙特卡洛法估计约 $2.8e11$

训练集大小占输入空间大小的比例：约 $5.36e-7$

任务相关设计思想和讨论：

1. 这个任务是为了进一步检验这个范式的物理相关的模拟能力，通过图像推理模型实现。
2. 虽然这个问题存在通过内插近似解决的可能性，但同一类模型和超参数配置也能够处理一些难以用朴素内插解释的任务，因此这里更倾向于把它视为对程序化物理规则映射的拟合。
3. 恒星的固定位置，行星的初始位置相对容易在输入图像中表示，关于行星的初始速度大小和方向，我采用这种方式表示：从行星初始位置发出一个较短的射线线段，用颜色代表速度大小，线段方向代表速度方向，关于速度大小和颜色对应方式见训练集生成脚本代码。

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该设置下能够生成接近目标的行星轨道图像，但这种模拟的精度仍需要进一步探究。训练过程展示了模型逐步形成轨道输出的过程；由于输出轨道设置为白色，在学习初期白色区域弥散在输出图像中，视觉上令人联想到量子力学中的概率云。

左边是输入图像，中间是模型生成图像，右边是 ground truth。

后面由于变化速度减缓，直接展示最后训练结果

4.7 arc-agi 任务相关

本节将展示该范式在处理 [12,13] arc-agi-1/2 任务上的能力。

我的实验方法介绍：

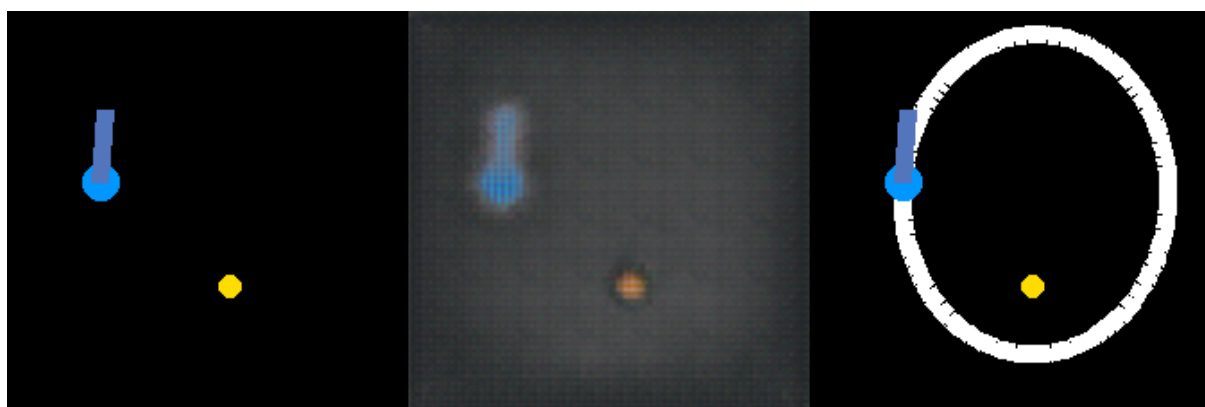


图 55: 200step

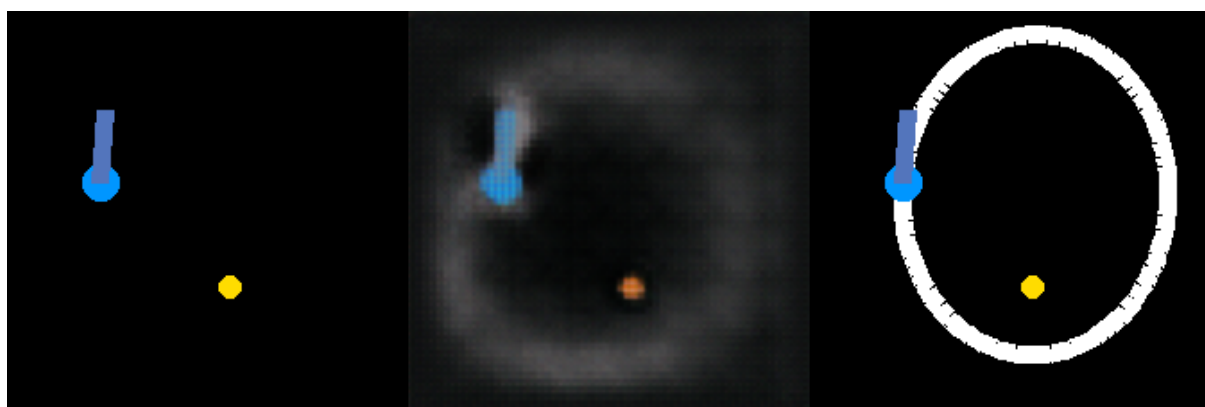


图 56: 400step

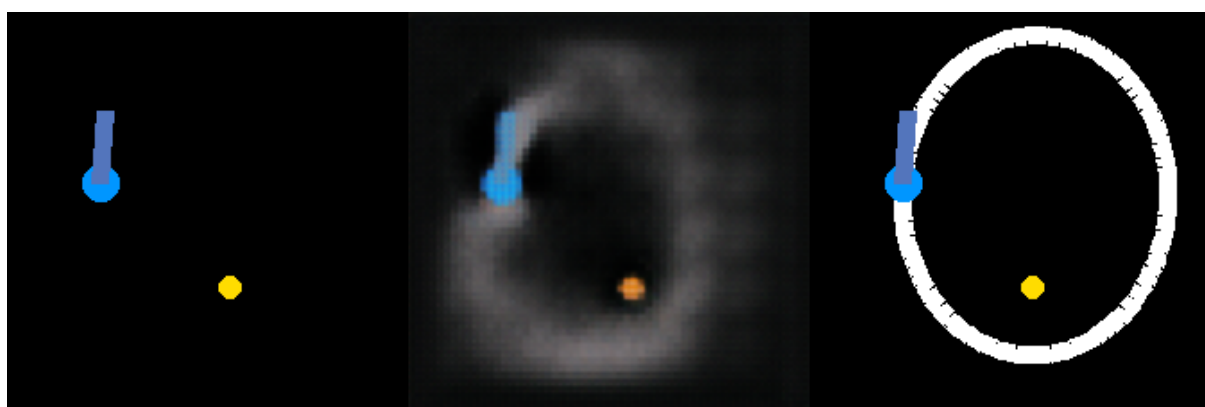


图 57: 600step

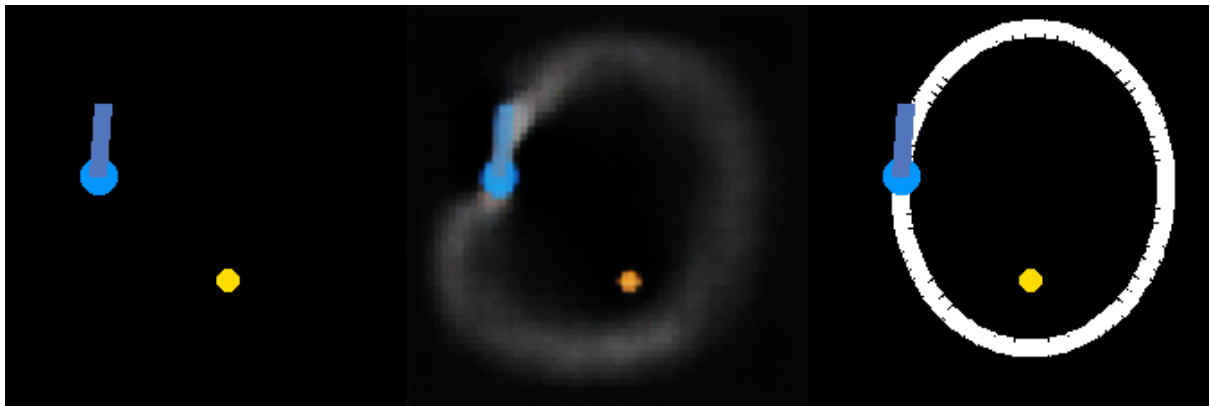


图 58: 800step

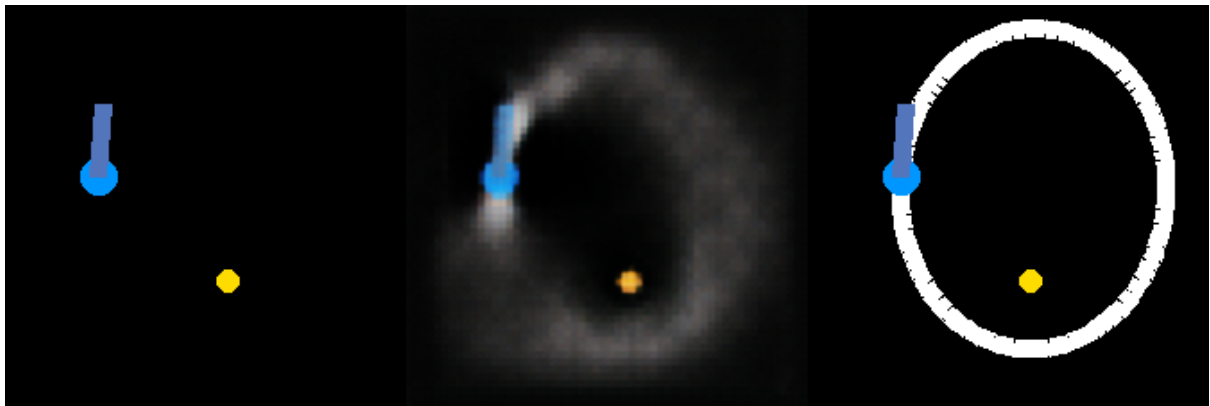


图 59: 1000step

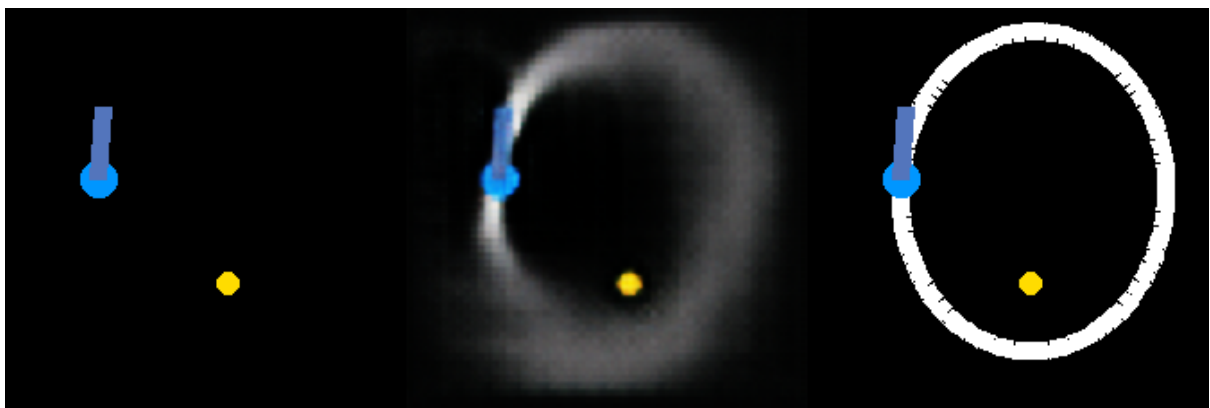


图 60: 2000step

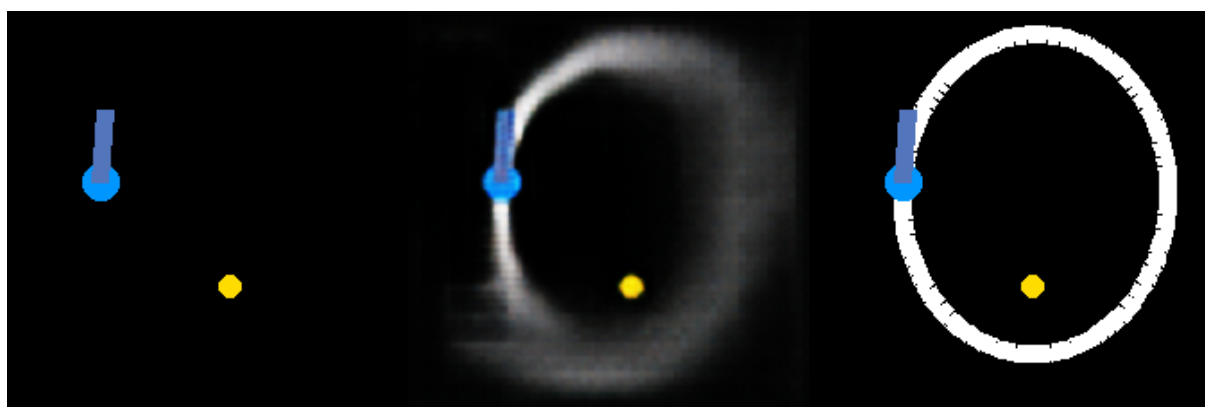


图 61: 3000step

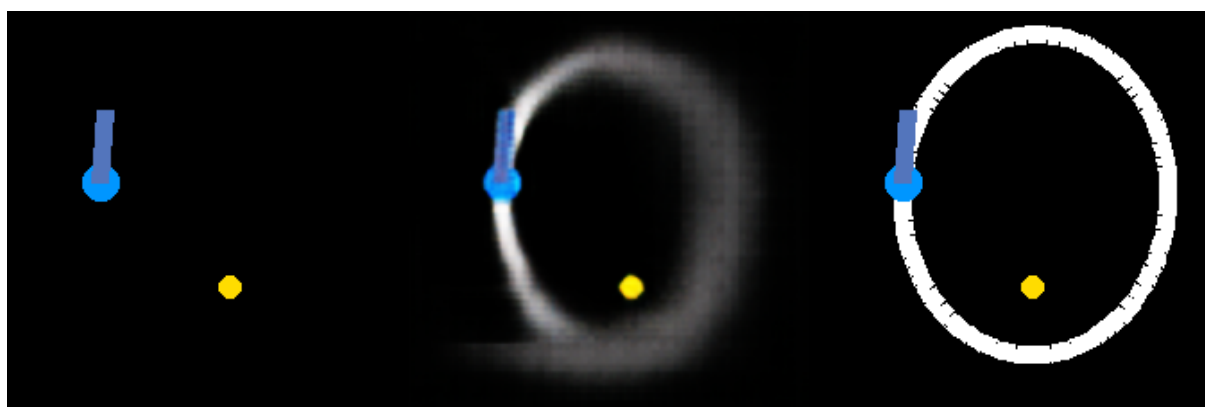


图 62: 4000step

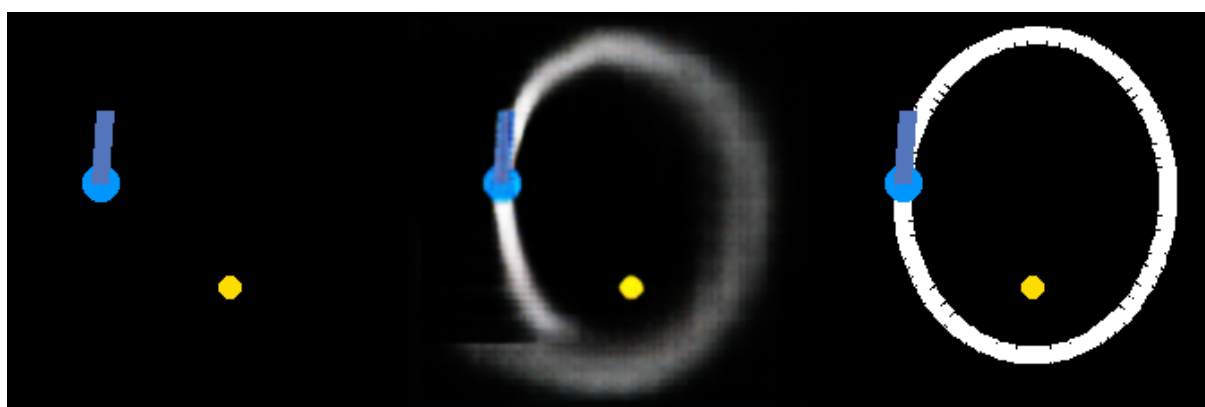


图 63: 5000step

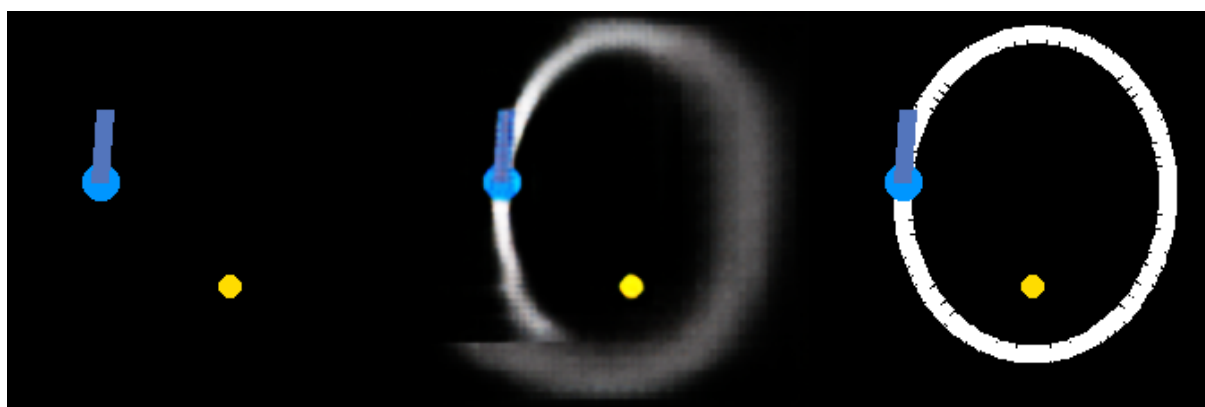


图 64: 6000step

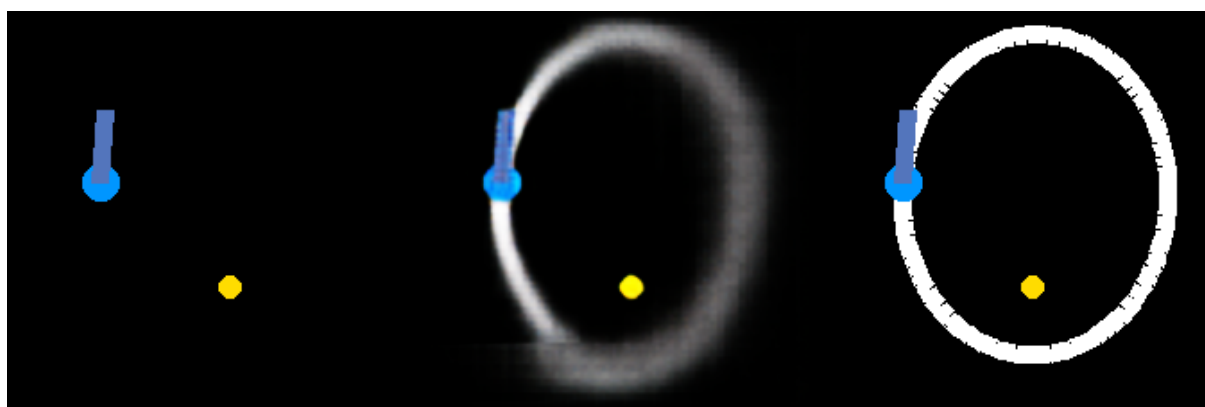


图 65: 7000step

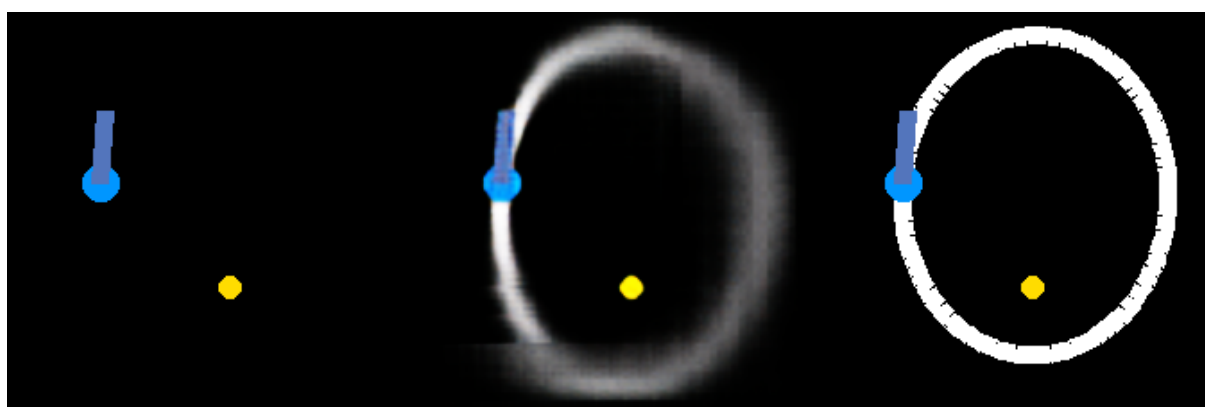


图 66: 8000step

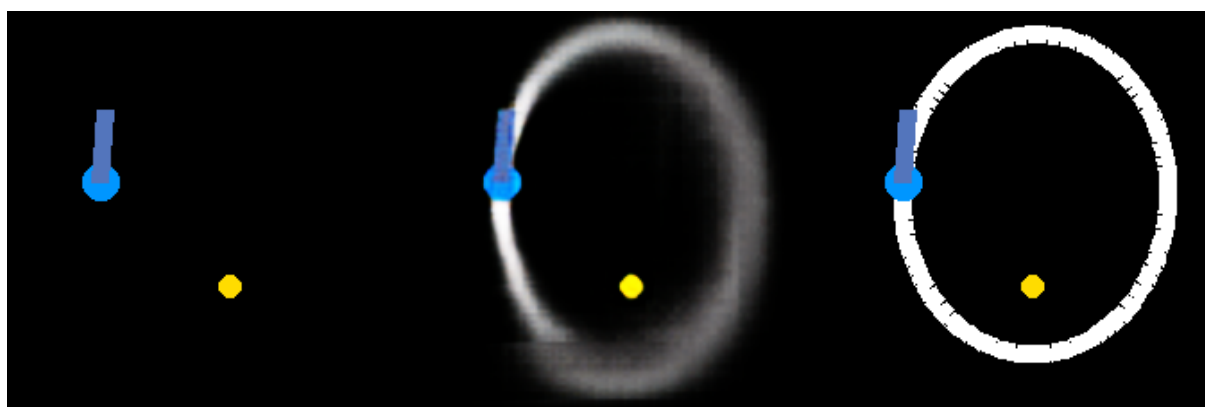


图 67: 9000step

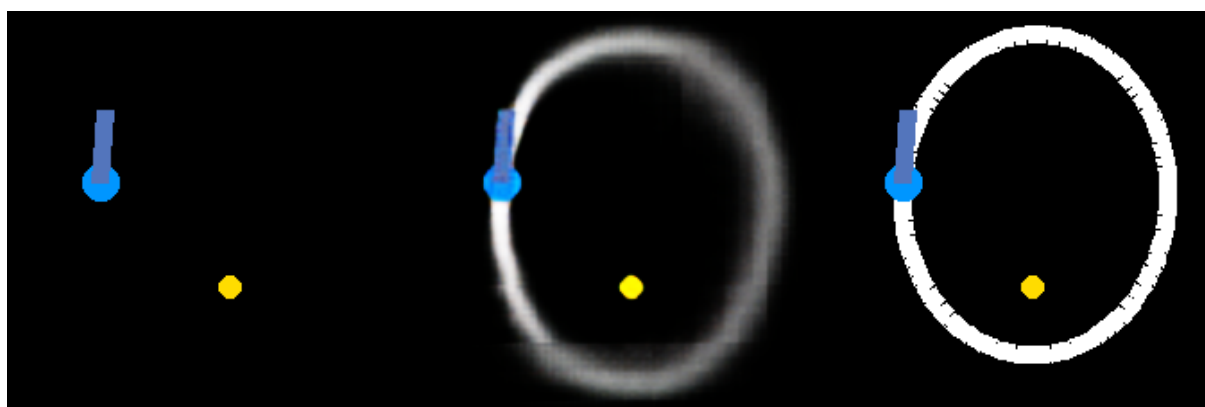


图 68: 10000step

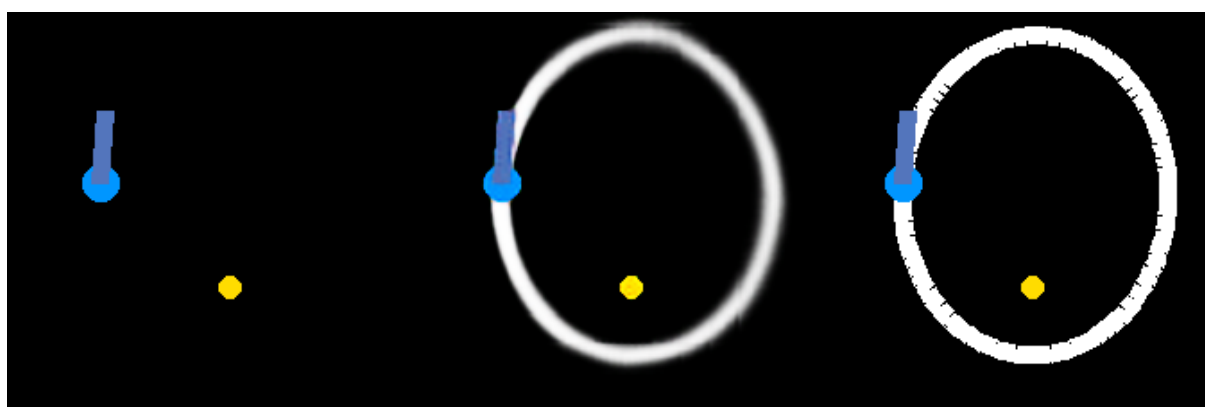


图 69: 156400step

1. arc-agi-1/2 数据集的模式都是给出非常少量的例子，3 个左右，要求在这些例子上领悟游戏规则，在新的例子上完成泛化和解题。我试过用我的方法严格按照 arc-agi-1/2 的要求是无法完成的，会出现极其严重的过拟合。因此我采用了一种迂回的方法，我先手动观察一个任务的逻辑，通过与 LLM（如 gemini 2.5 pro）的交互式编程，来辅助编写和调试生成这些复杂任务的训练数据脚本，然后运行脚本生成大量数据集，一般在 150000-200000 左右，最后在这个数据集上训练并寻求泛化。
2. 因为我的这种实验方法需要大量时间精力，包括人工观察任务的逻辑和生成及修改训练集生成脚本的时间，所以我只在 16 个任务上进行了尝试，8 个 arc-agi-1 任务，8 个 arc-agi-2 任务，按照随机挑选并且训练集生成脚本相对简单的原则。结果是非常值得注意的：在随机抽取的 16 个 ARC-AGI-1/2 任务上，按照程序化生成同构训练数据的方式，模型最终都实现了收敛；其中绝大部分问题较快收敛，少数问题需要更长时间训练。这说明，在超出官方 few-shot 限制、允许程序化扩展同构训练样本的设定下，神经网络可以稳定学习这些 ARC-AGI 风格任务中的规则结构。这个结果不等同于直接解决 ARC-AGI 官方设置，但它清楚地表明，这类抽象规则任务一旦被转化为可大量采样的规则学习问题，就进入了神经网络可以有效拟合的范围。
3. 采用图像推理模式，Swin Transformer+unet 模型。
4. 采用 arc-agi-1/2 数据集官网那种彩色图像的表达方式。
5. 格点的横纵维度在整个数据集中保持一致。
6. 一般来说 150000 个数的数据集就足以完成训练，但限于时间精力，我没有尝试过最少多少就可以。

4.7.1 水流模拟

任务描述：出自[ARC Prize - Puzzle #36a08778](#)

输入格式：224*224 图像

输出格式：224*224 图像

loss：MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_arc_fluid_simulation.py

数据集：arc_fluid_final_dataset 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：150000

输入空间大小估计：采用蒙特卡洛法，运行 2000000 次没有重复样本

训练集大小占输入空间大小的比例：如上，可以认为训练集大小占比输入空间极少

任务相关设计思想和讨论：

为了进一步检验 transformer 纯粹的图像推理能力，我把尝试扩展到已有的 arc-agi-1/2 的数据集，但采用另一种迂回的方法，已在上面说明。

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该程序化同构数据设定下能够稳定拟合这个任务的规则。需要强调的是，这并不是直接解决 ARC-AGI 官方 few-shot 设置，而是说明当任务规则被程序化扩展为足够多的训练样本后，模型可以学习其中的抽象变换结构。训练过程显示，模型一开始倾向于学习较简单但错误的局部规则，随后逐步抛弃这些规则并逼近真正的任务规则。这为研究 ARC-AGI 风格任务中的规则学习提供了一个可扩展的实验入口。而且这个任务本身的推理性质很大程度上排除了模型在进行朴素内插的可能。

实验过程展示：

左边是输入图像，右边是 ground truth，中间是生成图像。

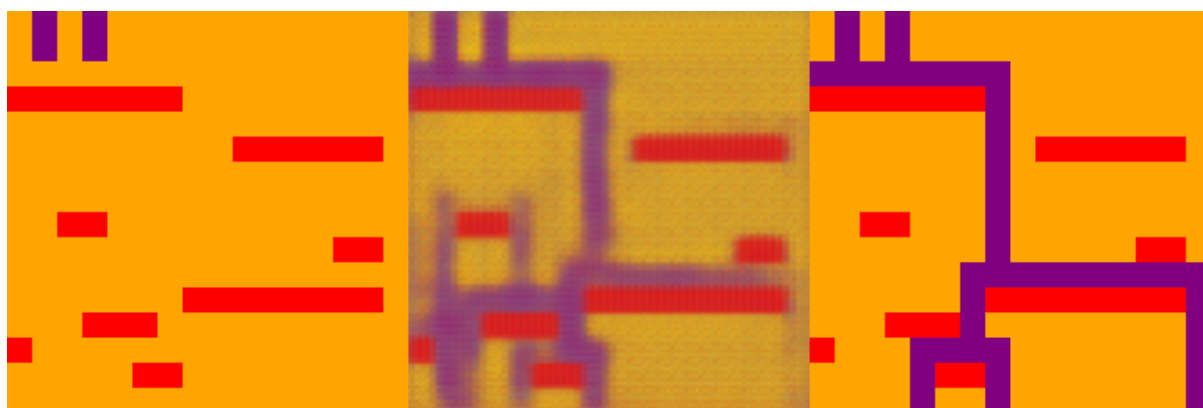


图 70: 200step

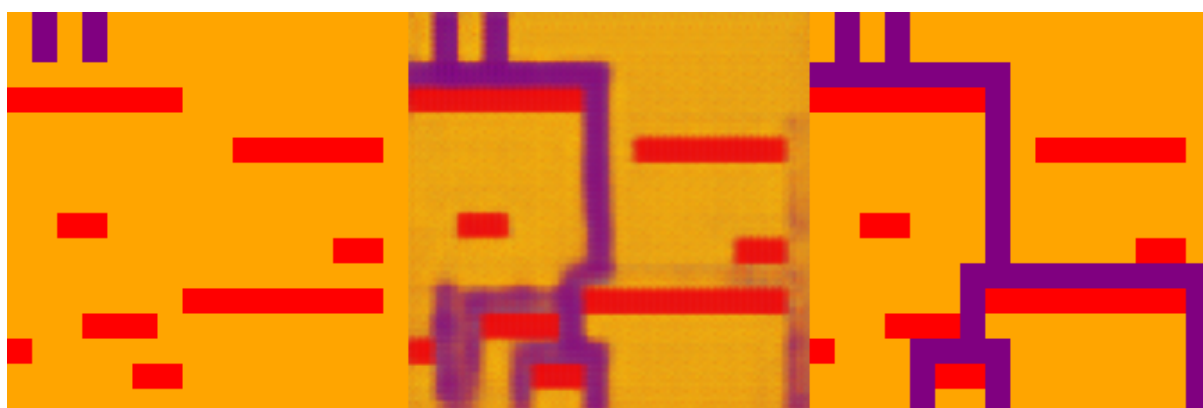


图 71: 400step

后面由于变化速度减缓，直接展示最后训练结果

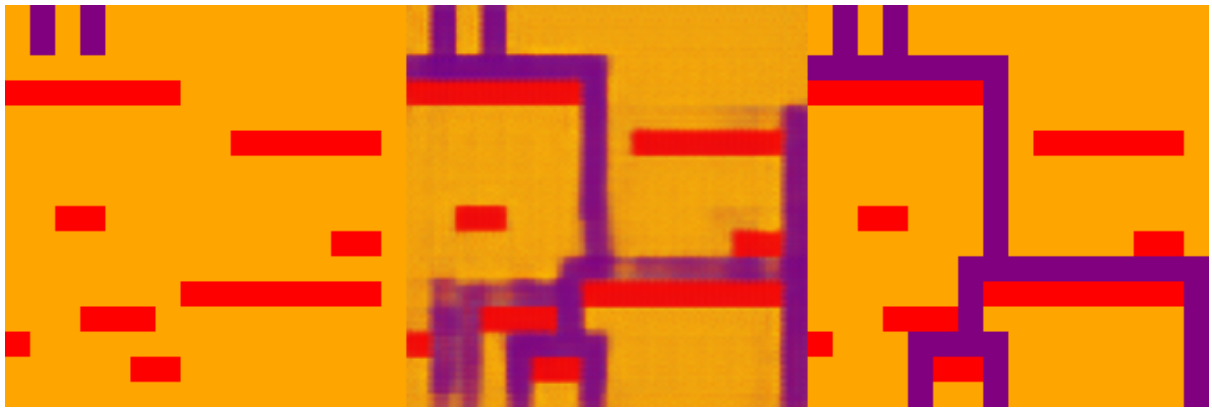


图 72: 600step



图 73: 800step

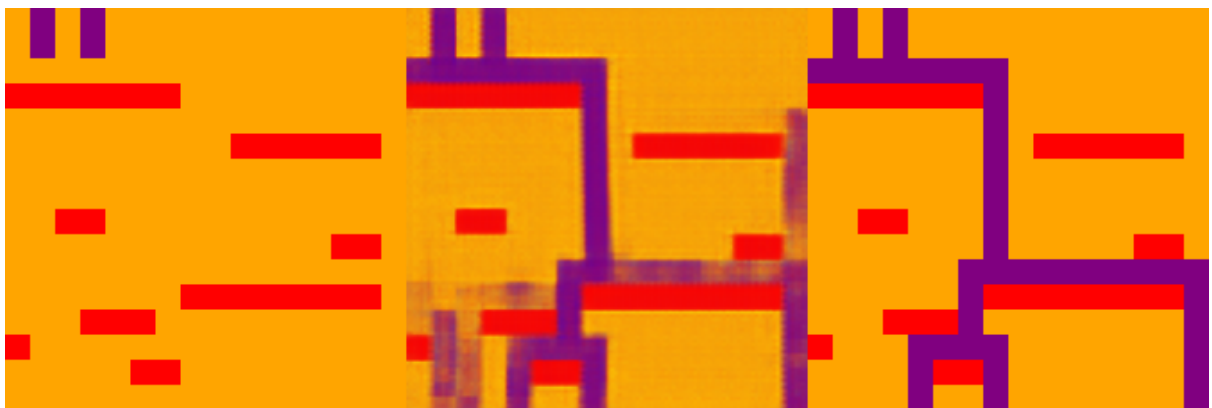


图 74: 1000step



图 75: 1200step



图 76: 1400step



图 77: 1600step

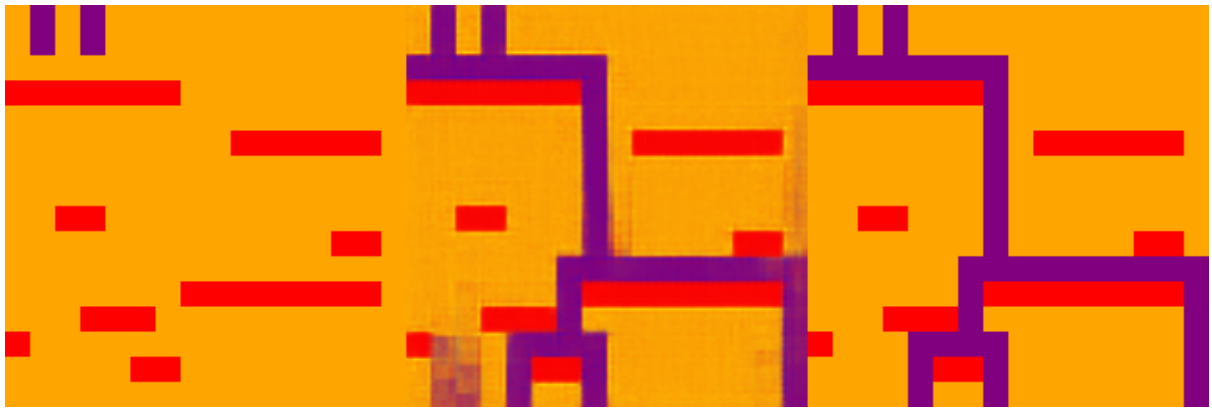


图 78: 1800step

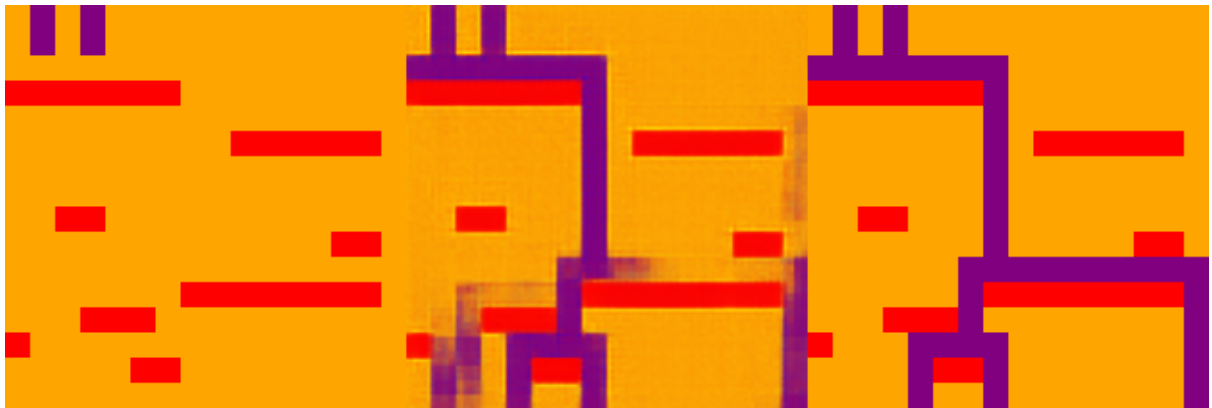


图 79: 2000step



图 80: 3000step



图 81: 4000step



图 82: 5000step



图 83: 39600step

4.7.2 积木游戏

任务描述：出自[ARC Prize - Puzzle #898e7135](#)

输入格式：224*224 图像

输出格式：224*224 图像

loss: MSELoss

训练配置：4090 显卡

训练集生成脚本：generate_arc_jigsaw_puzzle_advanced.py

数据集：arc_jigsaw_puzzle_mine_dataset 文件夹

训练代码：train_image2image.py

使用模型：Swin Transformer+unet

训练集大小：200000

输入空间大小估计：采用蒙特卡洛法，运行 1000000 次没有重复样本

训练集大小占输入空间大小的比例：如上，可以认为训练集大小占比输入空间极少

任务相关设计思想和讨论：

为了进一步检验 transformer 纯粹的图像推理能力，我把尝试扩展到已有的 arc-agi-1/2 的数据集，但采用另一种迂回的方法，已在上面说明。

一些任务参数：

图像分辨率：224*224

实验结果与分析：最终的训练结果表明，模型在该程序化同构数据设定下能够稳定拟合这个任务的规则。与水流任务类似，这一结果并不等同于直接解决 ARC-AGI 官方 few-shot 设置，而是表明程序化扩展后的抽象变换任务可以被图像模型学习。训练过程也展示了模型从背景色、形状位置到填充颜色规则的逐步学习过程。而且这个任务本身的推理性质很大程度上排除了模型在进行朴素内插的可能。

实验过程展示：

左边是输入图像，右边是 ground truth，中间是生成图像



图 84: 10000step



图 85: 20000step



图 86: 30000step



图 87: 40000step



图 88: 50000step



图 89: 60000step



图 90: 70000step



图 91: 80000step



图 92: 90000step



图 93: 100000step



图 94: 150000step

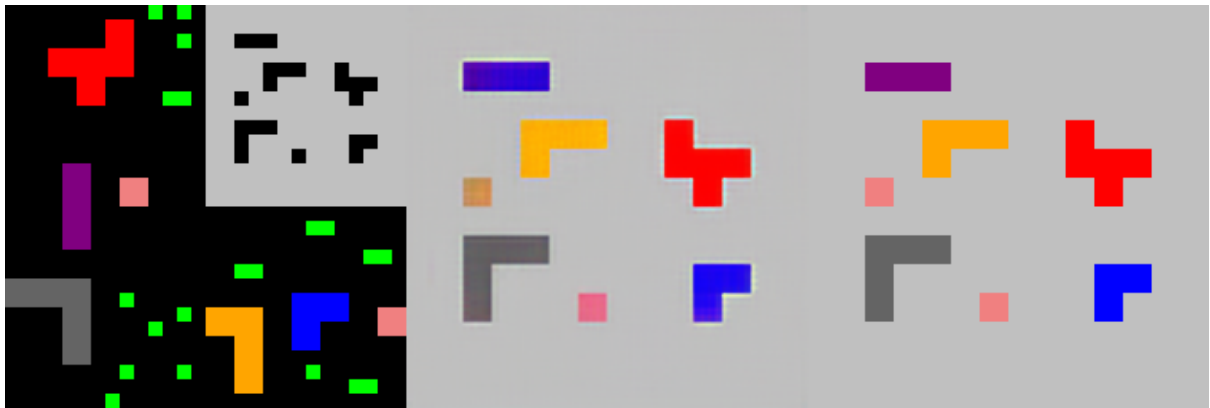


图 95: 200000step



图 96: 250000step

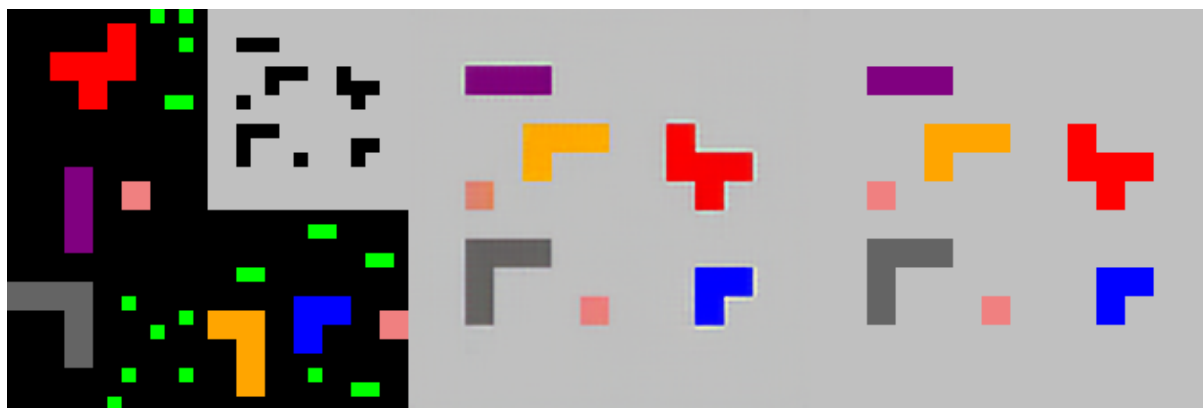


图 97: 300000step

后面由于变化速度减缓，直接展示最后训练结果

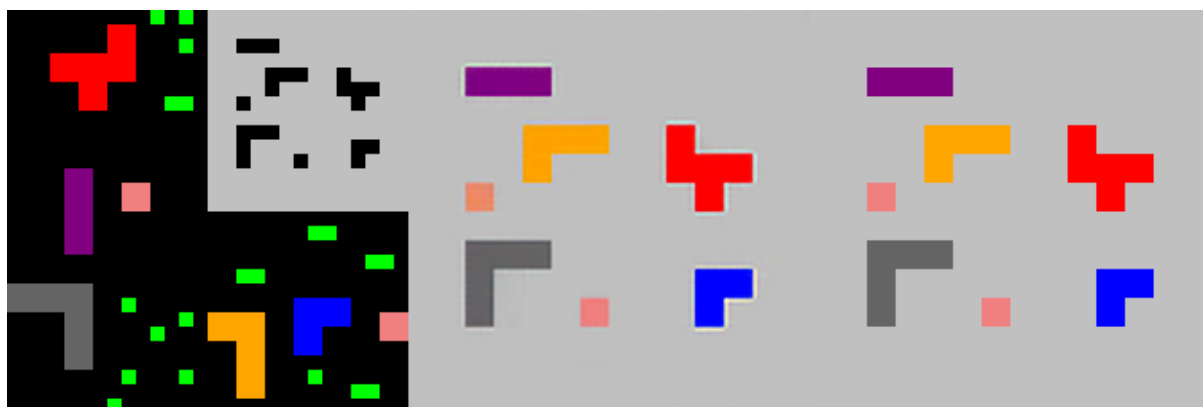


图 98: 530400step

4.8 可解释性探索

本节将展示该范式输出推理过程的能力。

4.8.1 编辑过程输出

任务描述：模型学习预测和输出两个 0/1 字符串之间的最短编辑过程

输入格式：30 位的由“0”或者“1”组成的字符串，前面 15 位表示字符串 A，后面 15 位表示字符串 B

输出格式：450 个标签的多标签二分类格式，等同于 450 个 01 字符串，每连续的 30 位分为一组，一组的前 15 位代表编辑过程中字符串的中间状态，后 15 位代表字符串的掩码，以区分字符串编辑过程中长度小于 15 位的情况

loss：平均二元交叉熵损失

训练配置：4090 显卡

训练集生成脚本：generate_edit_distance_explainable.py

数据集：edit_distance_path_unique_final.jsonl

训练代码：train_tiny_transformer.py

训练集大小：500000

输入空间大小估计：2 的 30 次方 = 1073741824

训练集大小占输入空间大小的比例：0.0466%

任务相关设计思想和讨论：

1. 这理论上来说并不算直接解释神经元权重或者某个模块的作用，而是一种在可解释标签指导下的可解释性。这种可解释性的操作方法是，直接把需要观察的内部过程加入标签，实践表明这种方法是实用的。比如有些问题，既然网络可以拟合答案，在很多情况下可以推测，其内部表示中很可能包含与中间状态或中间过程相关的信息，这个时候就可以通过这种方式暴露。至于更纯粹的神经网络的可解释性的原意，我认为这个范式可以做到更加干净的实验室环境来观察神经网络的权重变化，这个在后面会有讨论。

2. 一些任务细节：

- 通过脚本保证编辑过程中没有字符串长度超过 15 的时候，否则我的输出格式无法表示。
- 并且通过脚本保证最小编辑路径有且仅有一种，否则会大大提高网络收敛的难度。
- 编辑距离字符串选择限定在 01 两种，是为了降低任务难度，而不改变问题本质。

实验结果与分析：评估损失收敛至 0.001404。这个结果表明，该范式不仅可以训练模型给出答案，也可以通过附加可解释标签的方式输出部分中间过程，从而为研究黑箱模型中的中间状态提供一种可操作入口。

最终收敛结果：

— Step 66000 — Train Loss: 0.001995 Eval Loss: 0.001404

5 第五幕：论可学习性与可解释性的边界

我总结了这个范式的特点

1. 输入输出格式要设计好，输入格式要尽量减轻 transformer 解码负担，输出尽量采用多类别分类对应交叉熵 loss 或者多标签二分类对应平均二元交叉熵 loss。可以对应任务的特点自己设计输出格式的含义，只要保证整个数据集中的格式一致就可以，例如如果输出是一个数，可以用多标签二分类表示一个对应位数的二进制数。
2. 关于学习难度相关

之前在做符号规则学习任务的时候观察到一个现象，即任务的结构也很重要，看起来似乎是顺向符号规则任务才容易学，根据输出逆向求解输入则不容易学，除非这个逆向其实也是某个顺向符号规则的形式。现在看来可能是因为这种“逆向求解”任务更类似算法模拟问题，所以学习难度更大。

观察到的另一个现象是，带有分支条件判断的任务可能会加剧学习难度，比如某一个指示位指示用完全不同的规则计算后面的操作数。现在想想，分支判断其实也是规则本身，只不过会加剧符号规则的复杂程度，让它变得更难学习。

另外，输入中可以留有控制任务规则的位置，即输入 = 任务规则 + 操作数。这个设想已经在元胞自动机相关实验中得到成功验证。

另外，观察到的另一个现象是，串行混合过多简单逻辑的任务也很难学习，假如一个符号变换规则串行组合不同简单符号规则，而每一个规则本身学习起来都很简单，它仍然可能组合成一个极其难以学习的任务。一个简单的例子是实验部分的接雨水实验，通过把最后的雨水量加和步骤取消，对问题进行解耦，才使得那个任务得以快速收敛。

顺向符号规则学习有难易程度之分，比如元胞自动机 1 维 30 位，不同 rule 的学习难度不同，应用不同层数 rule 的学习难度也不同，基本上层数越多学习越难，还有一些基本不可学习的任务，比如 rsa 加密，虽然是一个顺向的简单数学计算，但确实无法学习。

所以说规则难度影响 loss 下降速度。关于这个，举一个具体例子来说，元胞自动机，rule 110，演化层数，至少在较少的时候，严格遵循一个层数越多，训练越困难，模型收敛越慢的规律。这让人联想到 wolfram 提出的 [15] 计算不可约性，他也确实是拿元胞自动机的演化来举例子的。

另外，训练过程本身就是一种计算不可约性，训练/雕刻的过程是不可省略的。而且越简单的问题，训练越快，越难的问题，训练越慢，如果不考虑这个问题的本质是否对神经网络友好，输入输出格式是否对神经网络友好的话，控制好不同问题的输入的规模大致一致，训练过程中 loss 下降的速度快慢甚至可以用来估计这个问题的可学习性或者计算不可约性。比方说我曾经试图训练 rsa 加密，loss 就完全不下降，始终没有学习到任何东西。这里还是拿元胞自动机举例，输入和输出都是 30 位 01 字符串，那么 rule 的类型和演化层数就可以完全决定输入到输出的映射。但这只是输入空间到输出空间映射方式的一种。为什么这一种就会比——演化层数更多的元胞自动机映射规则更容易学习？也显然比完全随机 permutation 的映射会更加容易学习？——完全随机 permutation 的映射显然是无法学习的，因为不包含任何规律。这是非常有意思的问题，但对它的探讨超出我的能力范围。另外，rsa 算法的无法学习证实了密码学家创造的简单算法在这个意义上是非常成功的。

所以，可以通过这个，有可能通过实际训练任务的方法，这种类 monte carlo 方法，测定任务难度，可学习性/计算不可约性的量化衡量。这在数学上可能有潜力，为什么有的规则就被认定为复杂，有的就被认定为简单？以及这是否会形成一个新的理论？

另外关于模型容量，按直觉猜测应该是模型容量更大/更适合这个任务，会让训练更快。因此，可能是模型容量和任务难度结合会决定一个训练的 loss 收敛速度和收敛过程。

3. 关于训练集中变换规则不一致时的现象。即使每条规则本身都很简单，如果训练集中不同样本实际遵循不同规则，而输入中又没有任何信息指示应当使用哪条规则，模型也很难学习到稳定映射，只能按照降低 loss 的原则在不同规则之间取得某种折中。如果混合了很多规则，但又无法被识别该用哪个简单规则，这个时候训练 loss 和 eval loss 会在一个高点震荡，因为不同规则在拉扯到一个均衡点。这个 loss 的值的高低将会取决于训练集中规则的纯度。

另外，mod 3 是一个令人非常意外的无法学习的规则。它非常简单，但 loss 又始终停留在高位，有可能和那个规则混合的现象有关，但是它至少在人类的视角直观看起来又不是混合规则。我做了一个实验，输入是 20 位 2 进制数，由 20 位 01 字符串表示，计算它的 mod 3 结果，由 2 位 2 进制数表示，loss 是 mean bce loss。现在有趣的结果来了，我生成了四个数据集，第一个数据集是不对输入有限制，第二个数据集是限制 mod 3 不能为 0，第三个数据集是限制 mod 3 不能为 1，第四个数据集是限制 mod 3 不能为 2。训练的结果是，他们分别 eval loss 稳定在下面的结果：

不做限制的数据集——0.636 限制 mod 3 不能为 0——0.693 限制 mod 3 不能为 1——0.347 限制 mod 3 不能为 2——0.347

分析这个结果是很有意思的，我先给出一个事实，就是假如模型从数据中学不到任何东西的话，对于一个 bce loss，假如标签为 1 的概率为 p ，那么模型会把 loss 降到

$$-p \ln p - (1 - p) \ln(1 - p)$$

。之后我就会看到这些 loss 结果证明模型没有从数据中学到任何东西，即 mod 3 是无法学习的。原因如下：

对于不做限制的数据集，输出的 2 位二进制数，高位为 1 的概率是 $1/3$ ，低位为 1 的概率也是 $1/3$ ，因此最佳

$$\text{loss} = -\frac{1}{3} \ln \left(\frac{1}{3} \right) - \frac{2}{3} \ln \left(\frac{2}{3} \right) = 0.6365,$$

与实验结果非常接近。对于限制 mod 3 不能为 0 的数据集，输出的 2 位二进制数，高位为 1 的概率是 $1/2$ ，低位为 1 的概率也是 $1/2$ ，因此最佳

$$\text{loss} = -\frac{1}{2} \ln \left(\frac{1}{2} \right) - \frac{1}{2} \ln \left(\frac{1}{2} \right) = 0.693,$$

与实验结果非常接近。对于限制 mod 3 不能为 1 的数据集，输出的 2 位二进制数，高位为 1 的概率是 $1/2$ ，低位为 1 的概率是 0，因此最佳

$$\text{loss} = -\frac{1}{2} \ln \left(\frac{1}{2} \right) = 0.347,$$

与实验结果非常接近。

对于限制 mod 3 不能为 2 的数据集，输出的 2 位二进制数，高位为 1 的概率是 0，低位为 1 的概率是 $1/2$ ，因此最佳

$$\text{loss} = -\frac{1}{2} \ln \left(\frac{1}{2} \right) = 0.347,$$

与实验结果非常接近。

因此，可以认为至少在当时这种实验设置下，模型并没有真正学到 mod 3 规则；这个失败更可能来自任务表示和训练地形，而不只是模型拟合能力不足。

在训练汉诺塔实验中有类似现象，具体现象为，假如我只训练一个固定盘子数量的汉诺塔问题，从初始态到终止态的最佳路径上的所有状态，训练这些状态的最佳 action，会发现

收敛非常迅速。但当我试图把状态扩展到固定盘子数量的整个汉诺塔状态空间时，会发现 loss 停留在一个较高的位置完全不再下降，而且很可能并非拟合能力的问题。注意观察汉诺塔的整个状态空间和 mod 3 问题的相似性，会发现 mod 3 在 2 进制格式下是有点类似递归或分形性质的，汉诺塔就更加是天生具有递归或分形性质。那这是因为输入格式或者输出格式的设计原因导致无法学习，还是因为问题本身？是否这个范式无法把涉及递归或分形的规则有效识别出来？我倾向于是后者。

那么，为什么只在汉诺塔从初始态到终止态的最佳路径上训练时可以快速收敛？一个可能解释是，在这条最佳路径上，任务可以被更简单的非递归识别规则刻画，因此学习难度大幅降低。

那么这两个和递归或分形相关的问题，是否和最开始提到的混合规则有关系？本文认为可能是有关的，因为当模型无法识别递归规则时，它可能会认为是某种复杂的混合规则形式，而且它无法识别哪个输入对应哪个规则。

（注：mod 3 问题和全空间汉诺塔问题已经得到解决，mod 3 问题的讨论见 8.1 节，全空间汉诺塔问题见开源代码 `algorithms/generate_hanoi_decoupled_scaffold.py`。其共同核心思想是，这类递归类型问题可以视为某种层数非常深的任务，但没有足够的中间过程提示，又由于递归的性质，很容易发生类似梯度抵消的现象，所以很难建立输入和输出间的联系，解决方法是提供中间过程提示的解耦信息。）

4. 在这个范式中，很多任务的数据可以由程序低成本、大规模生成。理论上我需要使得训练集和整个输入空间同分布，那么最简单的方法就是在输入空间随机采样。但这里会有另外一个问题，会不会存在另一种训练集的构造方式，使得模型的学习效果更好？比如假如规则的某一个分支状况出现地较少，随机采样很难覆盖足够的模式。我的设想是也许可以有更加精妙的数据集生成方式，更好地覆盖这个问题的各种模式。因为根据实践经验来看，只要有足够的模式覆盖，训练集就可以达到效果，具有抗过拟合的能力并使得满足模型学习的需求，这也是训练集的总量并不需要非常大的原因。如果从这个角度分析，过去模式识别的深度学习范式，需要大量训练集的原因就比较清楚了，因为那些任务没有精确的变换规则，需要更多的数据去覆盖输入空间的各种模式。

这个范式通常不以传统意义上的 OOD 泛化为主要目标。更准确地说，它关心的是：在训练数据只覆盖整个输入空间极小一部分的情况下，模型能否在同分布、未见过的巨大剩余输入空间中稳定泛化。对于输入空间固定、数据由同一程序分布生成的任务来说，这种泛化可以类比传统深度学习中的泛化，但边界更加清楚、可验证性也更强。

5. 可控 ai 和可解释性

关于可解释性，如前文所述，这并不是直接解释单个神经元权重或某个模块的作用，而是一种由可解释标签引导的功能性可解释性。这种可解释性的操作方法是，直接把需要观察的内部过程加入标签，实践表明这种方法是实用的。例如，在许多任务中，既然网络能够稳定拟合最终答案，就可以推测其内部表示中很可能包含与中间状态或中间过程相关的信息；此时，将这些中间过程加入监督标签，就可能把原本隐藏的内部计算过程以功能标签的形式暴露出来。

关于更可控的 AI，由于这个范式的任务形式本身，非自回归模型在拟合精确规则时，其错误形式与 LLM 开放式生成中的“幻觉”并不相同。实践中也没有观察到类似的幻觉式现象。原因可能是，这个范式基于规则任务，内在规则极其精确，本质上不是光滑的，无法仅靠内插解决问题；因此，在足够理想的数据和训练条件下，模型更可能表现为规则学习不足或精度不足，而不是开放式生成中的无根据编造。

另外一个相关发现是，训练过程中 batch size 不能过小；如果 batch size 过小，学习会慢很多。一个可能原因是，这类任务要求拟合精确规则，不同样本的梯度方向差异较大，因此需要足够大的 batch size 来平均梯度、稳定优化方向。

6. 在许多实验中可以观察到，训练 loss 和验证 loss 往往同步下降，而且验证 loss 经常低于训练 loss。这说明，在训练集覆盖了足够多任务模式时，该范式具有较强的抗过拟合能力。相反，如果任务难度较高、关键模式覆盖不足，模型仍然可能出现过拟合或无法收敛。
7. 这个范式学习的是符号之间的结构关系，而不是符号本身的人类语义。对于模型来说，0、1 或某个 emoji 都可以只是符号占位符；真正重要的是这些占位符之间在数据集中形成的结构关系。换句话说，“结构”才是真正的“语言”：模型最终学会的，不是 0 代表什么，也不是 1 代表什么，而是这些符号背后共同满足的运算结构。前面的 n 进制加法和语义洗牌实验已经支持了这一点。
8. 关于这个范式整体的哲学含义

数据需要覆盖问题中的各种关键模式，并尽量与目标输入空间保持同分布。是否必须严格同分布仍有待讨论，但随机采样通常是最方便、最直接的做法。这里真正重要的不是绝对数据量，而是模式覆盖是否充分。数据就像是给神经网络提供的参照样品或模特。这个过程有点类似 Monte Carlo 方法：不需要显式写出求解逻辑，而是通过大量样本和训练过程逐渐逼近目标映射。数据越充分，提供给神经网络的“参照”就越清晰、越具体。训练越多，神经网络拟合的就越好。从数据编程的角度来看，数据集的规模和质量决定模型效果的关键。

范式整体的比喻，就好像是，神经网络要在一个大理石上雕刻出类似数据的输入输出模式的样子，数据是模特，神经网络是雕塑师。用算力/训练，就好像在大理石上雕刻，是用连续性去逼近一个确定性的符号变换规则或者拟合一个未知但确定性的算法。而训练不足，loss 尚未下降到极致，就好像雕刻尚未完成，如果问题过难，超出了这个神经网络的能力，就好像这个雕塑需要的能力超出了雕塑师的能力。而就算训练不足或者问题过难，神经网络也进行了拟合，也有效地压缩了输出空间的不确定性，就好像一个雕塑尚未完工，也比原来的大理石更像模特。

这个范式最有意思的一点在于：它用连续的梯度下降过程逼近离散、确定、可验证的符号规则或算法映射。这个过程本身也许具有值得进一步研究的数学意义。而且本文认为我的范式有潜力在人们暂时没有找到通解方法的问题上，压缩问题的输出的不确定性，尽管可能无法给出精确的算法过程。这意味着，神经网络确实是在学习一个符号或算法任务，但它是自身独特的联结主义方式完成学习，而不是按照人类想象中一步步显式执行的算法步骤。它面对的是耦合的 loss 和梯度下降过程，本质上只是在寻找能够降低 loss 的参数

配置。例如，在一个算法问题上，如果试图在已经训练好的权重中精确对应出“哪一部分在执行算法步骤的哪一步”，本文认为这很可能是不可能的。神经网络并不知道输入符号在人类语境中的位置和含义，它面对的只是输入输出约束和梯度下降目标。

我现在感觉这个范式和 Demis Hassabis 所说的提倡压缩结构直觉和 Ilya 坚持的压缩即智能的信念是一致的。[1] AlexNet 证明了之前手工设计的特征是不够好的，用联结主义直接学习图像特征，会得到更好的结果。而这个范式证明了，即使对于精确的符号规则，这个路线在某种程度上也是正确的。而且，正如前面所说，即使在无法完全收敛的任务上，模型也往往可以压缩答案的不确定性。

至于为什么这种联结主义的压缩尝试可能是有效的。我的感觉是，Demis Hassabis 提到的，世界的复杂度正在超过人类语言与逻辑的承载极限，但这一解释可能存在局限性。我联想到 Bob Dylan 的歌曲《Things Have Changed》中的一句歌词，All the truth in the world add up to one big lie。但这并不意味着逻辑的失效，我的观点是，一个很长的逻辑链条的每一个命题都是有自己独特的语境和上下文的，而很多时候人们把他们组合为逻辑链条时会在某种程度上忽视这些语境和上下文，导致最终的逻辑链条的失真。现实生活中也经常可见此类场景，一个人在讲他的思考，尽管他说的每句话似乎都是正确的无可辩驳的，但最终却导向了一个荒谬的结果。这时候联结主义就派上用场，直接对接真实数据和现实验证，重新校准自身的直觉和逻辑。这就像是符号规则从联结主义中浮现，人类的思考也是先积累观察和经验，再总结规律的，先归纳规则，才有演绎，演绎是无法替代归纳的，正如前面所说，如果不够严谨地对待演绎，演绎会变得非常脆弱，这时仍然需要诉诸联结主义对应的观察和经验以及现实验证。

不过，单纯的端到端结构压缩解释可能仍然不够完整。对于某些递归、长链条或高度耦合的任务，即使底层规则是确定的，模型也未必能够在原始输入输出之间直接建立稳定的梯度路径。前面的解耦实验表明，引入中间过程标签、结构化表示或其他解耦信息，可能会显著改变训练地形，使原本难以学习的问题变得可学习。也许这正是逻辑认知或结构洞察的作用：人无法仅仅依靠直觉应对世界，还需要通过结构化分析把复杂问题拆成更容易学习和理解的形式。

9. 通过这个范式探索神经网络黑箱，可能会比直接研究自然图像、自然语言或音频任务更容易。原因之一是，对于可训练的问题，loss 可以降到非常接近 0。这种情况，第一它确实是梯度下降黑箱学到的参数，第二，和其他的图像识别，nlp 不同，符号规则被学习可以更加被有迹可循地研究，对同一个模型分别训练不同的可训练问题，然后研究训练过程中的神经网络的权重变化或者验证输出的变化等等，比之前的模式识别的图像和 nlp、音频的范式更容易在实验室的无菌环境研究。

而且，更有利于这一点的是，人工设计需要的任务规则，并且生成数据集是非常简单的。

同时，图像推理相关的实验可以可视化机器心智的形成过程。例如，我在三角形画内切圆的任务中，训练隔一段时间 step 就会保存一个当前 eval image，就可以很直观地看到学习过程的进展，以及模型可能是以什么方式学习相关任务的。

10. 如果这个范式未来能够发挥更大潜力，它应该可以在部分场景中用于学习未知系统的输入输出规律：即使人们暂时不知道系统的完整工作原理，也可以通过程序化或实验数据训练

神经网络，使其对新的输入给出可靠预测。这在某种意义上类似于 Andrej Karpathy 所说的 [21] Software 2.0 模式。

这种方法未来应该会在 AI for Science 中发挥潜力，尤其适合用于探索那些可大量生成或采集数据、但显式规律尚不清楚的系统。

11. 这个范式同样可以用于规则抽取，即给定输入和输出，让模型预测背后的变换规则。我已经在元胞自动机任务上进行了实验，并取得了很好的结果。不过，在该实验中，我通过控制数据集生成方式，保证了单个输入输出对可以唯一确定变换规则和演化层数。现实世界问题往往未必满足这种单样本确定规则的条件。因此，如果要从大量输入输出样本中抽取规则，可能需要设计多样本聚合或投票机制，使模型从一组样本中推断共同规则。
12. 如果可以对模型权重和拟合的规则的关系有一些认识，是否有可能主动地手动设计权重去模拟某一个规则？当然很有可能这是非常困难的，即使清晰的符号规则让它看起来简单了点。所以可能用训练来得到权重还是更好的方法。这个问题的价值可能主要是在理论上。
13. 在训练中观察到一种现象，即有时候当损失降低到非常接近 0 的时候，会突然一个跳变到高位 loss，推测可能是因为这类任务的非平滑性导致。
14. 一个进一步设想是，可以将这种实验流程高度自动化。人只需要与 AI 协同编写训练集生成脚本，后续的数据生成、训练、指标记录和初步分析都可以自动完成。这样，不同任务是否可学、学习难度如何、需要何种表示或解耦信息，本身就会形成一种新的实验数据，供算法研究者、神经网络理论研究者甚至数学家进一步分析。
15. 关于合成数据的限制，需要明确的是，本文并没有对模型本身进行根本性结构创新，而是改变了数据条件和任务形式。一旦数据变为有噪声、含混、多义的真实世界数据，神经雕刻范式的理想前提就不再完全成立。因此，这个范式未必能够直接用于现实世界中的所有复杂问题。它更重要的价值，可能在于提醒我们：深度神经网络本身就具有拟合精确符号规则的潜能，并且为研究神经网络行为提供了一个更可控、更干净的实验环境。通过这个环境进一步理解神经网络的本质，也许能够帮助我们更深入地理解 LLM 幻觉、CV 模型失效等现实问题，并为构建更可控、可解释的 AI 系统提供线索。

此外，这个范式可能会对某些符号世界的问题产生更直接的应用价值。

16. 关于最近苹果的论文，[14] The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity，在这篇论文中，叙述了一些大模型无法可靠推理的问题。我在 4 个问题，汉诺塔，积木问题，过河问题，跳棋问题上，用我的范式进行了训练，都取得成功，尽管汉诺塔问题存在上面所说的一些微妙之处，积木问题的收敛也不够完美。详细情况请见开源代码。

另外，当时我注意到有一些反对的声音，说是因为 token 不足的限制所以无法正常推理。所以，我针对汉诺塔问题，设计了一个格式 1>2 表示把第一个柱子最上方的盘子向第二个柱子移动，以分号分割。然后让大模型用这个格式来回复汉诺塔问题的解，结果发现基本上 $n=6$ 左右就是极限了，再往上就会出现错误，所以我认为大模型在不调用工具（如

python) 的情况下确实无法对这些问题进行有效推理。检验这个格式的答案是否正确的脚本，我也开源了，见代码仓库 `eval_hanoi.py`。

17. 关于这个范式含义的重新思考：这个范式似乎与计算机视觉领域在 [1] AlexNet 出现前的状况惊人地相似。当时，主流方法依赖于手工设计的特征（如 [17] SIFT, [18] HOG），这可以看作是一种对视觉世界的“先验符号知识”。而 AlexNet 的成功，恰恰在于用端到端的学习（归纳）取代了这种手工偏置。而在推理领域，当前的神经符号计算，无论是改造架构还是嵌入逻辑，其本质都是将人类对推理过程的理解作为一种先验知识注入模型，而这种事先确定的结构，这种归纳偏置可能是不真实或不合适的。
18. 关于这个范式的哲学暗示：我感觉即使没有驳斥概念的先验存在，也可以有力地证明从经验中可以自发生长出概念。至于进一步的探讨，由于个人能力所限，只能止步于此。
19. 解耦和 [22] chain of thought 或者 [23] 课程学习有非常明确的对应关系，这可能说明这个概念的深刻含义并不是这篇论文研究的任务独有的。

6 第六幕：超越架构的普适性

在这一节中，我探索了深度学习的其他模型是否具有和 transformer 同样的能力。我总共探索了 5 种，mlp, rnn/lstm, cnn, unet, diffusion。实验表明，拟合精确规则的能力并不是 Transformer 所独有的，而是深度神经网络更一般的内在潜能。与此同时，具体架构仍然非常重要：如果网络结构导致关键信息在传播中被削弱、丢失，或其归纳偏置无法适配任务的长程依赖结构，这种潜能就可能无法在具体任务上显露出来。因此，架构并不决定“有没有这种能力”，但会强烈影响这种能力能否被训练出来，以及以什么效率和路径收敛。

我使用 mlp 和 rnn/lstm 来拟合符号版本的元胞自动机和接雨水算法题，当然这里的输入不再是 transformer 类似的 token，而是实数。cnn 用来拟合输入图像，输出符号的元胞自动机数据集和图像版接雨水数据集。UNET 和 diffusion 用来拟合输入和输出都是图像的元胞自动机数据集。

数据集生成脚本：

符号版元胞自动机：`generate_cellular_automata_1d.py`

符号版接雨水：`generate_trapping_rain_water_decoupled.py`

图片版元胞自动机：`generate_cellular_automata_image_and_label.py`

图片版接雨水：`generate_trapping_rain_water_image_to_symbol.py`

6.1 元胞自动机

数据集介绍

均为 1 维 36 位元胞自动机数据集，数据集大小为 300000，演化规则为 110，演化层数为 2，唯一的例外是，mlp 模型可能过于适合元胞自动机这个任务，拟合能力过强，我加了演化层数到 6。

模型结构见开源代码。

6.1.1 cnn 模型

训练代码：train_convnext.py

我采用了 [5] ConvNeXt 模型，这是训练过程。

step	训练损失	验证损失	验证位准确率	验证完全匹配率
1000	0.49467600	0.27210647	84.55833333%	0.10000000%
2000	0.11814510	0.05764289	97.36018519%	36.80000000%
3000	0.04807972	0.03248511	98.63425926%	56.20000000%
4000	0.00763491	0.00266492	100.00000000%	100.00000000%
5000	0.00186690	0.00128128	100.00000000%	100.00000000%

可以看到以极快的速度完成了收敛，最后在 4000step 的时候就已经达到了验证集完全匹配的 100% 正确率，这表明至少在这个问题上，CNN 也具备符号规则学习能力。

6.1.2 mlp 模型

训练代码：train_mlp.py

对于一个巨型 mlp 来说，元胞自动机的演化层数为 2 的数据集，收敛地过于迅速，我不得不重新产生一个演化层数为 6 的数据集。事实证明也非常迅速地在 5 个 epoch 结束的时候达到了验证集完全匹配的 100% 正确率。

初步分析是因为 mlp 对于神经元的超距联系保存的实在是太好了，因此在这个问题上可能不会比 transformer 差。

6.1.3 rnn 模型

训练代码：train_lstm.py

这里有一些小小的设计，一是我必须在第一个时间步把元胞自动机的输入状态完全输入，而不是一个时间步输入一位，二是，我不可能采取自回归式输出，所以我必须把某一个时间步的输出当作结果，三是，我最后刻意选取了时间步和元胞自动机的演化步数不同，因为元胞自动机的演化步数为 2，所以我选择 rnn 模型的输出时间步为 3。虽然时间步和演化步数完全一致肯定是最理想的模型设计，但是我这里想要证明的是我前面的观点，就是模型并不是以人类通常的认为的逻辑或者算法步骤来执行任务，在这个实验设定下，模型被迫学习一个变换函数，它连续重复变换 3 次等于元胞自动机的规则 110 变化 2 次。这个设定的学习成功可以为我对这个范式的观点提供有力支持。

而实验的结果也证明，在 1000step 就完成了收敛，已经达到了验证集完全匹配的 100% 正确率。

后续我把 3 个时间步改为 5/7/9，同样可以在很短时间内达到验证集完全匹配的 100% 正确率。

6.1.4 unet 模型

训练代码：train_unet.py

采用 mseloss，但很快就卡在 0.0915 的高位完全不再下降，可能是网络架构本身不适合这个任务，有待进一步分析。

训练代码：train_unet_v2.py

采用 mseloss，经过几个小时的训练，mseloss 下降到 $1e-5$ 以下，视觉层面完全拟合，对于所有像素层面的精确匹配准确率最高达到 99% 以上

两个版本 UNet 表现的差异可能反映了第一个版本 UNet 的信息传递方式或归纳偏置无法适配这个任务。这一现象并不否定神经网络拟合精确规则的内在潜能，反而说明这种潜能需要通过合适的架构和信息通路显露出来：如果结构设计导致关键信息在传播中被削弱甚至消失，模型就可能无法拟合某些规则任务；而一旦信息通路和归纳偏置与任务结构更匹配，训练效率和收敛路径会发生显著变化。

6.1.5 diffusion 模型

训练代码：train_diffusion.py

经过几个小时的训练后，终于完成了拟合。虽然这个架构最终拟合了这个任务，但和 swin-unet 的拟合效率相差极大，再次支持我们的结论，归纳偏置决定了学习效率与收敛路径的显著差异。

6.2 接雨水

任务描述：源自 LeetCode 经典算法题，[42](#)。

[接雨水 - 力扣 \(LeetCode\)](#)

格式采用和上面实验部分相同的解耦输出格式。不同的是，把柱子数调整为 12，则输入输出都为 $12 \times 3 = 36 = 6 \times 6$ 。方便 cmn 相关实验输入一个 6×6 的黑白棋盘格图像。

6.2.1 mlp 模型

训练代码：train_mlp.py

最佳结果：Validation Loss: 0.0002, Bit Acc: 99.99%, Exact Match: 99.90%

6.2.2 rnn 模型

训练代码：train_lstm.py

我这里选择时间步为 3

最佳结果：Validation Loss: 0.0001, Bit Acc: 100.00%, Exact Match: 100.00%

6.2.3 cnn 模型

训练代码：train_convnext.py

我采用了 [5] ConvNeXt 模型。

最佳结果：Epoch 6 | 训练损失：0.00003849 | 验证损失：0.00001896 | 验证位准确率：100.00000000% | 验证完全匹配率：100.00000000%

6.3 总结和分析

我用元胞自动机实验考察符号规则推演能力，用接雨水实验考察算法拟合能力。而这一系列实验表明，这个范式并不是 transformer 的专属，其他很多模型也有类似的能力。这支持一个更一般的判断：精确规则学习很可能不是某个特定架构的偶然性质，而是联结主义的深度学习神经网络在合适数据条件下可以显露出的内在能力。而且，很有可能 transformer 也未必是最佳模型，或者说对于这个范式的不同问题，最佳模型也会不同。此处限于时间精力，我个人无力再探索。

7 第七幕：跨越鸿沟：从理想规则到现实世界的连续谱

这一节，我主要讨论两个问题。在以上所有讨论内容里，我集中精力在有精确变换规则的数据集中。所以在这一节的第一部分，我会对有精确规则的数据集的输入输出进行随机扰动，并观察训练进程和最终 loss 收敛情况。

第二部分，我设计了一个实验，用以展示神经网络的规则学习能力和连续值内插能力并不矛盾，而是可以在同一个任务中同时出现并相互耦合。

7.1 精确规则的扰动

我选择一维元胞自动机的任务，演化规则选择为 110，演化层数选择为 2，一维元胞自动机的位数选择为 30 位。具体的扰动做法为，在生成数据的过程中，对输入或输出以一定的概率进行随机翻转（即 1->0 或者 0->1），为了简单，我只测试了单独翻转输入或单独翻转输出的情形。验证集采用不进行随机翻转的精确规则的元胞自动机数据集。

数据集代码：generate_cellular_automata_1d_perturbed.py

训练代码：train_tiny_transformer.py

7.1.1 对输出的随机翻转测验

随机翻转概率	eval loss 稳定值
0.1%	0.000692
1%	0.008490
10%	0.094094

7.1.2 对输入的随机翻转测验

随机翻转概率	eval loss 稳定值
0.1%	0.001895
1%	0.021377
10%	0.230316

7.1.3 总结讨论

可以看到不管是输入还是输出扰动，都会随着扰动概率的增大而增大任务学习的难度的，这也是很自然的，另外输入的扰动，比输出的扰动影响更大，这可能是因为规则任务本身固有的非平滑性质，可以猜测如果把演化层数增加，输入的扰动还会使任务学习变得更加困难。所以，粗略地说，从无噪声精确变换规则数据集到真实世界的数据集，中间可能存在一个连续谱。

7.2 内插和规则学习的一体性

这一节用一个实验展示神经网络的规则学习能力和连续值内插能力不是互斥的，而是可以在同一个训练过程中耦合出现。

7.2.1 实验描述

为了验证神经网络在一个单一任务中融合离散规则学习与连续值处理的能力，我设计了一个逻辑感知混合实验。该实验基于元胞自动机图像生成任务，在输入和输出图像中，把一维 36 位的元胞自动机状态以行优先排列成 6*6 的棋盘格图像，黑色代表 1，白色代表 0。但对输入和输出进行了如下修改：

输入端：输入图像不再是纯黑白色块，而是将代表逻辑黑与白的格点，分别替换为在两个不同灰度区间内随机取值的色块，以此引入连续的感知信息。

输出端：要求模型生成的输出图像必须同时满足两个条件：

- 逻辑正确性：格点的黑白模式必须严格遵循元胞自动机的多步演化规则。
- 感知关联性：每个格点的最终灰度值，必须根据其对应的输入格点原始灰度值和一个简单的条件函数（白则保持，黑则反转）来精确计算。具体的，假设输入格点的灰度值为 x ，如果输出格点和输入格点的逻辑模式相同，则输入格点的灰度值保持为 x ，否则为 $255-x$ 。

这个设计巧妙地迫使模型必须同时看穿输入的连续灰度值以执行离散的逻辑推理，并记住这些灰度值以完成最终的连续值映射，从而在一个不可分割的任务中，同时考验其规则学习与内插两种核心能力。

数据集代码：generate_cellular_automata_1d_to_grid_image_interp.py

训练代码：train_image2image.py

loss：MSELoss

一些任务参数：

格点维度：6*6

格点大小：40*40 像素

图像分辨率：240*240

元胞自动机规则：rule 110

规则演化层数：2

7.2.2 实验结果

在训练 3200 步之后，验证集的 mseloss 就已经来到 0.000899

这是 3200 步的 eval image

左边是输入图像，右边是 ground truth，中间是生成图像

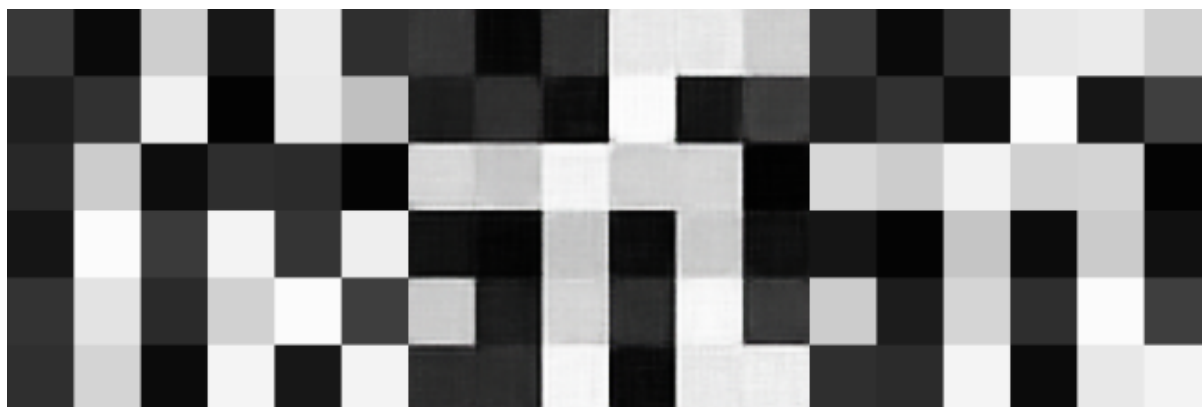


图 99: 元胞自动机插值实验：3200 步评估图像，左为输入图像，中为模型生成图像，右为 ground truth

7.2.3 总结讨论

可以看到这个实验取得了很大的成功。它强烈支持了一个判断：连续值处理和规则学习都可以作为神经网络在合适数据条件下显露出的内在能力，而且在这个任务中二者不是简单并行，而是被同一个训练过程耦合在一起。模型并不是先按人类可读步骤理解元胞自动机演化规则，再单独记住每一个格点的灰度值；它实际在做的仍然是梯度下降，是以联结主义的方式同时逼近离散规则和连续值映射。

7.3 模式识别和规则学习的一体性

7.3.1 MNIST+ 元胞自动机：模式识别到规则执行

这一节用一个实验展示，模式识别和规则学习能力可能都是神经网络的内生能力，可以在同一个任务中耦合出现，有可能它们本就是同一种能力。

7.3.1.1 实验描述 为了验证神经网络在一个单一任务中融合离散规则学习与模式识别的能力，我设计了一个实验，用来展示模式识别和符号推理可以在同一任务中自然耦合，并强烈暗示二者都可能是神经网络的内生能力。该实验基于元胞自动机图像生成任务，在输入和输出图像中，把一维 36 位的元胞自动机状态以行优先排列成 6*6 的棋盘格图像，黑色代表 1，白色代表 0。但对输入进行了如下修改，现在我把输入的黑色格点换成 mnist 数据集中的 1，白色格点换成 mnist 数据集中的 0。可以预想到应该不会完美收敛，但我可以分别把元胞自动机演化 0 层——即原局面的元胞自动机表示的 eval loss（这里所有的 eval loss 都是因为模式识别），和演化比如说 3 层后的 eval loss 对比，以确认元胞自动机的局面演化带来了多少额外 eval loss。输出的格式不变，仍然是黑白棋盘格格点。

这个设计巧妙地迫使模型必须同时体现模式识别能力以理解元胞自动机初始局面，并通过符号推理能力执行离散的逻辑推理，从而在一个不可分割的任务中，同时考验其规则学习与模式识别两种核心能力。

数据集代码：generate_mnist_ca_110.py

训练代码：train_image2image.py

loss：MSELoss

一些任务参数：

格点维度：6*6

格点大小：40*40 像素

图像分辨率：240*240

元胞自动机规则：rule 110

规则演化层数：3

7.3.1.2 实验结果 当输出是输入局面的元胞自动机表示（即演化 0 层）时，在训练 2600 步之后，验证集的 mse loss 就已经来到 0.000393。这是 2600 步的 eval image。可以看到网络在识别 mnist 数字上没什么问题。左边是输入图像，右边是 ground truth，中间是生成图像。

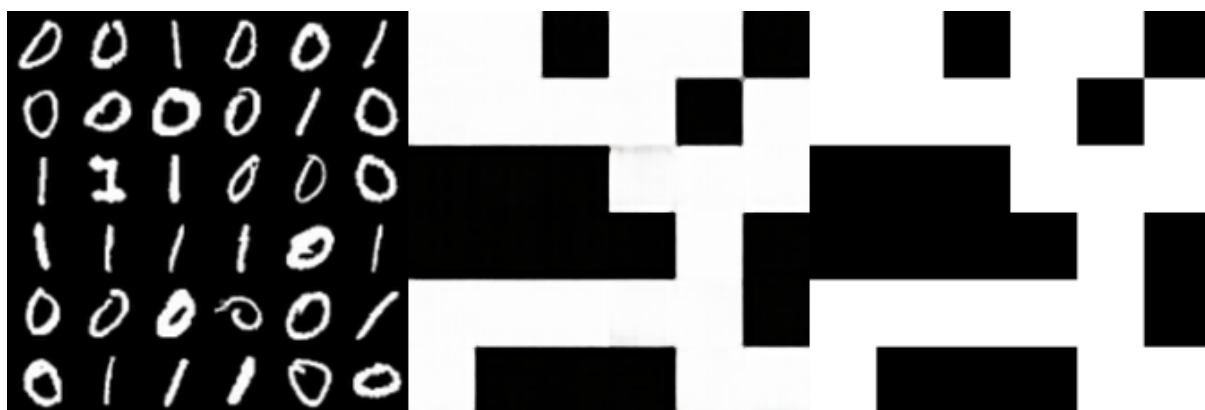


图 100: MNIST+ 元胞自动机实验：演化 0 层的 2600 步评估图像，左为输入图像，中为模型生成图像，右为 ground truth

当输出是输入局面的元胞自动机表示按 rule 110 演化 3 层后时，在训练 6800 步之后，验证集的 mseloss 就已经来到 0.001320。这是 6800 步的 eval image。左边是输入图像，右边是 ground truth，中间是生成图像。

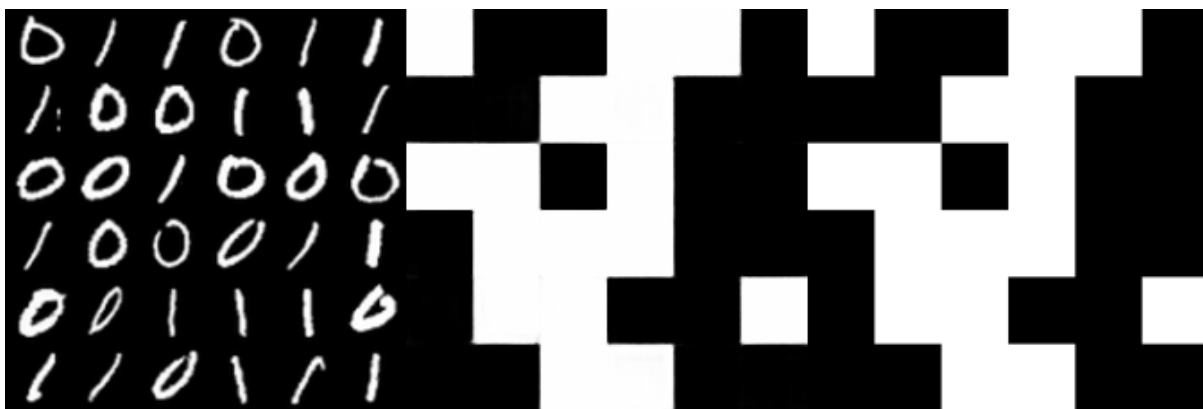


图 101: MNIST+ 元胞自动机实验: rule 110 演化 3 层的 6800 步评估图像, 左为输入图像, 中为模型生成图像, 右为 ground truth

7.3.1.3 总结讨论 可以看到这个实验取得了很大的成功。充分证明不管是模式识别还是规则学习能力，都是神经网络固有的本质能力。而且它们可能不是分离的两种能力，而是并行一体的，因为这是同一个网络结构表现出的两种截然不同的能力，而且我上面已经证明，这种网络结构也不是符号规则学习能力的根源，而是神经网络的本质能力。并且，在这个例子中，模型并没有试图先理解识别 mnist 的数字值，把它转换为初始局面的黑白棋盘格图像表示，然后再按元胞自动机的演化规则演化局面，模式识别和符号规则学习能力的学习过程也是耦合的，并无先后之分。它实际在做的应该只是梯度下降，仍然是前面所说的，联结主义的梯度下降可以逼近离散的符号规则。而且，由于模式识别和符号规则是耦合的能力，更准确的描述可能是，联结主义的梯度下降可以同时逼近离散的符号规则和拟合模式识别，至于神经网络表现出什么能力主要由数据集决定。

7.3.2 MNIST+ 元胞自动机：规则控制与图像变换

这一节用一个实验来证明神经网络的规则学习能力和模式识别能力不是互斥的，而是一体的，并着重证明了神经网络的推理是耦合式的，而非有清晰的先后步骤。

7.3.2.1 实验描述 为了验证神经网络在一个单一任务中融合离散规则学习与模式识别的能力，并融合了规则控制相关，我设计了一个实验，用来进一步展示模式识别和符号推理可以在同一模型中自然耦合，并强烈暗示二者都可能是神经网络的内生能力。该实验基于元胞自动机图像生成任务，在输入和输出图像中，把一维 36 位的元胞自动机状态以行优先排列成 6*6 的棋盘格图像。但输入图像的构成是这样的，首先从 mnist 中随机取数字 0 或者 1 的图像，拉伸到 240*240 的全图，然后按照初始元胞自动机的状态对初始图像进行修改，元胞自动机为 0 的位置，保持格点不变，否则进行颜色翻转变为反色。对输出图像也根据元胞自动机的演化状态，对输入图像修改前同一个数字的图像，做类似处理。元胞自动机的演化层数仍然为 3 层，但由 mnist 的数字 0 或者 1 来确定演化规则，0 代表 rule 30，1 代表 rule 110。

这个设计巧妙地迫使模型必须同时体现模式识别能力以理解元胞自动机初始局面，识别出初始局面的数字，并对应演化规则，然后通过符号推理能力执行离散的逻辑推理，最后通过元胞自动机最终演化状态修改最初的 mnist 数字图像，对应最终的输出图像。这个任务的复杂性不仅同时考验神经网络的规则学习与模式识别两种核心能力，也通过其长步骤链条有力支持了神经网络符号推理和模式识别在时间尺度和空间尺度上的耦合性。

另外这个实验设计之所以可行也是因为 mnist 是黑底白字，边缘一定是黑色，给了神经网络一个锚点去理解和对应元胞自动机的状态。

数据集代码：generate_rulemnist_ca.py

训练代码：train_image2image.py

loss：MSELoss

一些任务参数：

格点维度：6*6

格点大小：40*40 像素

图像分辨率：240*240

元胞自动机规则：输入图像的数字 0 对应 rule 30，输入图像的数字 1 对应 rule110。

规则演化层数：3

7.3.2.2 实验结果 在训练 10800 步之后，验证集的 mseloss 就已经来到 0.001011。这是 10800 步的 eval image。左边是输入图像，右边是 ground truth，中间是生成图像。



图 102: MNIST+ 元胞自动机规则控制实验：10800 步评估图像（一）

7.3.2.3 总结讨论 可以看到这个实验取得了很大的成功。它强烈支持了一个判断：模式识别和规则学习都可以作为神经网络在合适数据条件下显露出的内在能力，而且它们可能不是完全分离的两种能力，而是可以在同一模型和同一训练过程中耦合出现。并且，由于神经网络需要做的工作过于复杂，包括识别数字，对应元胞自动机演化规则，识别元胞自动机初始状态，演化元胞自动机，按演化后的元胞自动机状态修改原始 mnist 数字图像到输出。这么长的链条进一步表明，模型不可能是在按人类可读的步骤顺序串行执行任务，而更可能是在同一个耦合的优化



图 103: MNIST+ 元胞自动机规则控制实验：10800 步评估图像（二）

过程中学习整体映射。而且在后面的讨论中也会进一步直接验证这一点。

所以，模式识别和符号规则学习能力的学习过程也是耦合的，并无先后之分。它实际在做的应该只是梯度下降，仍然是前面所说的，联结主义的梯度下降可以逼近离散的符号规则。而且，由于模式识别和符号规则是耦合的能力，更准确的描述可能是，联结主义的梯度下降可以同时逼近离散的符号规则和拟合模式识别，至于神经网络表现出什么能力主要由数据集决定。

8 第八幕：运用解耦手段辅助训练——重新思考符号主义和联结主义的关系

在这一节中，我将运用上文中提过的接雨水实验的解耦思想，加速一些任务的训练，甚至把原本在直接输出设置下长期难以收敛的任务变得可收敛——即 2 进制数 mod 3 的任务，最后我会展示并不是所有的解耦格式都会加速训练，而坏的解耦格式甚至会减缓训练，并对其中蕴含的深刻哲学思想进行讨论。

下面详细介绍什么是解耦，即考虑到神经网络直接预测最终输出可能会存在困难，人工给定过程中的某一步或者几步作为额外标签给神经网络来辅助训练。比如，在上面提到的接雨水实验中，我并没有预测原题要求的总雨水量 的二进制表示，而是分别预测了每一个柱子所盛雨水量的二进制表示，这个方法大大降低了收敛所需的时间。但这个可以被质疑为我改变了原来的目标，下面我介绍两种不改变训练目标的解耦方法。

第一种，就类似于把上面接雨水解耦实验的输出——即每个柱子所盛的雨水量，作为输入，原本的目标——总雨水量作为输出，再用另一个神经网络训练（当然也可以不改变网络结构用同一个神经网络，只需要改变 linear head 层的输出个数）。这一种方法，相当于需要不同的网络预测不同的步骤，再把他们接合在一起。当然也可以把一个较大的网络手动切为几个部分，而每一个部分负责一个分段目标。这种方法的好处是比较符合人的自然直觉，前面的网络完成前面的任务，后面的网络完成后面的任务。但是这种方法会给人一个额外的负担，使得这种方法并不是最自然的方法。即人需要手动指定网络的哪一部分去完成哪一个阶段的任务，除了直觉和经验之外，很难有更好的指导。而且这种方法很难充分地利用网络容量。另外，在下面，我会展示对一个已经收敛的神经网络的研究，充分说明了一个自然收敛的神经网络，并不是按照这种解耦方法所设想地一步步串行逻辑地计算，而是不同阶段和步骤相当耦合的计算逻辑实现方法。

第二种，可以直接把解耦的标签拼接到最终结果中，如果解耦的阶段步骤更多，也可以非常方便地添加，因为只需要改变输出标签。而且这种方法的好处是不需要选择网络的哪一部分进行哪个阶段的计算，可以更加充分地利用网络容量。

以下我一致地选用第二种解耦方法。

8.1 2 进制数 mod 3 实验

8.1.1 实验部分

非解耦格式的实验我前面已经介绍过：在本文最初采用的实验设置下，模型在合理训练时间内几乎看不到有效的 loss 下降。后续实验显示，当输入位数减小时，该任务并非绝对不可训练，而是训练地形非常困难、收敛所需时间显著增加。正因如此，mod 3 是一个很好的例子：它说明某些规则在描述上很简单，却可能对神经网络的训练动力学非常不友好。下面介绍解耦实验的输入输出格式。

输入格式：30 位二进制数

输出格式：dfa 轨迹，总共 60 位，其中每两位对应输入二进制数的前面部分的 mod 3 结果，例如，1-2 位对应第 1 位二进制数的 mod 3 结果，3-4 位对应第 1 位第 2 位组成的 2 位 2 进制数的 mod 3 结果，那么，很显然地，59-60 位对应整个 30 位二进制数的 mod 3 结果。

解耦格式讨论：这种解耦格式每两位代表一个状态，仅包含三种状态：

- 状态 S0: 代表当前已处理的前缀数值 V ，满足 $V \bmod 3 = 0$ 。如果读入 $b = 0$ ，则 $S_{\text{next}} = (0 * 2 + 0) \bmod 3 = 0$ ，状态转移为 $S0 \rightarrow S0$ ；如果读入 $b = 1$ ，则 $S_{\text{next}} = (0 * 2 + 1) \bmod 3 = 1$ ，状态转移为 $S0 \rightarrow S1$ 。
- 状态 S1: 代表当前已处理的前缀数值 V ，满足 $V \bmod 3 = 1$ 。如果读入 $b = 0$ ，则 $S_{\text{next}} = (1 * 2 + 0) \bmod 3 = 2$ ，状态转移为 $S1 \rightarrow S2$ ；如果读入 $b = 1$ ，则 $S_{\text{next}} = (1 * 2 + 1) \bmod 3 = 0$ ，状态转移为 $S1 \rightarrow S0$ 。
- 状态 S2: 代表当前已处理的前缀数值 V ，满足 $V \bmod 3 = 2$ 。如果读入 $b = 0$ ，则 $S_{\text{next}} = (2 * 2 + 0) \bmod 3 = 1$ ，状态转移为 $S2 \rightarrow S1$ ；如果读入 $b = 1$ ，则 $S_{\text{next}} = (2 * 2 + 1) \bmod 3 = 2$ ，状态转移为 $S2 \rightarrow S2$ 。

训练集脚本：generate_binary_mod3_dfa_explain.py

训练脚本：train_mlp.py

训练结果：使用 mlp 网络，迅速收敛，并在 25epoch 达到 99.96% 的 dfa 序列 exact match。

8.1.2 实验结果讨论

这是个令人非常意外的结果。不仅仅是因为这种解耦格式可以辅助训练，它甚至可以把原来在直接输出设置下长期难以学习的问题变得可以高精度学习。这显然并非单纯网络容量的问题。更令人费解的是，如果这个问题在表示上如此简单，为什么必须加 DFA 辅助序列才能收敛，单独训练最后的结果就不可以？一种可能的解释是，解耦信息改变了训练地形，使梯度更容易把模型引向正确的规则结构。这是一个我目前无法完全回答的问题，但至少在这个实验中，可

以明确看到这种解耦方法的有效性：它虽然增加了输出标签的长度，看起来增加了训练负担，却让训练变得可行，而且速度更快。这是一个非常反直觉的结果。

8.2 接雨水实验的再一次解耦

这一节我针对上面已经解耦过的接雨水实验，再一次解耦，把不同的算法细节暴露出来。

8.2.1 实验部分

输入格式介绍： $N \times \text{BITS_PER_HEIGHT}$ 位二进制数， N 为柱子数， BITS_PER_HEIGHT 为每一个柱子的高度的二进制数位数。

输出格式介绍：算法解耦过程部分 + 结果部分。

结果部分： $N \times \text{BITS_PER_HEIGHT}$ 位二进制数， N 为柱子数， BITS_PER_HEIGHT 为每一个柱子所容纳雨水量的二进制数位数。

算法解耦部分：三种算法思想介绍见[42. 接雨水 - 力扣 \(LeetCode\)](#)。

- **DP 解耦格式：**一个长度为 $2 \times N \times \text{BITS_PER_HEIGHT}$ 的向量。它由两个部分直接拼接而成：前半部分为 `leftMax` 数组的二进制表示，该数组的第 i 个元素代表输入序列中位置 0 到 i 的最高柱子高度；后半部分为 `rightMax` 数组的二进制表示，该数组的第 i 个元素代表输入序列中位置 i 到 $N-1$ 的最高柱子高度。每个高度值均被编码为 BITS_PER_HEIGHT 位二进制数。
- **单调栈解耦格式：**一个长度为 $N \times (2 \times \text{BITS_PER_INDEX} + \text{BITS_PER_HEIGHT})$ 的向量。该向量由 N 个固定大小的“事件槽”拼接而成。每个事件槽记录一个三元组 (`left_idx`, `right_idx`, `top_height`) 的信息：`left_idx` 表示形成“凹槽”的左边界墙索引，`right_idx` 表示形成“凹槽”的右边界墙索引，`top_height` 表示被计算雨水的“槽底”柱子的高度。未被使用的事件槽以 0 填充。
- **双指针解耦格式：**一个长度为 $N \times \text{BITS_PER_HEIGHT}$ 的向量。该向量的第 i 个元素块记录了在计算位置 i 的雨水时，所依据的 `leftMax` 或 `rightMax` 的值。如果位置 i 的雨水是由 `leftMax` 决定的（即，当左指针移动时），则记录当时的 `leftMax` 值；如果是由 `rightMax` 决定的（即，当右指针移动时），则记录当时的 `rightMax` 值。
- 如果位置 i 没有储水或作为边界墙，则其对应的值为 0。

具体参数：

$N=15$, $\text{BITS_PER_HEIGHT}=5$, 在 500000 条的训练集上训练 10 个 epoch

训练集脚本：`generate_rain_water_final_showdown.py`

训练脚本：`train_mlp.py`

实验结果（验证集）：

	DP	单调栈	双指针	无解耦格式
解耦部分 exact match	96.02%	17.36%	95.46%	

	DP	单调栈	双指针	无解耦格式
结果部分 exact match	75.60%	6.20%	85.16%	56.60%

如果只关心结果部分的 exact match，可以看到训练效果从好到差的排序是双指针 > DP > 无解耦格式 > 单调栈

8.2.2 实验结果讨论

可以看到 DP 算法和双指针算法促进了任务的收敛，但单调栈这种算法反而减缓了任务的收敛！首先，某种解耦方法可以促进收敛的事实，再一次证明端到端训练并非一定是最好的选择。自深度学习时代以来，有一种信仰就是，不要引入过多的人工偏置，去发现如何解决问题。我个人认为一方面原因是在感知问题上可能这种做法就是会比较自然有效，另一方面也是因为模式识别范式的问题上，很难给出一个清晰而无信息损失的辅助标签且不引入人工偏置。那么，在我发现的这个范式上，后面的困难完全不存在了，对于复杂的规则问题，可以很简单地给出不同的解耦信息辅助训练。所以，这个实验就很清晰地说明了，如果给出一些解耦作为引导，是完全可以降低任务的学习难度的。而且要注意，这只是一个小的算法问题，就已经可以通过这种手段加速收敛，那么也很容易可以想到如果是一个复杂的多阶段的大规模的规则/算法学习任务，如果不加入解耦步骤信息，只是不停延长训练时间几乎是不会有什么帮助的。

另外单调栈这种解耦方法，反而减缓了任务收敛，这是个非常有意思的现象。根据这种解耦格式的说明，它一般是三种解耦标签中长度最长的。而且描述的不只是信息，还是一个串行算法的执行过程，这和 DP 算法和双指针算法的解耦格式完全不同。它反而会降低任务的收敛速度。这暗示一种可能性：对神经网络而言，拟合单调栈这种串行过程本身的成本，可能超过了它为最终任务带来的收益。所以，最终收敛速度反而甚至不如无解耦辅助标签，直接训练任务目标。

8.3 哲学意义相关讨论

如果把端到端地联结主义学习看做是直觉，虽然上面已经有很多实验这种直觉可以应对很多符号规则相关的任务，但实验也已经证明这种直觉远非完美，不管是从收敛速度来看还是对于某些问题的可学习性来说，加入一些解耦——即中间过程阶段的信息都可能会促进收敛。而我认为这一点的意义远不限于一个单纯的技术实验观察，就像人类也无法仅仅依靠直觉去应对世界，有一些逻辑思考是绝对必要的。而深度学习时代到来以来的对端到端训练神经网络的信任，基本上可以等价地对应于现在流行地所谓“第一性原理思考”或者“解构”，其根源在于很多思想已经过时不能再反映事实，例如上面提到的计算机视觉依赖于手工设计的特征（如 [17] SIFT, [18] HOG），这可以看作是一种对视觉世界的“先验符号知识”。或者前面提到的一串失去上下文的逻辑链条会推出错误的结论，在这个语境下，端到端的训练就类似于重新审视这个问题，不依赖过去的经验。而上面提到的接雨水实验的单调栈算法解耦方法，如果神经网络亦步亦趋地学会了那个单调栈的算法，它是否可以解决接雨水问题？可以！但是，最终会发现是不必要的，因为采用这种解耦方法，甚至比不采用解耦辅助，收敛地还要更慢。这就有点类似于，思想原来是帮助人去更好地解决问题和生活的，但是在有些时候它们会完全脱离语境，失去效力，反而不如一个人根据自己的独立思考来应对——神经网络根据它自己的原生能力应对表现地更好了。但这并没有否认有些解耦方法是会加速训练的，例如上面所举例子 DP 和双指针法。

也许可以把解耦当做一种破坏思维的分布式表示，使其明确表示的操作，类似于精神分析中的潜意识意识化。这种操作在有的场景可以使得纠缠在一起的思维变得清晰，更容易进行思考，在另一些场景则可能因为过度的符号化降低分布式表示的计算效率。

这绝对不是一个简单的矛盾，中国古语有云，学而不思则罔，思而不学则殆。mbti 人格类型中，有偏向于察觉和感知的 p 人，有偏向于做计划和判断的 j 人。强化学习中有 explore 和 exploit 的矛盾。也许人类概念的形成对应于认知经济效用最大化的选择，而随着时间推移和环境改变，认知注定要被一次次解构和建构。思考就要做判断，判断就会出错，未来可能就会被修正，但不做判断可能又更加糟糕了。没有绝对正确的选择，存在的只有某种认知的倾向。

但这个解耦格式，再一次回归了人类的手动选择，它无法自然涌现出来，这似乎违背了深度学习的哲学。那么是否有可能通过神经网络自发形成这个解耦格式信息或者增加对问题结构的洞察来加速训练？我没办法回答这个问题，但是如果神经网络有能力这样做，会出现一个更不可思议的场景，神经网络有能力迅速拟合任意极度复杂的问题，it's to good to be true，个人认为智能本身没有这么简单。

但对于这个问题，我觉得有另一种迂回的方法，即如果同一个神经网络面对多任务同时进行训练，当神经网络被迫在同一个参数空间内，同时解决大量不同但相关的符号任务时，梯度下降的认知经济学压力，会促使它去发现和构建那些跨任务通用的、可复用的中间表示——这有可能和我之前手动设计的解耦格式的相关。举个例子，假如把 imagenet 的猫的图片 and mnist 的 1 的图片分别拿出来合并在一起，要求神经网络分辨这两类，它可能会找到基于纹理或颜色的取巧捷径；而一个需要同时区分 ImageNet 中 1000 个精细类别的网络，则被迫在内部形成了从边缘到部件、再到对象的、可复用的、层次化的抽象表示。所以，深度学习的自发涌现的哲学的延续可能依赖于多任务同时训练。这种对问题结构的洞察力，其诞生的关键，可能并不在于对单一任务的深度钻研，而在于经验的广度。正如我们在人类认知中观察到的，没有经验的广度就很难有认知的深度。当然了，这个设想的正确与否依然取决于未来的实验和理论研究。

9 第九幕：对已经收敛的神经网络的研究

这一节主要目的是研究一个已经对于某个具体问题收敛的神经网络，包括它的计算方式，每一个隐藏层携带的信息等等。使用的神经网络采用 mlp，研究方法主要是在神经网络收敛之后，冻结某个隐藏层，后面接另一个 mlp 网络试图解码一些特定的信息，然后用收敛速度和最终收敛精度，间接判断这些信息是否已经较容易地存在于该隐藏层表示中，以及是否与其他信息过度纠缠在一起。至于为什么不能只以最终收敛精度判断的原因是，在有的情况，如果给充分时间，可能对于很多特定的信息，都可以收敛。比如某些极端情况，在一些隐藏层中甚至可以解码出输入的精确信息，所以神经网络是有可能先解码出输入信息，再拟合指定的特定信息的。但是，在这些情况里，对不同特定信息的收敛的速度经常是不同的，所以只能通过收敛的速度来判断信息在隐藏层中的存在方式。

9.1 通过接雨水实验研究最后一个隐藏层

9.1.1 实验部分

我用 8.2 节的接雨水的结果部分作为标签来训练一个 mlp 网络，等它收敛之后，再提取出最后一个隐藏层，后面接一个新的 mlp 网络，输出指向一些指定的信息，用以探测这个隐藏层里这些信息的存在和存在方式。

这些指定的信息就是 8.2.1 节介绍的三种算法的解耦信息——DP，单调栈，双指针——在此不再赘述。

数据集参数设置：

柱子个数：10

柱子高度占用二进制位数：4

训练脚本：train_mlp_prefer.py

实验结果：

解耦算法	Best exact match
DP	26.90%
单调栈	42.34%
双指针	92.74%

9.1.2 实验结果讨论

这个实验结果表明了神经网络在收敛后，在最后一个隐藏层会有大量任务相关的信息，可以通过探测这些信息来尝试了解神经网络是如何学习这个任务的。而且，神经网络在学习某一个任务时，会形成自发的认知风格倾向，比如在这个问题上，神经网络对这些算法的倾向性：双指针 > 单调栈 > DP。那么，为什么会是这样呢？这是有一个很有趣的问题，不会很容易回答。即使考虑到这些解耦格式并不能完全等同于相应算法，回答这个问题也仍然没那么容易。再加上考虑到单调栈的解耦格式比 DP 的解耦格式实际上是要更长的，这个结果就更有意思了。

9.2 用元胞自动机任务探测一个收敛后的 mlp 的隐藏层

9.2.1 实验部分

实验设置：先拟合 1 维 30 位元胞自动机对 rule 110 进行 8 层演化后的状态。再依次提取每一个隐藏层，对 1 到 8 层演化后的状态进行拟合，使用另一个 mlp 网络，训练同样的 epoch 数，观察 exact match 的数值。

数据集参数设置：

元胞自动机位数：30

演化规则：110

演化次数：8 次

训练脚本：train_mlp_ctscan.py

实验结果：见下表，S_1 EM 代表拟合 1 层演化状态的 exact match

从哪个隐藏层开始拟合	S_1 EM	S_2 EM	S_3 EM	S_4 EM	S_5 EM	S_6 EM	S_7 EM	S_8 EM
H_0	100.00%	100.00%	99.96%	59.20%	3.40%	0.04%	0.00%	0.00%
H_1	100.00%	100.00%	99.90%	90.12%	76.56%	33.72%	1.50%	0.24%
H_2	95.02%	86.64%	72.88%	65.08%	28.02%	11.42%	8.26%	99.88%
H_3	32.92%	29.18%	31.52%	35.40%	21.24%	21.90%	41.08%	100.00%

9.2.2 实验结果讨论

根据上面的实验结果图表，可以得出如下几个定性结论：

- 直接从 H_0/H_1 开始拟合，演化层数越多越难拟合，这很容易想到也很合理，因为 H_0/H_1 非常接近于初始输入，随着问题难度的增加，自然会越来越难拟合
- 拟合最终态的难度，随着 H_0 到 H_3 逐渐降低，这个也很合理，如果把神经网络看做某种分布式计算的方式，整个神经网络计算的总和是拟合这个任务。而前面的神经网络层已经承担了很多计算，使得越向后的隐藏层表示越接近最终需要拟合的目标。
- 从 H_0 到 H_3，对于 S_1/S_2/S_3 的拟合能力逐渐降低，对 S_4/S_5/S_6 的拟合能力基本遵循先升高再降低（对于 S_6 来说 H_2 似乎是个例外，但不影响最终结论），对于 S_7/S_8 的拟合能力逐渐增加。这是个很有趣的现象，表明随着隐藏层从神经网络的前面向后，其所蕴含的信息越来越接近最终的任务，逐渐开始丢失最初的输入信息和浅层演化的状态信息，但又没有完全丢失，因为这些数据只是在非常小的 epoch 和小规模的 mlp 得出的。下面将看到，对于一个演化层数较小的元胞自动机，甚至可以从已收敛的神经网络的最后一个隐藏层中恢复完整的演化信息，包括完整的输入。
- 另外可以得出的结论是，神经网络的计算方式也并不是按照 rule 110 一层层的明确的演化逻辑，因为如果是这样的话，H_3 或 H_2 对 S_7 的拟合 exact match 将会同样非常高。实验结果表明，H_3 对于 S_7 远远没有想象中容易拟合，比如实际上 H_3 对 S_7 拟合的 exact match 仅比 H_3 对 S_1 拟合的 exact match 高不到 10%。所以，整个神经网络只是指向 S_8 的拟合，而 S_7 并没有自然地占据某种特殊的地位。

9.3 从最后一个隐藏层解码元胞自动机的完整演化序列

9.3.1 实验部分

实验设置：先拟合 1 维 30 位元胞自动机对 rule 110 进行 4 层演化后的状态，训练 15 个 epoch。再提取最后一个隐藏层，使用另一个 mlp 网络，对 1 到 4 层演化后的状态拼接后的状态进行拟合，观察 exact match 的数值。

数据集参数设置：

元胞自动机位数：30

演化规则：110

演化次数：4

训练脚本：train_mlp_fulltrace.py

实验结果：在进行充分训练后，可以达到 exact match 100%，即在验证集上对于元胞自动机的演化序列达到 100% 的序列准确率。

9.3.2 实验结果讨论

这个实验结果表明，最后一个隐藏层里确实包含大量与任务演化序列相关的信息。对于演化层数较少、难度较低的任务，这些信息甚至可以被完整恢复；而前面的探针实验也显示，在更复杂的任务中，隐藏层同样包含不同层次的演化状态信息，只是这些信息的可恢复程度和纠缠方式会随隐藏层位置和任务难度而变化。这个性质可能并不局限于最后一个隐藏层，但越靠后的隐藏层可能越包含对应于任务目标的高层语义。

10 第十幕：一些更多的探索

这一节简述一些额外探索，仅作为补充观察，不做过多深入讨论和分析。

10.1 训练可视化

在前面输出是图像的那些实验中，已经展示可以通过定期的 eval 过程中的 eval image 来可视化学习过程，那么如何在一般任务这么做呢？我发现只要在每次 eval 时对每个 bit 分别进行 eval loss 和 accuracy 统计，就可以完成比较好的可视化，以研究神经网络的学习过程。

当这种方法用在二进制加法上时，发现了两种现象，一种是容易想到地串行学习，即从低位到高位准确率逐渐上升，因为加法很难在无法知道低位准确结果的时候预测高位结果，另一种现象是在结果的最高位也收敛地相对较快，分析可能是发现了某种启发式的简单模式。

当这种方法用在元胞自动机实验上时，发现几乎每一位都在同步地收敛，元胞自动机的每一位的收敛程度在每一次 eval 时都差不多。

训练脚本：train_mlp_visualize.py

10.2 自回归

令人有些意外的是，使用 decoder-only transformer，也可以拟合元胞自动机数据集，只是把它的格式完全改成了字符串，例如：

1011...1110 -> 1010...0001

然后从箭头后面的任意位置预测下一个 token（元胞自动机的某一位输出），实验证明是可以收敛的。但问题是，这个实验可能其实并未脱离这个范式，想象预测输出的每一位其实都是一种任务类型，而神经网络可以通过输入格式分辨出是哪一种任务，并学会预测。这其实只是一个带有分支的符号规则学习。至于目前的大模型范式，是否可以精确无幻觉地演化元胞自动机到一个比较深的层数，我没办法给出确信的答案，但倾向于不太可能在不借助工具的情况下做到，而且在实际测试中也确实出错了，正如大模型不能在不借助工具的情况下进行可靠的大数乘法等等。

数据集脚本：generate_ca_text_format_dataset.py 训练脚本：train_ar_transformer.py

11 第十一幕：理论解释

本节尝试对这些实验现象做一个粗糙的唯象分析。如果暂且不考虑后面做的一些尝试性实验，也暂时忽略只有加上解耦信息才能收敛的实验，我们可以把它总结为这样一个事实：给定相

当于任务空间的极小量的数据，对于绝大部分任务，神经网络都可以通用地拟合到背后的精确规则。这里我们可能需要对任务做一个精确的定义，考虑到我在构思和选择任务的时候并没有什么限制，它真正的限制就是 python 程序编写的输入到输出的转换。那么现在考虑这个事实，神经网络可以通过极小量数据，通用地拟合到背后的精确规则，这意味着什么？首先，我们可以看到，因为给定训练集数据是极小量的，因此它是一个极度欠定的问题，因为对应于给定的训练集其实可以对应几乎无限的规则。而且考虑到这样一个实验观测，即训练集样本需要一定数量才会收敛，我们就可以把训练集样本增加的过程，看成约束增加的过程，每个样本都是一个约束，而神经网络的训练类似于解某个方程，或者也可以看成随着样本增加贝叶斯信念更新的过程。这样，只有当样本数目多于一定量时，基于样本的约束和神经网络解的方程形式，解开始稳定，基于贝叶斯的信念更新视角，后验也逐渐变窄。那么问题在于，为什么神经网络总是能够选择到我们背后生成训练集的真实规则？我们现在只能得出这个结论，即神经网络有其内在喜好或者偏向，神经网络在解的方程也是按照某个喜好或者偏向来解，贝叶斯信念更新也是，那么这个喜好或者偏向是什么？考虑到我们的实验结果，神经网络始终如一地拟合到背后的真实规则，他们中的共同点就是这是描述数据集最简单的方式，而因为描述或者压缩训练集的最好方法就是找到这个逻辑变换规则。所以我们可以得出，在理想训练数据下，神经网络通过 SGD 寻找平坦极小值，隐式地执行 [25] Kolmogorov 压缩，或寻找最短程序描述，即 [26] mdl 原则。

这是一个很强的假设，因此我们需要给这篇论文重点研究的规则学习任务一个尽可能精确的定义，简单来说，可以定义为程序化生成的确定性变换

可以将本研究关注的规则，形式化为一类特殊的函数，其核心特征源于其生成过程：- 源代码定义：不由数据点定义，而是由一段确定性的、有限长度的计算机程序定义。这意味着 Kolmogorov 复杂度远小于其输入空间的大小。- 零熵确定性：对于任意输入，程序生成的输出是唯一的。这消除了偶然和不确定性，使得理论上的最小损失可以达到绝对的零，也因此具有算法可压缩性。

下面我们考虑具体的支持证据

11.1 理论假设和证据

需要强调的是，这种现象并不依赖复杂的专门架构。后期许多关键实验已经可以由朴素 MLP 完成：模型没有外部符号系统、没有显式算法模块，也没有人为设计的推理结构，只是由参数、非线性变换和梯度下降构成。这使得本文的观察更接近于联结主义系统本身的性质，而不是某种特定架构技巧的结果。

首先，我们在大量实验中观测到，随着任务难度的增加，训练集需要支持神经网络拟合的样本数越多，这正好和 MDL 原则是一致的，因为复杂的规则需要更多的样本来覆盖模式。而且观察到随着任务难度的增加，需要训练的时间也越长。我们可以唯象地去理解为什么神经网络在学习规则：神经网络在学习规则和记忆样本之间做选择。对于符号规则来说：规则本身的复杂度是恒定的且相对较低。然而，记忆的难度随着样本量增加而增加。一旦样本量超过某个很低的阈值，拟合规则就变得比记忆更“便宜”，从而迫使模型选择泛化。与模式识别的对比：在传统任务（如图像）中，学习是从粗粒度到细粒度的。到了训练后期，挖掘极致细节的难度超过了记忆的难度，导致模型转向记忆和过拟合。而关于记忆压力的迅速增长，也许可以这样解释：符号数据的输入输出是离散的阶跃跳变，缺乏局部平滑性，即输入变一位，输出可能全变，这种崎岖的拓扑结构使得基于插值的记忆成本极高。我们也许可以把它叫做数据的内生正则化。

由此可以提出一个关于样本量与泛化过程的三阶段假设：当数据量严重不足时，训练集约束不足以排除大量记忆型或偶然型解，模型会长期停留在过拟合状态；当数据量达到某个中间区间时，模型可能先记忆训练集，随后在 SGD 隐式正则化作用下发生 grokking，验证集准确率突然上升到接近 100%，且根据 grokking 相关实验观察，这一转变速度通常会随着数据量增加而加快；当数据量进一步充足、模式覆盖足够充分时，训练 loss 和验证 loss 可以同步下降，不再表现出明显的先过拟合后泛化阶段。这正对应本文大量实验中观察到的理想训练情形。

如果以上面的理论来考虑这个问题，当样本数少于拟合规则需要的样本数时，神经网络训练中表现出过拟合，但这里会有一个问题，如果我们理解神经网络在训练时发现记忆更容易就选择记忆，发现拟合规则更容易就选择拟合规则，这个选择的相变点和上面提到的按照 MDL 原则，复杂的规则需要更多的样本来覆盖模式这样的相变点，这两个相变点是一致的吗？我认为很可能是一致的，在微观的神经网络动力学上，这表现为梯度方向和大小，唯象的理论上可能表现为记忆和拟合规则的难度比较，宏观上可能对应于 MDL 的描述长度原则：如果训练样本较少，记忆或者偏向于记忆的拟合方式可能小于规则的描述长度。而这三种视角应该对应于同一件事物，它们之间应该完全契合。

再从 [34] 隐式正则化理论角度思考，随机梯度下降的特性使其隐式地偏好平坦区域：优化轨迹持续逃离尖锐的极小值，因为易因扰动而失效，最终沉降在平坦的极小值区域。权重空间中，平坦极小值（误差近似恒定的宽阔区域）允许权重以低精度存储，而尖锐极小值（狭窄谷底）要求高精度。从 MDL 视角，前者对应短程序描述（仅需少量比特定位“盆地内的任意点”），后者对应长程序（需精确坐标）。或者从另一个角度思考，规则解的梯度在不同训练集样本中是相干的，会互相加强，而非规则解或者说记忆解在不同训练集样本中则是不相干的，会互相抵消。这解释了我们的观察，因此，SGD 通过寻找平坦极小值，实现了对最短程序描述的隐式优化。这在统计层面是某种不可避免的现象。SGD 的随机性如同在能量景观中扰动的小球，因其自由能最小化倾向——同时追求低能量（低损失）与高熵（宽平坦区域，允许低精度存储）——必然沉降于平坦盆地。这恰是非平衡态热力学中耗散结构的形成：网络作为开放系统，通过持续耗散梯度能量，自发从高熵混沌涌现低熵有序的规则结构。其统计本质遵循自由能最小化原理——系统在约束条件下自动选择复杂度代价最低的路径，使智能成为非平衡条件下能量——信息转换的必然涌现。

从这一角度看，神经网络训练可以被启发性地理解为一个自由能最小化过程：损失函数对应能量项，平坦盆地对应熵项，而 SGD 的随机性则使优化过程不仅追求低损失，也偏好更宽、更稳定、描述代价更低的参数区域。换言之，训练并不只是寻找任意一个能拟合训练集的解，而是在能量、熵和复杂度之间进行某种隐式权衡。这个热力学类比并不等同于严格的物理推导，但它明确提示了一个可能的理论方向：神经网络训练也许可以被理解为能量、信息与结构压缩之间的转换过程。

最后，如果和压缩即智能这个概念联系起来的话，结合 [32] 2023 年 ilya 的演讲和 [33] jack rae 的线上演讲，他们的主要思想是，最优的无监督学习就是执行常规 Kolmogorov 压缩（而非以数据为条件的压缩），而神经网络 +SGD 是我们目前逼近不可计算的 Kolmogorov 压缩器的最佳实用方法。在这里，我们的实验提供了这一理论的最纯粹检验方法，由于使用程序化生成的理想数据是零噪声的，我们消除了真实世界数据中不可避免的不可约误差，因此神经网络的训练就是在选择用什么样的方式去压缩训练集中从输入到输出的映射。而我们的大量实验也证

明了，给定足够的训练样本，神经网络将会一致地拟合到真实的规则，这给了以上命题的强烈实验证据。这与“压缩即智能”的核心论断形成呼应：当神经网络在零噪声条件下选择生成规则的短程序描述时，它正是在执行对数据的无损压缩——不是表面的统计模式匹配，而是对底层因果机制的压缩编码。这种压缩的极限，即最小程序长度，正是 [24] Solomonoff 归纳所追求的目标。

11.2 相关实验证据

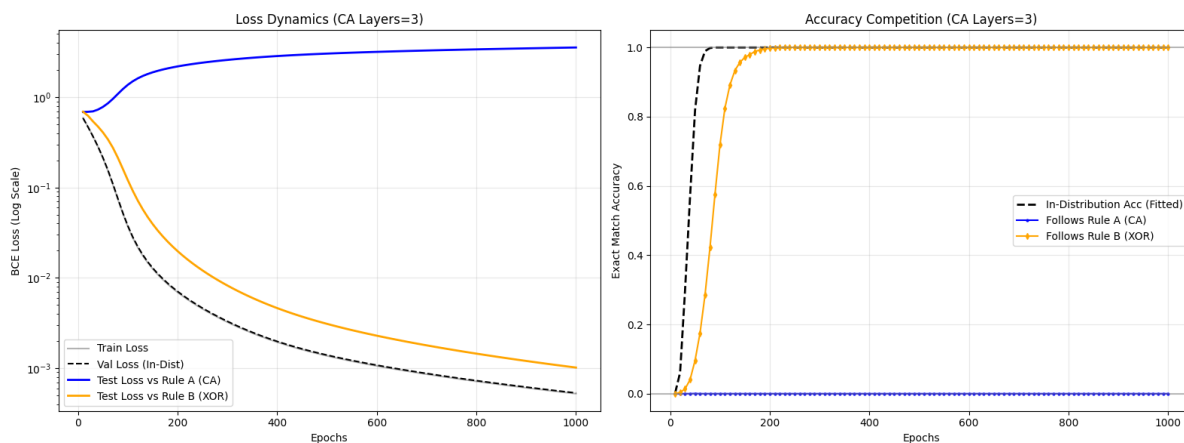
这部分相关脚本见代码仓库根目录下的 `research` 子目录，极端过参数化实验的参考根目录下的 `train_mlp.py`，隐藏层改为 8192

11.2.1 [27] ntk 相关实验

我用一个实验脚本粗略地测试我的发现是否只涉及 ntk 理论中的懒惰学习，答案是否定的，对于一个 1 维 30 位元胞自动机，演化 3 层 rule110 规则的任务，训练集 30000 样本。解析求解 ntk 的泛化结果，在测试集只有 1.4% 精确匹配准确率，而以同样的训练集，真正进行神经网络训练则可以达到 100% 的测试集精确匹配准确率。这不是误差可以解释的，这样的对比在其他的一些实验也得到了证实。

11.2.2 规则偏好实验

在这个实验中，我设计了在训练集样本中共存两种规则都可以从输入精确推出输出，然后在测试集测试神经网络内部到底更倾向于那种规则。实验显示当两种规则难度差距过大时，训练一段时间后神经网络会完全接受其中一种规则而抛弃另一种规则。这明确地证明了神经网络选择简单规则的倾向。另外，有趣地是，分布内验证集的拟合速度更快，提前达到 100% 准确率，注意这不是记忆，因为和训练集不重合，此时应该是学到了同时利用两种规则 and 对应数据来预测的某种纠缠态。此后一段时间，神经网络才完全转向更简单的那个规则，从利用两种规则和数据的纠缠态，完全转向更简单的规则，在这个规则的测试集准确率才达到 100%。这其实就是 [28] grokking 现象！而这也说明了，神经网络即使在完全拟合训练集后，SGD 的隐式正则化仍在驱动网络权重向更简单的规则漂移。因此，SGD 通过寻找平坦极小值，实现了对最短程序描述的隐式优化。



如下图所示而当两个任务难度差距不大时，即元胞自动机演化层数为 1 层或 2 层时，两种规则都在分布外泛化很差，呈现一种规则的纠缠态。

11.2.3 极端过参数化实验

给定一个 269M 的朴素 mlp 网络，让它拟合一些极小样本的训练集，仍然不会过拟合。包括，对于 1 维 30 位元胞自动机，演化 1 层 rule110，只需要 1500 样本可以保证不过拟合，演化 2 层 rule110，只需要 4500 样本可以保证不过拟合，等等其他一些实验。这样极端的实验配置，让记忆训练集样本变得极其容易，但却没有过拟合，挑战了大模型 + 小数据 = 过拟合的常识，并支持了深度神经网络作为 MDL 求解器的假设，即神经网络在寻找一个最短程序描述。

11.2.4 规则 ood 泛化实验

相关有 3 个实验，第一个实验的输入是元胞自动机的初始状态，以及元胞自动机的演化规则，默认演化 2 层。第二个实验加上一个规则 bit 控制演化 1 层还是 2 层。第 3 个实验加上一个 bit 控制输出是否右移 15 位。实验方式是，元胞自动机的初始状态随机采样，但规则部分始终限制在所有可能规则的一部分随机采样，而这些部分规则本身也是在所有可能规则随机采样出来的，对于一个数据集保持一致，占总体比例分别取了 90%，70%，70%。实验现象是，在同分布验证集很快收敛了，即在限制内的规则很快收敛，但是如果继续训练，会发现验证集内从未在训练集中出现过的规则也收敛了。

这个实验可以再一次证明，神经网络即使在分布内收敛后仍然会持续向更简单的算法描述漂移，这样才能解释对于分布外规则的泛化——神经网络认为对于规则 bit 结构性的理解对应于更简单的算法描述。

11.3 另外一些思考

- 关于 MDL 的本质的进一步思考，神经网络可能是进行某种逻辑的自动化简，以我在这个论文中的各种实验为例，很有可能在无意引入可以化简的逻辑，比如举个 trivial 的例子，假如我的程序在计算 $(x+1)+(x-1)$ ，那么神经网络接受到的数据只是 $2x$ 而已，它只能看到数据，所以它也只会对数据进行拟合，至于描述数据有多少种等价逻辑，这是神经网络不知道也不关心的。但程序的逻辑却是 $(x+1)+(x-1)$ ，可能人的理解也是和程序一样。这个只是拿来打个比方，实际情况要复杂得多，这种情况更可能发生在叠加很多层逻辑步骤的情况下，会出现某种可化简的情况或者逻辑捷径，这个时候神经网络会注意到并进行化简。更实际的例子则是拿我做的实验来说，我并未给神经网络指定（当然也不可能）哪个输入对应什么含义，哪个输出对应什么含义，写程序的时候这些多半是我自己的任意指定以及为了方便神经网络训练，但神经网络仍然拟合了。在这里，我们没有理由认为神经网络拟合的逻辑和人类的逻辑或者数据集生成的逻辑一致，所以，如果存在某种逻辑捷径，神经网络可能就会利用它。注意，这里的捷径是指某种逻辑化简，而非和原有逻辑不同的 [35] 捷径学习。而且，我认为情况不止于此，考虑到神经网络承载逻辑的方式是分布式表示和分布式计算，我认为它可能会把人类看来不可化简的逻辑以它自己的形式进一步化简并自组织地分配到神经网络权重的不同位置，以满足自身的 MDL 原则。因此从这个视角看，存在这样一种可能性：寻找神经网络计算过程与人类形式逻辑之间的精确一一对应，可能

本来就是不可能的，因为神经网络的计算和表示并非人类可理解的形式逻辑，把网络权重和人类可理解的概念进行初步关联是可能的，但如果一定要进行精确对应，转变成形式逻辑，神经网络将会马上失去它的优势和本质，它也不再是神经网络。

- 我在 7.3 节探讨的 mnist 和元胞自动机结合的实验，暗示模式识别和规则学习可能都是神经网络的原生能力，甚至可能是同一种能力。那么，一个很自然的推论就是，我们上述对规则学习的解释，即神经网络在寻求最短程序描述很有可能同时适用于我们熟悉的模式识别任务，但这个假设需要进一步实验验证和理论研究。
- 关于分布外泛化的问题，首先我的实验网络结构的输入和输出的格式和位数都是不变的，因此不可能扩展到更多的输入或输出位数，也因此不具有真正的符号主义的可组合的特点。另外一种情况，如果网络结构不变的话，那在分布外泛化的表现主要由 mdl 原则决定，即由训练数据分布和 mdl 原则共同决定的。
- 规则 OOD 泛化实验还引出一个更深的问题：在什么条件下，采用符号主义表示反而是更低 Kolmogorov 复杂度的解释？如果模型只是把每个规则当作独立模式来记忆，那么它无法稳定泛化到未见规则；只有当它以某种方式把规则 bit 理解为规则表、逻辑条件或变换程序的一部分时，才可能在 OOD 规则上泛化。这说明，在某些数据分布下，符号主义表示并不是外部强加的先验，而可能正是对数据生成机制的最短描述。问题在于，当数据少到什么程度时，直接记忆或直觉式拟合更“便宜”；而当数据增加到什么程度时，抽象出符号规则反而成为更低复杂度的解？这个问题可能触及符号主义与联结主义、逻辑与直觉之间更深层的关系。
- 一个关键的问题，神经网络的偏好与 MDL 原则是完全等同的吗？可能需要更精确地定义：传统的“简单”通常指描述长度更短，Kolmogorov 复杂度更低，但神经网络的“简单”更可能是动力学意义上的简单——即 SGD 轨迹易于到达且稳定的平坦极小值。如果是这样的话，那么微观的神经网络动力学，如何对应到宏观的 mdl 原则，可能是一个需要认真探讨的问题。mod3 等任务的困难或许正揭示了这种差异：它在描述长度上很简单，却可能处于 SGD 动力学难以到达的区域。目前似乎没有太多不经过实际的实验研究神经网络性质的方法，所以未来可能需要更多实验，建立相关的动力学理论体系，以及研究如何和宏观的 mdl 原则对应起来。所以，目前把神经网络和梯度下降视作一个对于 Kolmogorov 压缩的实用逼近方法，这个假设可能比较合适。

11.4 相关工作

- 与 Grokking 的联系：前面的规则偏好实验是和 grokking 实验高度同构的，也暗示了 grokking 现象的一种可能解释，即神经网络在拟合训练集后，SGD 的隐式正则化仍在驱动网络权重向更简单的规则漂移。因此，SGD 通过寻找平坦极小值，实现了对最短程序描述的隐式优化。而我的大量实验也证明了，对于良好的数据集，包括足够的数据量和适中难度的任务，完全不需要经过一个先过拟合后转向泛化的过程，神经网络的训练过程会直接表现出 train loss 和 val loss 同步下降，val loss 始终略低的情形，直到达到精确的拟合。
- [29] Othello-GPT：他们的实验表明，在限定长度内，自回归的方式也可以做到不错的精度，这种实验可以解释为 transformer 在维护一个世界模型，也可以解释为是在进行一个

多任务学习，不同任务对应不同的输入长度。假如另外一个类似的游戏允许推广到更长的变长输入，是否还可以保持高精度的预测，这可能暂时还是个问题。另外，我的隐藏层探针实验也和这篇论文的探针实验有相似之处。

- Amortized Planning with Large-Scale Transformers: A Case Study on Chess: 这篇论文启发了我的研究，他们也得出一个重要结论，transformer 可能是通用算法近似器。我现在的研究表明，类似能力可能不仅限于 Transformer，也不仅限于近似；在某些受控任务中，标准神经网络可以接近精确地拟合并执行算法映射。而且这篇论文所研究的问题也非常有趣，让 transformer 直接拟合下国际象棋的算法取得了显著的成功，这表明作为算法拟合器，神经网络可以做到的远不止于我在这篇论文重点研究的那些离散规则变换。
- [30] A Generalist Neural Algorithmic Learner: 我的发现表明不需要特定架构和中间信号监督，也可以完成一些算法的拟合。对于这些任务，算法的中间步骤事实上可以在内部自发涌现。
- [31] What Can Neural Networks Reason About?: 这篇论文强调不同的架构对于不同的问题有不同的适应性，我的发现并没有否认这一点，但我的论文主要是想要借着通用的神经网络架构对于不同理想数据的拟合能力，来引发神经网络的某种相变，从而探讨神经网络本身的性质，而非看重特定神经网络架构在解决特定任务的效率。

12 第十二幕：从炼金术到科学的深度学习

本研究的终点，为一个困扰人工智能领域数十年的根本性问题，提供了一个清晰而深刻的答案：一个为概率与统计而生的神经网络，是否能够真正理解并执行精确的、确定性的逻辑法则？

实验结果强烈暗示这是肯定的。而实现这一点的关键，并非依赖于更庞大、更复杂的模型架构，而是源于一个根本性的实验转向：从嘈杂现实数据转向由纯粹规则程序化生成的理想数据；从脆弱的自回归预测链，转向并行的、一步到位的整体求解框架。

本文并未止步于展示某一特定先进架构的优越性，而是进一步进行了系统性的架构测试，并得到一个出乎意料、但又符合第一性原理的发现：一个没有任何特殊结构偏置、纯粹由参数和非线性层构成的多层感知机，也能在复杂的逻辑演化任务上展现出显著的、接近零误差的泛化能力。而循环神经网络的成功，则将这一发现推向了更深层的理论问题——它暗示神经网络甚至能够为了满足最终的数学约束，而形成一种人类无法直观理解、但在功能上完全正确的迭代过程。

这一系列实验共同指向一个强烈的判断：精确的、算法性的规则学习能力，并非任何特定架构的“专利”，而很可能是联结主义系统在合适数据条件下能够显露出的普遍潜能。架构的差异并不只决定“能与不能”，也决定这种潜能被显露出来的效率、稳定性与能力边界。

更重要的是，这些实验显示，神经网络的能力边界可能比通常印象中更宽。它们不仅能够感知任务中进行模式识别，也能够理想条件下学习接近精确的规则映射；不仅能够拟合连续图像和自然数据中的统计结构，也能够逼近离散、确定、可验证的算法关系。进一步的感知-规则复合实验还暗示，模式识别与规则执行可能不是两种彼此外接的能力，而可能是同一种联结主义学习能力在不同数据形式下的表现。

从理论解释上看，一方面，本文的实验观测为压缩即智能假设提供了直接的实验支撑，另

一方面，压缩即智能也能很好地解释本文的核心实验现象：如果只考虑训练集拟合，模型本可以选择许多同样能很好拟合训练样本的复杂程序，甚至仅仅记忆训练集，但实验中，神经网络在足够数据约束下，总是稳定地收敛于背后的真实规则，这说明训练动力学存在某种偏向低复杂度解释的机制。由此，规则学习、模式识别和智能本身，都可以在压缩与最小描述长度的框架下被重新理解。

因此，本文的主要意义在于为未来 AI 研究指出一个值得认真对待的实验事实：神经网络也许并不只是概率性的模仿器或统计模式识别器，在合适的数据与训练条件下，它们可以显露出更接近规则学习、算法执行和结构压缩的能力。这一事实可能会影响我们理解神经网络、构建可靠 AI、研究可解释性，以及重新思考智能、压缩与规则之间的关系。

关于本报告的性质

本文是作者作为独立研究者完成的第一版系统整理。本文覆盖符号规则、算法任务、元胞自动机、结构化图像变换、架构对比和感知-规则复合任务等多个方向。当前版本的重点是报告这一系列实验现象、给出可继续验证的代码入口，并提出一种初步解释框架；它并不试图完成封闭的最终理论。由于实验范围较广，部分实验尚未在完全统一的条件下重新复跑，完整指标记录、严格复现实验和更系统的相关工作对比将在后续版本中继续补充。作者欢迎建设性反馈与合作。

致谢

首先，作者由衷感谢家人在过去数月高强度实验与论文撰写期间给予的坚定支持与理解，没有他们的包容，这项工作将无法完成。

同时，本研究的顺利进行在很大程度上得益于前沿大型语言模型所提供的强大助力。在整个研究过程中，从早期的 GPT-4o 到中后期的 Gemini 2.5 Pro 与 Kimi K2，再到后期论文撰写与深度理论探讨阶段的 Gemini 3.0 Pro 及 Kimi K2.5，这些人工智能助手在数据集生成脚本的编写、实验思路的探讨、数学推导的验证以及论文结构的优化过程中，均扮演了不可或缺的合作伙伴角色，极大地加速了研究的迭代进程。

特别地，作者需要感谢 DeepMind 团队的工作 [16] Amortized Planning with Large-Scale Transformers: A Case Study on Chess。该研究启发作者将模型的输出范式从自回归生成转向整体分类（Amortized Inference），这一转变是解锁模型在巨大问题空间内泛化能力的关键突破口，并直接促使了本研究对神经网络符号学习能力的系统性探索。

最后，本文作为第一版系统整理，仍可能存在计算、表述或细节上的疏漏，作者将非常感谢读者们的批评与指正。

13 参考文献

1. Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks[C]//Proc. of the 25th International Conference on Neural Information Processing Systems. Lake Tahoe, NV: Curran Associates, 2012: 1097-1105.

2. Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Proc. of the 31st International Conference on Neural Information Processing Systems. Long Beach, CA: Curran Associates, 2017: 6000-6010.
3. Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16×16 words: Transformers for image recognition at scale[C]//Proc. of the 9th International Conference on Learning Representations. 2021.
4. Liu Z, Lin Y, Cao Y, et al. Swin Transformer: Hierarchical vision transformer using shifted windows[C]//Proc. of the IEEE/CVF International Conference on Computer Vision. Montreal, QC: IEEE, 2021: 9992-10002.
5. Liu Z, Mao H, Wu C Y, et al. A ConvNet for the 2020s[C]//Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. New Orleans, LA: IEEE, 2022: 11976-11986.
6. Graves A, Wayne G, Danihelka I. Neural Turing machines[arXiv]. arXiv preprint arXiv:1410.5401, 2014.
7. Graves A, Wayne G, Reynolds M, et al. Hybrid computing using a neural network with dynamic external memory[J]. Nature, 2016, 538(7626): 471-476.
8. Rocktäschel T, Riedel S. End-to-end differentiable proving[C]//Proc. of the 31st International Conference on Neural Information Processing Systems. Long Beach, CA: Curran Associates, 2017.
9. Manhaeve R, Dumancic S, Kimmig A, et al. DeepProbLog: Neural probabilistic logic programming[C]//Proc. of the 32nd International Conference on Neural Information Processing Systems. Montréal, QC: Curran Associates, 2018: 3749-3759.
10. Battaglia P W, Hamrick J B, Bapst V, et al. Relational inductive biases, deep learning, and graph networks[arXiv]. arXiv preprint arXiv:1806.01261, 2018.
11. Andreas J, Rohrbach M, Darrell T, et al. Neural module networks[C]//Proc. of the IEEE Conference on Computer Vision and Pattern Recognition. Las Vegas, NV: IEEE, 2016: 39-48.
12. Chollet F. On the measure of intelligence[arXiv]. arXiv preprint arXiv:1911.01547, 2019.
13. Chollet F, Knoop M, Kamradt G, et al. ARC-AGI-2: A new challenge for frontier AI reasoning systems[arXiv]. arXiv preprint arXiv:2505.11831, 2025.
14. Shojaei P, Mirzadeh I, Alizadeh K, et al. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity[arXiv]. arXiv preprint arXiv:2506.06941, 2025.
15. Wolfram S. A new kind of science[M]. Champaign, IL: Wolfram Media, 2002: 1-1197. ISBN 1-57955-008-8.
16. Ruoss A, Deletang G, Medapati S, et al. Amortized planning with large-scale transformers: A case study on chess[arXiv]. arXiv preprint arXiv:2402.04494, 2024.
17. Lowe D G. Distinctive image features from scale-invariant keypoints[J]. International Journal of Computer Vision, 2004, 60(2): 91-110. DOI:10.1023/B:VISI.0000029664.99615.94.

18. Dalal N, Triggs B. Histograms of oriented gradients for human detection[C]//Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition. San Diego, CA: IEEE, 2005: 886-893. DOI:10.1109/CVPR.2005.177.
 19. LeetCode. Trapping rain water[EB/OL]. [2026-02-04]. <https://leetcode.cn/problems/trapping-rain-water/description/>.
 20. Yang A, Yang B, Hui B, et al. Qwen2 technical report[arXiv]. arXiv preprint arXiv:2407.10671, 2024.
 21. Karpathy A. Software 2.0[EB/OL]. (2017-11-11)[2026-02-04]. <https://karpathy.medium.com/software-2-0-a64152b37c35>.
 22. Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Proc. of the 35th International Conference on Neural Information Processing Systems. New Orleans, LA: Curran Associates, 2022: 24824-24837.
 23. Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]//Proc. of the 26th Annual International Conference on Machine Learning. Montreal, QC: ACM, 2009: 41-48.
 24. Solomonoff R J. A formal theory of inductive inference. Part I[J]. Information and Control, 1964, 7(1): 1-22.
 25. Kolmogorov A N. Three approaches to the quantitative definition of information[J]. Problems of Information Transmission, 1965, 1(1): 1-7.
 26. Rissanen J. Modeling by shortest data description[J]. Automatica, 1978, 14(5): 465-471.
 27. Jacot A, Gabriel F, Hongler C. Neural tangent kernel: Convergence and generalization in neural networks[C]//Proc. of the 32nd International Conference on Neural Information Processing Systems. Montréal, QC: Curran Associates, 2018: 8571-8580.
 28. Power A, Burda Y, Edwards H, et al. Grokking: Generalization beyond overfitting on small algorithmic datasets[arXiv]. arXiv:2201.02177, 2022.
 29. Li K, Hopkins A K, Bau D, et al. Emergent world representations: Exploring a sequence model trained on a synthetic task[arXiv]. arXiv:2210.13382, 2022.
 30. Ibarz B, Kurin V, Papamakarios G, et al. A generalist neural algorithmic learner[C]//Proc. of the 1st Learning on Graphs Conference. Virtual Event: PMLR, 2022.
 31. Xu K, Li J, Zhang M, et al. What can neural networks reason about?[C]//Proc. of the 8th International Conference on Learning Representations. 2020.
 32. Sutskever I. An observation on generalization[EB/OL]. (2023-08-14)[2026-02-04]. https://www.youtube.com/watch?v=AKMuA_TVz3A.
 33. Rae J. Compression for AGI[EB/OL]. (2023-02-27)[2026-02-04]. <https://www.youtube.com/watch?v=dO4TPJkeaaU>.
 34. Hochreiter S, Schmidhuber J. Flat minima[J]. Neural Computation, 1997, 9(1): 1-42.
 35. Geirhos R, Jacobsen J H, Michaelis C, et al. Shortcut learning in deep neural networks[J]. Nature Machine Intelligence, 2020, 2(11): 665-673.
-

14 附录

14.1 这里列出了一些有趣的实验图片

学习中结果的可视化

任务描述：输入图像是白色背景，两个任意不重合的绿色实心三角形，要求画一条红色线段，指示两个三角形最近距离。

分析：似乎模型学到了最近距离的线段不是在顶点到另一个三角形对应边的垂线，就是两个顶点间的连线，但是在此处它很难确定哪个出现距离更近，因此就两个垂线都画了，但都是模糊的。左边是输入图像，中间是模型生成图像，右边是 ground truth。

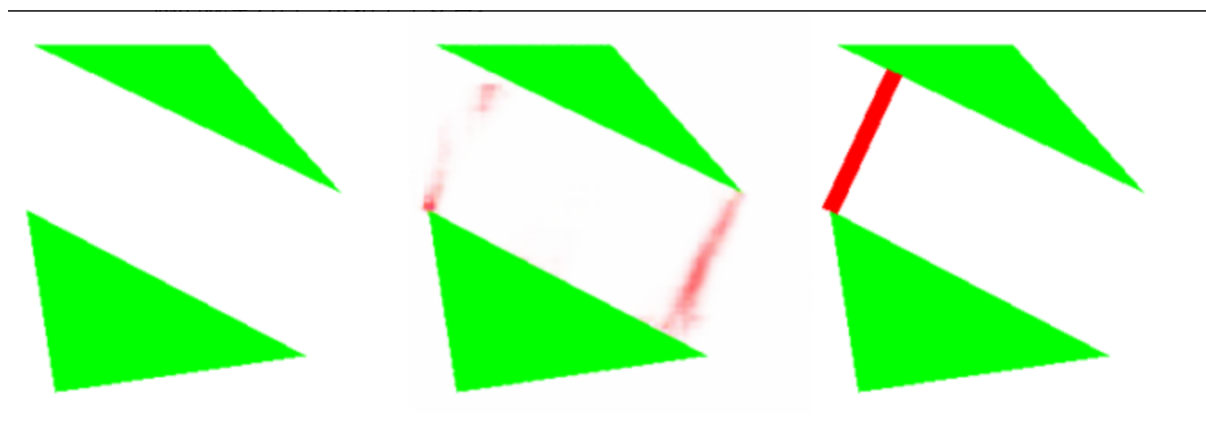


图 104: 学习中结果的可视化

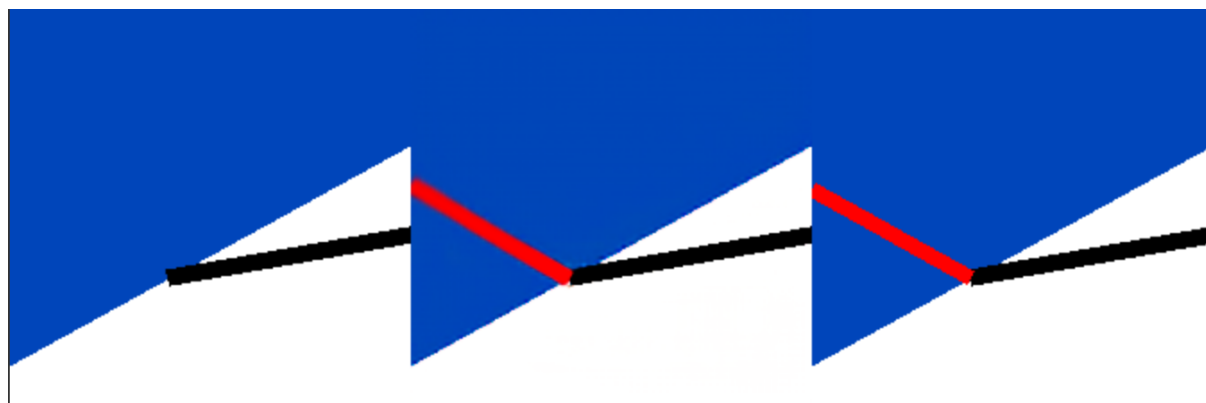


图 105: 可变折射率的训练结果（一）

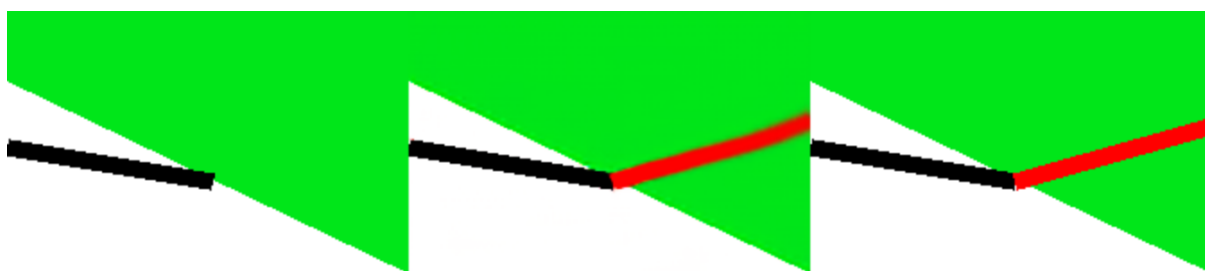


图 106: 可变折射率的训练结果 (二)

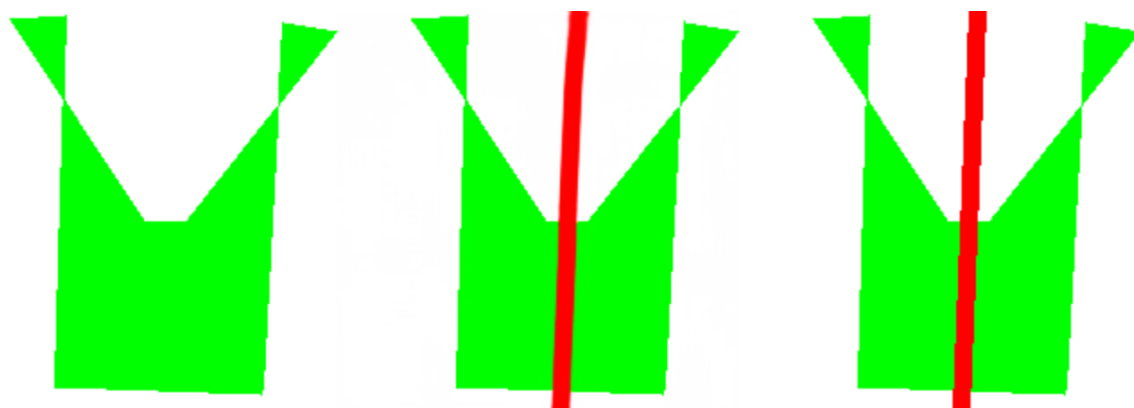


图 107: 对称轴学习 (一)



图 108: 对称轴学习 (二)

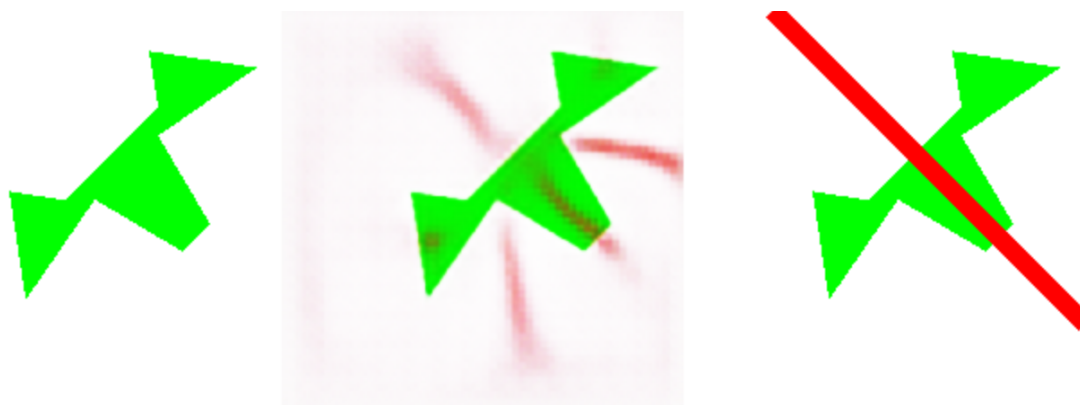


图 109: 学习中间状态 (一)

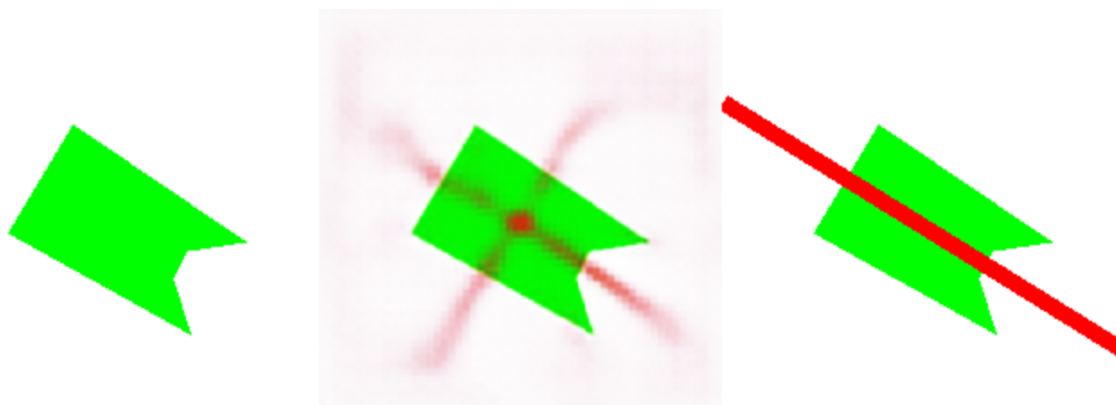


图 110: 迷宫实验

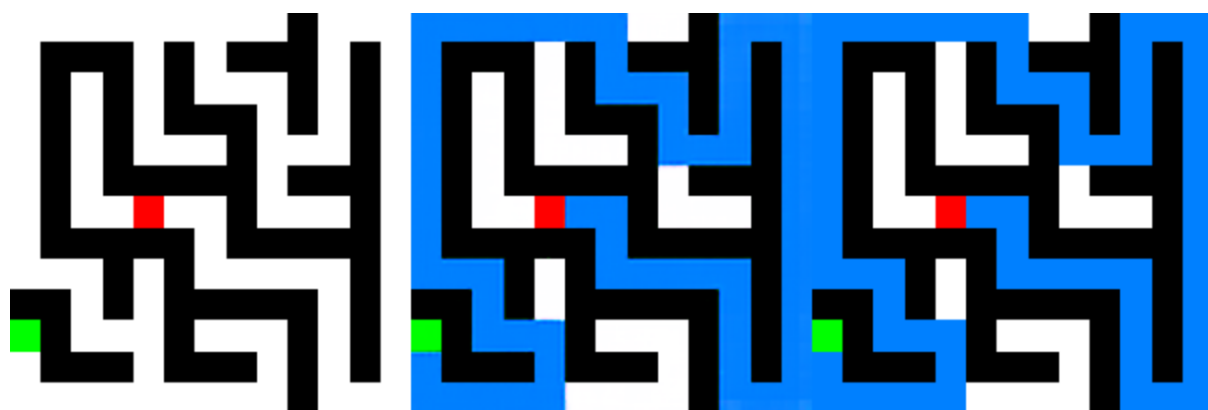


图 111: 学习中间状态 (二)

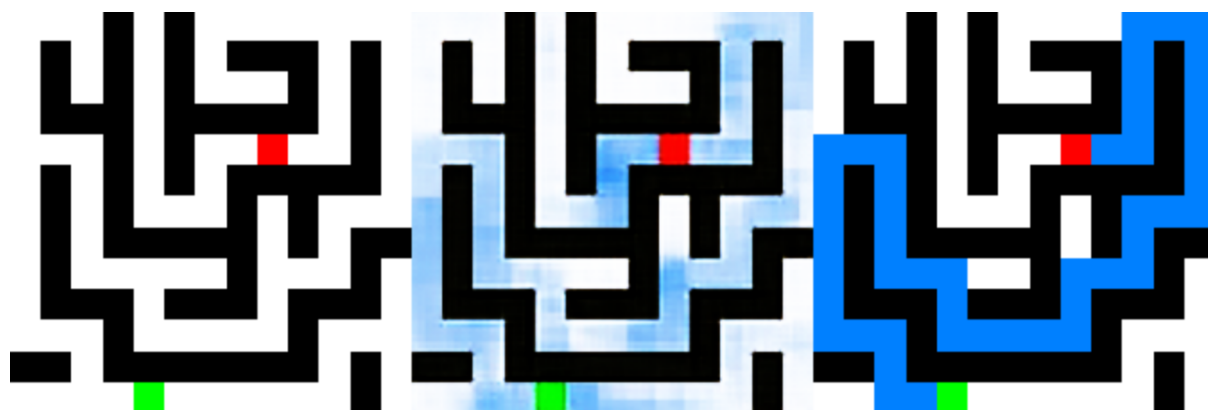


图 112: 学习中间状态 (三)

14.2 这里列出所有做过的实验

A. 符号数学逻辑 (Symbolic Math Logic)

`generate_conditional_add_subtract.py`: 脚本生成两个 N-bit 整数的加法或减法（取绝对值）问题。它包含两种模式：

`generate_add_binary_modulo.py`: 这是一个早期的基础算术实验，用于测试模型学习模加法（或称“截断加法”）的能力，这种运算常见于计算机硬件的定宽整数运算。

`generate_multiply_binary.py`: 作为二进制算术能力的一个基准测试，生成 N-bit 整数的乘法数据集。

`generate_multiply_binary_no_carry_phase1.py`: 这是乘法“解耦”实验的第一阶段。旨在测试模型是否能学会乘法的第一步：无进位的按位相乘和错位相加，将一个复杂的乘法问题分解为一个更简单的计数问题。

`generate_multiply_binary_from_counts_phase2.py`: 这是乘法“解耦”实验的第二阶段。旨在验证一个独立的模型能否学会处理复杂的进位逻辑，即从一个“无进位计数向量”中计算出最终的二进制乘积。

`generate_add_hexadecimal.py`: 对比模型在不同符号系统下的学习能力。此脚本旨在验证模型学习的是加法这一抽象数学概念，还是仅仅是特定于二进制符号的模式。

`generate_multiply_decimal.py`: 测试模型处理非二进制符号输入（0-9 字符），并执行算术运算（乘法）的能力。

`generate_add_binary_with_position_shuffle.py`: 这是“语义洗牌”系列实验中的“位置洗牌”部分。它旨在验证模型是否依赖于输入的固定空间结构，还是能学习到与位置无关的抽象关系。

`generate_symbol_add_shuffle_dataset.py`: 这是我的研究中一项关键的决定性实验，旨在彻底分离模型的“表面模式匹配”能力和“抽象结构学习”能力。

`generate_add_hidden_constant.py`: 测试模型在没有任何直接线索的情况下，从大量样本中推断出隐藏规则或参数的能力。这类似于一个简化的系统辨识（System Identification）问题。

`generate_multitask_alu.py`: 此脚本旨在构建一个模拟算术逻辑单元 (ALU) 的多任务学习场景。它测试模型能否在一次前向传播中，对同一份输入并行执行多种不同的、定义明确的计算任务。

`generate_modulo_operation.py`: 探究模型学习模运算（Modulo Operation）的能力，这是一个在数论和计算机科学中至关重要但具有“循环”性质的运算。

`generate_rsa_encryption.py`: 测试模型学习高度非线性的、在计算上被认为是“困难”的确定性规则的能力。RSA 加密是一个典型的例子。

`generate_deduction_chain_text.py`: 生成多步逻辑推理任务，测试模型执行符号演绎（deduction）的能力，类似于一个简化的定理证明器。

`generate_deduction_multirule_text.py`: 测试模型在面对多个独立的、互不相干的规则时，能否根据查询（Query）正确地“路由”到相应的规则并进行判断。

`generate_deduction_multirule_text_v2.py`: 测试模型在面对多个独立的、互不相干的规则时，能否根据查询（Query）正确地“路由”到相应的规则并进行判断。

`generate_deduction_multirule_binary.py`: 这是对多规则推理任务的格式优化版本，旨在

测试紧凑的二进制编码是否比稀疏的文本格式更有利于模型学习。

`generate_deduction_fixed_depth.py`: 测试模型在有明确结构、固定深度的符号演绎任务中的多步推理能力。

`generate_function_composition.py`: 测试模型学习函数组合 (Function Composition) 的能力。这要求模型像解释器一样, 按顺序解析指令并对数据进行变换。

`generate_count_set_bits.py`: 测试模型执行全局聚合操作的能力。与局部规则不同, 计数需要模型综合整个输入序列的信息。

`generate_sum_pattern_positions.py`: 测试模型执行更复杂的、分组式的并行聚合任务的能力。模型需要先分割输入, 然后对每个分割后的模式进行分类, 最后对属于同一类的模式的位置信息进行累加。

`generate_sum_pattern_positions_v2.py`: 测试模型执行更复杂的、分组式的并行聚合任务的能力。模型需要先分割输入, 然后对每个分割后的模式进行分类, 最后对属于同一类的模式的位置信息进行累加。

`generate_sum_pairwise_hamming_distance.py`: 测试模型执行一个需要两层嵌套聚合操作的复杂任务。模型需要先在每个比特位上进行全局统计, 然后再将所有比特位的结果累加起来。

`generate_circular_shift.py`: 测试模型学习位移操作的能力, 特别是循环位移 (circular shift), 这是密码学和底层编程中的常见操作。

`generate_multiply_matrix_3x3.py`: 测试模型学习结构化代数运算 (矩阵乘法) 的能力, 这比简单的标量运算需要更复杂的“数据路由”和“乘积累加”能力。

`generate_evaluate_boolean_expression_text.py`: 测试模型解析一个简单的领域特定语言 (DSL) 并执行求值的能力, 这比前面固定结构的表达式求值更进了一步。

`generate_evaluate_arithmetic_expression.py`: 训练模型执行符号表达式的求值任务, 这要求模型理解运算符优先级 (通过树状结构隐式表达)、变量替换和算术运算。

`generate_evaluate_arithmetic_expression_no_multiply.py`: 这是对 `generate_evaluate_arithmetic_expression.py` 的简化版本, 旨在通过移除乘法运算来降低学习难度, 以测试模型在更基础的算术表达式求值上的能力。

`generate_evaluate_arithmetic_expression_no_multiply_small_range.py`: 这是在前一个“无乘法”版本基础上的进一步简化, 通过缩小数值范围来进一步降低学习难度, 用于精确诊断模型在最简单表达式求值任务上的性能瓶颈。

`generate_check_boolean_equivalence.py`: 测试模型对布尔代数逻辑等价性的判断能力。这是一个抽象的符号推理任务, 要求模型理解表达式的结构和布尔运算法则。

`generate_polynomial_shift_coefficients.py`: 测试模型学习一个抽象的代数变换规则的能力。这个任务需要模型理解多项式展开的内在结构。

`generate_convolution_2d.py`: 测试模型学习二维卷积 (Conv2D) 这一基本图像处理操作的能力, 并探究其是否能从输入输出对中推断出隐藏的固定规则 (即卷积核本身)。

`generate_simple_block_cipher.py`: 测试模型“破解”或学习一个简单但非平凡的自定义加密算法的能力。该任务代表了一类复杂的、具有高度混沌和雪崩效应的符号变换规则。

`generate_sin_function_float32.py`: 测试模型拟合连续、周期性、非线性函数 ($\sin(x)$) 的能力, 使用标准的 32 位浮点数格式进行输入和输出。

`generate_sin_function_float64_to_int12_deprecated.py`: 这是对 `sin` 函数拟合任务的另一种编码尝试, 旨在探索使用更高精度的浮点输入和更低精度的量化二进制输出对学习效果的影响。

`generate_sin_function_float32_to_quantized_int.py`: 测试模型拟合连续、周期性、非线性函数 ($\sin(x)$) 的能力, 并探索不同输入/输出编码方案对学习效果的影响。

`generate_multiply_binary_modulo.py`: 作为基础算术实验的一部分, 测试模型对截断乘法 (或称模乘法) 的掌握能力。

`generate_explainable_two_step_calculation.py`: 测试模型输出计算“中间步骤”或“思维链”的能力, 是“功能性可解释性”的一个直接验证。

`generate_min_swaps_for_checkerboard.py`: 解决 LeetCode 782 题“变为棋盘”(<https://leetcode.cn/problems/transform-to-chessboard/>) - 通过任意交换行和列, 将一个 0/1 矩阵变为“棋盘”模式 (相邻元素不同) 所需的最少交换次数。

`generate_min_flips_for_alternating_binary.py`: 测试模型解决一个基于位翻转的字符串优化问题, 该问题可以被巧妙地映射为一个滑动窗口问题来求解。

`generate_min_swaps_for_checkerboard_v2.py`: 解决 LeetCode 1536 题“排布二进制网格的最少交换次数”(<https://leetcode.cn/problems/minimum-swaps-to-arrange-a-binary-grid/>) - 通过交换相邻行, 将一个二进制网格变为上三角形式 (主对角线以上全为 0) 所需的最少交换次数。

`generate_min_prefix_flips.py`: 测试模型学习一个依赖于历史状态的、顺序处理的贪心算法的能力。

`generate_min_flips_for_chunked_binary.py`: 测试模型学习一个基于局部块 (chunk) 的字符串变换优化问题的能力。

`generate_largest_island_by_adding_one_cell.py`: 解决一个涉及图遍历和全局优化的算法问题 (LeetCode 827. 最大人工岛)。模型需要评估所有可能的“填海”位置, 并选出能使合并后岛屿面积最大的那一个。

`generate_largest_island_by_adding_one_cell_v2.py`: 解决一个涉及图遍历和全局优化的算法问题 (LeetCode 827. 最大人工岛)。模型需要评估所有可能的“填海”位置, 并选出能使合并后岛屿面积最大的那一个。

`generate_sat_solver_text.py`: 测试模型解决一个标志性的 NP 完全问题——布尔可满足性 (SAT) 问题的能力。

`generate_sat_solver_compact_text.py`: 这是对 `symbolic_math_logic/generate_sat_solver_text.py` 的一个变种, 采用了不同的输入编码格式来解决同样的 3-SAT 问题。

`generate_binary_mod3_dfa_explain.py`: 这是一个可解释性实验, 旨在测试模型是否能学会并输出有限状态自动机 (DFA) 的内部状态转移过程。

`generate_add_binary_explainable.py`: 测试模型是否能学会二进制加法的内部机制 (进位传播)。这是一个可解释性实验, 要求模型不仅输出结果, 还要输出每一位的进位状态。

`generate_add_quaternary_float_encoding.py`: 测试 MLP 是否有能力解析实数并将其对应成编码的二进制, 完成计算。事实证明它可以做到, 说明在训练 MLP 时, 可以将间隔一定距离的实数当作纯粹的符号使用。

`generate_add_binary_masked_output.py`: 测试模型在“条件输出”任务上的能力。模型不仅需要计算加法,还需要根据输入的掩码 (Mask) 动态决定只输出结果的哪几位。这类似于 Attention 机制,需要模型学会根据指令 (Mask) 来聚焦和路由信息。

`generate_multitask_prefixed_cal10.py`: 生成“前置 CA 演化 + 4 个后置任务”的多任务学习数据集,用于测试模型在单次前向传播中执行共享表示提取和多个独立下游任务的能力。这是对 `generate_multitask_alu.py` 的重要扩展,增加了共享前置变换层的概念。

`generate_convergent_abstraction.py`: 生成“收敛式抽象”(多输入 $\rightarrow$ 单一中间表示枢纽)实验数据集。这个实验旨在验证论文第八幕提出的关键假设:不同的输入任务能否自然地收敛到一个共享的抽象中间表示,并基于该表示完成后续的推理任务?

`generate_multiply_binary_showdown.py`: 二进制乘法的三算法对决实验。扩展 `generate_rain_water_final_showdown.py` 的思想到乘法任务,提供三种解耦方法对比。

B. 算法学习 (Algorithm Learning)

`generate_sort_integers.py`: 测试模型执行基本排序算法的能力,这是一个非局部的、需要对输入元素进行比较和重排的经典算法任务。

`generate_edit_distance.py`: 测试模型学习解决动态规划问题的能力。编辑距离是一个典型的 DP 问题,需要模型在概念上构建一个二维的求解矩阵。[LeetCode 72. Edit Distance](#)

`generate_edit_distance_explainable.py`: 这是“功能性可解释性”的一个核心实验。它要求模型不仅给出最终答案(编辑距离),还要输出达成答案的完整“思维链”(编辑过程)。[LeetCode 72. Edit Distance \(Explainable / Path Construction Version\)](#)

`generate_maze_random_walls.py`: 测试模型在随机生成的“多孔”迷宫中的基础寻路能力。

`generate_maze_dense.py`: 测试模型在复杂的、类似人类设计的“稠密”迷宫中进行路径规划的能力,这比随机墙壁迷宫更具挑战性。

`generate_blocks_world_arbitrary_goal.py`: 解决经典的“积木世界”(Blocks World)规划问题。该问题作为衡量大语言模型推理能力的标准任务之一,在苹果公司发布的著名论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity”中被重点讨论,作为评估模型在状态空间搜索和规划能力上的基准测试。该研究通过积木问题、过河问题、汉诺塔等一系列经典规划任务,系统地揭示了当前大语言模型在精确符号推理和状态空间规划上的根本局限性。本脚本精确实现了论文中的“积木世界”通用版本,允许指定任意的初始状态和终止状态,作为验证神经网络在复杂规划问题上推理能力的核心对照实验。

`generate_blocks_world_fixed_goal.py`: 这是对“积木世界”任务的简化版本,通过固定目标状态(所有积木有序地堆叠在第一个柱子上),旨在测试模型在目标明确、状态空间更结构化的情况下的学习能力。该问题同样源自苹果公司论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity”中的基准测试任务。该研究指出,即使对于这样目标状态明确的规划问题,大语言模型在状态空间搜索和最优策略学习方面仍然存在显著困难。本脚本实现了论文中的积木世界简化版本,通过固定目标状态来降低任务复杂度,作为研究目标明确性对模型推理性能影响的对照实验。

`generate_blocks_world_fixed_goal_multilabel.py`: 进一步改进“积木世界”任务,通过允

许多个最优解，测试模型处理多标签分类问题的能力，更真实地反映了规划问题中可能存在的等效最优路径。该问题同样源自苹果公司论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity” 中的基准测试任务。该研究强调，现实世界中的规划问题往往存在多个等价最优解，这对模型的多义性推理能力提出了更高要求。本脚本在固定目标版本基础上进一步改进，为每个状态找到所有能导向最优路径的动作，生成多热（multi-hot）编码输出，作为研究神经网络处理多最优解规划问题的对照实验。

generate_blocks_world_fixed_goal_multilabel_fixed_format.py: 这是“积木世界”任务的最终优化版本，通过改进输入表示法，旨在为模型提供一个更清晰、更结构化的学习目标。该问题同样源自苹果公司论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity” 中的基准测试任务。该研究指出，输入表示法对模型的学习效率和最终性能有着至关重要的影响。本脚本在多标签版本基础上进一步优化，采用固定槽位（fixed-slot）表示法替代可变长度输入，消除了序列化带来的复杂性，为 Transformer 等架构提供了更友好的结构化输入。这使其成为研究输入表示法对神经符号推理性能影响的对照实验。

generate_checkers_jump_1d.py: 解决一维空间中的棋子交换规划问题。该问题作为衡量大语言模型推理能力的标准任务之一，在苹果公司发布的著名论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity” 中被重点讨论，作为评估模型在状态空间搜索和规划能力上的基准测试。该研究通过积木问题、过河问题、汉诺塔等一系列经典规划任务，系统性地揭示了当前大语言模型在精确符号推理和状态空间规划上的根本局限性。本脚本精确实现了论文中的“跳棋交换”通用版本，作为验证神经网络在复杂规划问题上推理能力的核心对照实验。

generate_river_crossing_puzzle.py: 解决一个经典的约束满足和状态空间搜索问题——“N 对伴侣过河”。该问题要求在满足“任何女性不能在没有其伴侣在场的情况下，与其他男性共处”的约束下，将所有人运到对岸。该任务源自苹果公司的一篇著名论文 [14] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity”，该研究通过过河问题、汉诺塔、跳棋等基准测试，揭示了大型语言模型在某些类型推理任务上的根本局限性。本脚本精确复现了论文中的“N 对伴侣过河”问题，作为验证神经网络符号推理能力的对照实验。

generate_trapping_rain_water_aggregate.py: 这是解决“接雨水”算法问题的初步尝试，旨在测试模型学习一个聚合输出（而非解耦输出）的能力。实验结果表明，要求模型直接输出总和值（一个单一的聚合数字）比输出每个位置的详细信息要困难得多。这成为一个关键的对比实验，证明了输出格式设计对模型学习效率的系统性影响。对应 LeetCode 题目：[42. Trapping Rain Water](#)。该脚本精确实现了 LeetCode 原题的聚合输出格式（只输出总量），与 **generate_trapping_rain_water_decoupled.py**（解耦输出，输出每个位置的水量）形成鲜明对照，用于研究输出表示对神经网络学习难度的影响，验证了论文中“解耦加速收敛”的核心发现。

generate_trapping_rain_water_decoupled.py: 解决经典的“接雨水”算法问题（LeetCode Hard [42. Trapping Rain Water](#)）。这个任务的成功展示了模型学习需要全局信息（如全局最高

点)的复杂算法的能力,并通过问题解耦的思想,证明了输出格式设计对模型学习效率的巨大影响。

`generate_trapping_rain_water_2d.py`: 作为一维“接雨水”问题的扩展,解决二维版本的“接雨水”问题。该任务要求模型理解二维空间中的“包围”和“边界”概念,是一个更复杂的全局信息处理挑战。对应 LeetCode 题目: [407. Trapping Rain Water II](#) (困难题)。

`generate_skyline_max_height_aggregate.py`: 这是解决“天际线”问题的初步尝试,要求模型从所有建筑的最终高度中,只预测出那个最高的高度值。此任务用于对比聚合输出和解耦输出的学习难度。对应 LeetCode 题目: [1840. Maximum Building Height](#) (聚合输出版本)。

`generate_skyline_all_heights_decoupled.py`: 测试模型解决一个带有一维空间约束的全局优化问题的能力。问题原型是 LeetCode [1840. Maximum Building Height](#) (解耦输出版本)。该题目要求: 给定 N 个建筑的位置和限高,相邻建筑高度差不超过 1,求每个建筑能达到的最大高度。通过解耦输出,要求模型预测每一栋建筑的高度,而非仅仅是最大值,作为研究输出格式设计对模型学习效率影响的对照实验。

`generate_hanoi_tower_path_strategy_sep_format.py`: 作为对比组 A,此脚本用于生成汉诺塔最优路径策略数据集,采用分隔符 + 二进制编码的输入格式。

`generate_hanoi_tower_path_strategy_fixed_format.py`: 作为对比组 B,此脚本同样生成汉诺塔最优路径策略数据集,但采用结构化的固定槽位输入格式(重命名自原来的 `global` 脚本以反映真实逻辑),用于验证数据表示对学习效率的影响。

`generate_hanoi_tower_compare_formats.py`: 这是一个对比实验脚本,它为同一个汉诺塔问题生成两种不同的输入格式(分隔符 vs. 固定槽位),用于系统性地评估不同数据表示法对模型学习递归策略的影响。

`generate_hanoi_tower_compare_formats_and_strategies.py`: 这是一个更全面的汉诺塔对比实验脚本。它不仅生成两种输入格式,还生成两种不同的数据集:一种只包含最优路径上的状态(“路径策略”),另一种包含所有可达状态(“全局策略”),用于探究模型在学习局部最优路径和全局最优策略上的能力差异。

`generate_hanoi_tower_build_full_state_graph.py`: 这是一个“汉诺塔问题”研究的集大成者,旨在通过多种不同的数据表示和采样策略,深度剖析模型对递归结构的理解能力。它是一个自给自足的数据工厂。该研究对应于论文中引用的 Apple Research 相关工作,重点在于通过完整状态图分析模型对递归结构的掌握。

`generate_hanoi_tower_sample_from_state_graph.py`: 这是一个后处理和采样脚本,它利用 `generate_hanoi_tower_build_full_state_graph.py` 生成的完整知识库,来精确地提取特定类型的训练数据子集,例如“扭曲路径”(twisted path)或“最难部分”,用于进行更精细的消融实验。

`generate_hanoi_decoupled_scaffold.py`: 这是一个引入解耦辅助标签的汉诺塔实验脚本。它通过生成每个盘子的阶段性目标(Scaffold),将复杂的递归逻辑转化为局部的状态转移任务,从而解决了全状态空间下模型无法收敛的难题。

`generate_sokoban_planning_astar.py`: 解决经典的“推箱子”(Sokoban)规划问题。这个任务比简单的路径规划更难,因为它涉及到改变环境状态(箱子位置),状态空间巨大。

`generate_sokoban_planning_full.py`: 解决经典的“推箱子”(Sokoban)规划问题。这是一

个高难度的 AI 任务，因为它涉及到在一个巨大的状态空间中进行搜索，并且动作会改变环境的状态。

`generate_sokoban_planning_claude_deprecated.py`: (已弃用) 这是一个早期的、逻辑更复杂的尝试，但未能稳定地生成高质量数据集，已被更可靠的 `generate_sokoban_planning_full.py` 取代。

`generate_matrix_flip_strategy.py`: 解决一个矩阵优化的经典问题 (Score After Flipping Matrix)。此版本旨在测试模型能否学习到一个“策略”而非最终结果。对应 LeetCode 题目: [861. Score After Flipping Matrix](#)

`generate_matrix_flip_max_score.py`: 测试模型学习一个矩阵优化问题的能力，该问题需要通过两步贪心策略（先行翻转，后列翻转）来达到全局最优。该版本要求模型直接输出最终的聚合结果（分数）。对应 LeetCode 题目: [861. Score After Flipping Matrix](#)

`generate_min_k_bit_flips.py`: 测试模型学习一个依赖于历史状态的、顺序处理的贪心算法的能力，并且测试其能否将输入的一部分 (k) 作为“参数”来指导对另一部分 (nums) 的处理。对应 LeetCode 题目: [995. Minimum Number of K Consecutive Bit Flips](#)

`generate_min_k_bit_flips_fixed_k.py`: 测试模型学习一个依赖于历史状态的、顺序处理的贪心算法的能力。此版本中，环境参数 (k=2) 是固定的、隐藏的，模型必须从数据中隐式学习。这是一个对照实验，用于对比 k 可变的版本。对应 LeetCode 题目: [995. Minimum Number of K Consecutive Bit Flips](#) (k 固定版)

`generate_special_binary_string_recursion.py`: 测试模型学习一个递归定义的字符串变换规则的能力。该问题对应 LeetCode 题目: [761. Special Binary String](#) (困难题)。题目要求: 一个特殊的二进制序列具有相等的 0 和 1 数量，且任何前缀中 1 不少于 0；可以通过交换相邻的特殊子串来得到字典序最大的结果。本脚本精确实现了该问题的递归分解和字典序排序算法，作为测试神经网络学习复杂递归规则能力的基准实验。

`generate_count_connected_components.py`: 测试模型对图结构的基本理解，特别是“连通性”这一核心概念 (对应 LeetCode [323. Number of Connected Components in an Undirected Graph](#), 注: 此为会员题)。

`generate_check_graph_connectivity.py`: 这是对模型图论基础能力的又一个核心测试，任务是判断图中任意两点之间是否存在一条路径 (对应 LeetCode [1971. Find if Path Exists in Graph](#))。

`generate_minimize_malware_spread.py`: 解决一个基于图论的病毒传播优化问题 (LeetCode Hard “Minimize Malware Spread”)。模型需要理解图的连通性，并评估移除不同节点对全局传播的影响。本脚本提供两种输出格式，用于对比哪种更容易学习。对应 LeetCode 题目: [924. Minimize Malware Spread](#)

`generate_count_islands_1d.py`: 测试模型在一维序列上进行模式识别和计数的能力。

`generate_find_articulation_points.py`: 测试模型识别图的“割点” (Articulation Point) 或“桥” (Bridge) 的能力，这是一个图论中的重要概念。[LeetCode 1568. Minimum Number of Days to Disconnect Island](#)

`generate_nim_game_zeckendorf.py`: 这个实验旨在测试我的范式能否学习一个基于复杂数论 (齐肯多夫表示法) 的非直观博弈论问题。它脱离了简单的模式匹配，需要模型理解更深层次的

数学结构。

`generate_longest_subsequence_constrained.py`: 测试模型处理一个混合了序列操作和数值约束的复杂优化问题的能力。对应 LeetCode 题目: [2311. Longest Binary Subsequence Less Than or Equal to K](#)

`generate_treasure_hunt_tsp.py`: 解决一个复杂的状态空间搜索问题, 它结合了图的遍历 (BFS) 和组合优化 (状态压缩 DP), 是算法竞赛中的经典难题。

`generate_freedom_trail_dp.py`: [LeetCode 514. Freedom Trail](#) 测试模型学习解决一个需要动态规划和路径回溯的复杂优化问题的能力。

`generate_sum_of_subset_with_mask.py`: 测试模型根据一个二进制掩码从一个集合中选择元素并执行聚合操作 (求和) 的能力。这是一个创新性的对照实验, 旨在验证任务结构本身 (而非模型容量) 对神经网络可学习性的关键影响。

`generate_sudoku_6x6.py`: 测试模型在处理有强约束满足问题 (Constraint Satisfaction Problem) —— 数独 —— 上的能力。

`generate_valid_parentheses_path_random_deprecated.py`: (早期探索/已弃用) 这是解决“合法括号路径”问题的早期尝试。

`generate_valid_parentheses_path_balanced.py`: 解决一个二维网格上的路径查找问题, 但路径的合法性受到栈式结构 (括号匹配) 的约束。这是一个算法和逻辑约束结合的复杂任务 ([LeetCode Hard Check if There Is a Valid Parentheses String Path](#))。

`generate_point_in_polygon.py`: 测试模型学习一个计算几何中的经典算法——射线法 (Ray Casting Algorithm) —— 的能力。

`generate_shortest_path_in_matrix_bfs.py`: 测试模型在一个二维网格中, 基于经典的广度优先搜索 (BFS) 算法寻找最短路径的能力。

`generate_sudoku_4x4_stepwise_deprecated.py`: (废弃) 旨在测试模型进行“步进式” (stepwise) 推理的能力, 即在每个状态下只预测下一步的最优动作, 而不是一次性输出完整解。

`generate_tiling_problem_deprecated.py`: (已弃用) 旨在测试模型解决一个经典的平铺覆盖优化问题的能力, 这是一个 NP-hard 问题。

`generate_hanoi_tower_twisted_path_deprecated.py`: (废弃) 此脚本意图生成一个汉诺塔问题的“扭曲路径”数据集, 即从一个非标准但困难的起始状态到标准终点的最优路径。

`generate_checkers_jump_1d_v2.py`: 这是一个针对跳棋交换问题的序列学习对比实验脚本。与 V1 版本关注单步最优策略不同, 本脚本旨在生成多种类型的完整路径数据集 (包括最优路径、子路径、非最优路径等), 用于系统性地研究模型在不同数据分布下学习长序列规划的能力。该任务同样源自苹果公司论文 [14] 中的跳棋交换问题, 但侧重于探究序列级泛化与路径多样性的影响。

`generate_maze_symbolic_to_image.py`: 将符号化的迷宫路径规划数据集转换为图像格式, 以测试视觉模型 (如 CNN、ViT) 直接从像素进行路径规划的能力。

`generate_trapping_rain_water_visualizer.py`: 这是一个数据转换与可视化脚本。它的作用是将已经生成的、符号化的“接雨水”数据集转换为一个 image-to-image 格式的数据集, 以使用视觉模型来解决同一个问题。

`generate_shortest_path_in_tree_deprecated.py`: (早期探索/已弃用) 这是一个早期的实

验，旨在测试模型从图像中寻找图上最短路径的能力。

`generate_rain_water_final_showdown.py`: 这是一个算法对比实验，旨在通过提供三种不同算法（动态规划、单调栈、双指针）的中间过程作为辅助标签，来研究哪种算法逻辑更符合神经网络的归纳偏置。

`generate_bricks_falling.py`: 解决 LeetCode 困难题 [803. Bricks Falling When Hit](#)。测试模型对物理稳定性和连通性的理解。砖块只有在与顶部直接相连或间接通过其他稳定砖块相连时才是稳定的，否则会掉落。

`generate_maximal_rectangle_coords.py`: 解决 LeetCode 困难题 [85. Maximal Rectangle](#)。

`generate_median_island_median_finder.py`: 这是一个复杂的符号推理任务，要求神经网络在二进制序列中执行多层逻辑操作，结合了序列分割、中位数计算和定位操作。

`generate_rain_water_summation.py`: “接雨水”问题的子任务变体（Task B）。输入不是地形高度，而是已经计算好的每格雨水量（中间状态）。任务是计算这些雨水量的总和。这用于测试模型单纯进行“求和”这一数学聚合操作的能力，与完整的接雨水任务（分析地形 + 求和）形成对比。

`generate_rain_water_then_cellular_automata.py`: 这是一个跨领域链式推理任务（Task D）。测试模型能否在一次前向传播中，连续执行两个逻辑上完全无关的复杂步骤：首先解决“接雨水”问题计算每格水量，然后立即将该水量数据作为初始状态，执行“元胞自动机”演化。

`generate_maze_decoupled.py`: 这是稠密迷宫寻路任务的解耦实验版本。在 `generate_maze_dense.py` 只输出最优方向（4 分类）的基础上，本脚本额外输出完整的 BFS 距离图作为“解释标签”，用于研究解耦辅助信息对模型学习效率的影响，以及探测训练后模型内部表示中距离信息的存在性。

`generate_edit_nextstep_mlp.py`: 这是编辑距离问题的一个新颖的解耦实验。与 `generate_edit_distance_explainable.py` 输出完整编辑路径不同，本脚本聚焦于预测“下一步最优操作”，并提供两种解耦辅助信息：操作掩码（哪些操作是最优的）和完整 DP 表的操作类型信息。

`generate_edit_distance_dp_table_full.py`: 编辑距离问题的完整 DP 表解耦实验。本脚本探索一个深刻的研究问题：是否每一个问题都一定存在能加速收敛的解耦信息？与 `generate_edit_nextstep_mlp.py` 相比，本脚本提供了更“完整”（但也更冗余）的 DP 表作为解耦标签。

`generate_median_one_finder.py`: 生成“中位数‘1’定位”任务数据集。这是一个基础的统计定位任务。

`generate_mean_of_medians.py`: 生成“奇偶孤岛中位点均值”任务数据集。这是一个没有任何中间提示的复杂逻辑推理任务，被称为“九重炼狱”。

`generate_exam_seats.py`: 生成“考试安排”任务数据集 ([Maximum Students Taking Exam - LeetCode](#))。测试模型在二维网格上的状态压缩动态规划推理能力。

`generate_parity_accumulator.py`: 生成“奇偶指令累加器”任务。这是一个包含顺序依赖和条件算术的复杂逻辑任务。

`generate_neural_turing_machine.py`: 模拟一个简化的“神经图灵机”，在循环磁带上执行条件写入操作。这是对联结主义系统能否学习执行算法操作的直接测试。

`generate_lights_out_forward.py`: 实现经典的“Lights Out”游戏正向模拟。这是一个测试神经网络学习局部规则传播能力的理想平台。

`generate_lights_out_gaussian_elimination.py`: 将 Lights Out 逆向问题建模为线性方程组求解, 并通过高斯消元法提供解耦信息。这是一个突破性的实验, 证明了通过合适的解耦方法, 神经网络可以成功学习求解原本无法收敛的复杂逆向问题。

C. 视觉推理 (Visual Reasoning)

`generate_checkerboard_to_binary.py`: 这是一个基础的视觉到符号转换任务, 用于测试模型从原始像素数据中解码结构化信息的能力。

`generate_line_angle_to_vector.py`: 测试模型从图像中提取精确几何信息 (角度) 的能力, 这是一个比简单识别棋盘格更高级的视觉推理任务。

`generate_count_shapes_from_image.py`: 测试模型同时进行物体识别 (形状)、属性识别 (颜色) 和计数 (聚合) 的多重视觉任务能力。

`generate_sokoban_symbolic_to_image_no_labels.py`: 这是一个数据转换脚本, 用于将符号化的推箱子数据集 (.jsonl 格式) 仅转换为图像格式, 用于纯视觉任务或作为更复杂数据处理的中间步骤。

`generate_sokoban_symbolic_to_image_with_labels.py`: 这是一个数据转换脚本, 用于将符号化的推箱子数据集 (.jsonl 格式) 转换为一个完整的图像分类数据集, 以供计算机视觉模型 (如 ViT, Swin Transformer) 进行训练。

`generate_triangle_to_incircle.py`: 这是展示“用梯度下降雕刻精确规则”的一个标志性实验。它测试模型能否学习到一个纯粹的、非平凡的几何构造规则 (三角形内切圆)。

`generate_polygon_to_symmetry_axis.py`: 测试模型从一个完整的对称图形中反向推断出其隐含的对称轴的能力。

`generate_triangle_to_centroid.py`: 测试模型学习另一个基础几何概念——重心的能力。

`generate_triangle_to_tessellation.py`: 这是这个范式能力的一个标志性展示。它测试模型能否学习一种无限的、基于晶格的生成规则。由于镶嵌图案的全局关联性和细节的精确性, 它有力地排除了模型仅仅是靠“插值”或“记忆”来解决问题的可能性。

`generate_shortest_distance_between_triangles.py`: 测试模型在包含多个对象的情况下, 进行全局几何关系 (最短距离) 推理的能力。

`generate_coords_to_triangle.py`: 这是一个基础的符号到几何的渲染任务, 测试模型将抽象的坐标信息转换为具体像素形状的能力。

`generate_triangle_coords_to_tessellation.py`: 这是一个高级的、混合了符号指令和几何生成规则的推理任务。

`generate_trapping_rain_water_image_to_symbol.py`: 这是一个图像到符号的转换任务, 用于测试 CNN 等视觉模型是否能从图像中提取精确的物理量。

D. 元胞自动机 (Cellular Automata)

`generate_cellular_automata_1d.py`: 用于生成一维元胞自动机 (CA) 的演化数据集, 以测试模型学习和执行局部、确定性规则的能力。

`generate_game_of_life_2d.py`: 生成二维元胞自动机——Conway's Game of Life 的数据集。此任务比一维 CA 更复杂, 需要模型理解二维空间中的邻域关系。

`generate_cellular_automata_1d_multistate.py`: 作为一维元胞自动机实验的扩展, 测试模型处理非二进制状态空间的能力。

`generate_cellular_automata_programmable.py`: 测试模型的“可编程性”或“元学习”能力。模型不仅要学会 CA 的演化过程, 还要能根据每次输入中给出的不同规则来执行演化。

`generate_cellular_automata_inverse_rule90.py`: 测试模型解决“逆问题”(Inverse Problem) 的能力。给定一个确定性系统的输出, 模型需要反向推断出满足特定约束(最稀疏且唯一)的可能输入。

`generate_game_of_life_image_to_image.py`: 这是二维元胞自动机的 image-to-image 版本, 测试模型能否直接在像素空间中执行基于局部规则的演化。

`generate_cellular_automata_spatial_conditional.py`: 测试模型在单一模态(图像)内部分区和解析“指令”与“数据”的能力, 是一种“伪多模态”或“空间条件化”的实验。

`generate_cellular_automata_multimodal_deprecated.py`: (已弃用) 生成一个真正的多模态数据集, 用于训练能够同时理解图像输入和文本指令的模型。

`generate_cellular_automata_1d_to_grid_image_interp.py`: 该脚本旨在设计一个“逻辑/感知混合”任务, 用以证明神经网络的规则学习能力和内插能力并非互斥, 而是可以一体化地在单个任务中得到体现。它迫使模型必须同时“看穿”输入的连续灰度值以执行离散的逻辑推理, 并记住这些灰度值以完成最终的连续值映射。

`generate_cellular_automata_1d_to_grid_image.py`: 测试模型能否直接将一维的符号计算结果“渲染”成结构化的二维图像。

`generate_cellular_automata_inverse_rule.py`: 这个实验是检验模型逆向推理(Inverse Reasoning) 能力的第一个尝试。我的问题是: 如果模型能从规则正向推出结果, 那么它能否从“输入-输出”对中反向推断出其背后的规则?

`generate_cellular_automata_inverse_rule_and_steps.py`: 这是在实现“唯一解”版本之前的一个早期版本, 它同样旨在让模型学习预测规则和迭代次数。

`generate_cellular_automata_inverse_rule_and_steps_unique.py`: 这是对逆向推理任务的一次重大升级。我不仅要求模型推断出什么规则被应用了, 还要推断出它被应用了多少次。

`generate_cellular_automata_1d_perturbed.py`: 该脚本旨在系统性地测试“神经雕刻”范式在面对不完美数据时的鲁棒性。通过向输入(模拟观测噪声)和输出(模拟标签噪声)中引入可控的随机扰动, 它探索了模型性能从理想规则世界向嘈杂现实世界过渡的连续谱。

`generate_ca110_full_trace.py`: 这是一个深度分析实验, 旨在生成元胞自动机演化的完整轨迹, 用于研究神经网络内部表示与演化步骤之间的关系(即“神经心智扫描仪”实验)。

`generate_ca_text_format_dataset.py`: 这是一个自回归实验, 旨在测试标准的 Decoder-only Transformer (如 GPT) 是否能够通过文本形式的 Prompt 来学习元胞自动机规则。

`generate_cellular_automata_image_and_label.py`: 这是一个通用数据集生成器, 用于为元胞自动机任务同时生成图像格式(Img2Img)和符号格式(Img2Label)的数据, 支持多进程加速。

`generate_mnist_ca_110.py`: 这是一个感知与推理融合的实验。它将 MNIST 手写数字作为元胞自动机的“细胞”, 测试模型是否能在识别数字的同时执行逻辑演化。

`generate_rulemnist_ca.py`: 这是一个更高级的感知与推理融合实验, 引入了规则控制。

`generate_ca_continuous_transform.py`: 这是一个连续-离散-连续混合的元胞自动机实验,专注于测试 MLP 等模型处理“模糊逻辑”输入的能力。

`generate_ca_nested_masked.py`: 生成“俄罗斯套娃”式(嵌套掩码)的元胞自动机数据集,测试模型在动态掩码条件下进行多阶段隐式推理的能力。

`generate_ca_masked_output.py`: 生成“部分可观测”的元胞自动机数据集,测试模型根据动态掩码选择性输出计算结果的能力。

`generate_ca_to_abstract_real.py`: 生成“CA 演化 $\rightarrow$ 抽象实数符号”的数据集,测试模型能否将离散演化结果编码为任意实数符号表示,验证神经网络对符号抽象性的理解。

`generate_ca_rule110_minimalist_trace.py`: 生成元胞自动机 Rule 110 的极简文本解释格式数据集,专门用于训练自回归语言模型(如 GPT)。与 `generate_ca_text_format_dataset.py` 仅输出最终状态不同,本脚本输出包含逐层计算过程的 CoT (Chain-of-Thought) 式解释。

`generate_ca_multirule_cot_trace.py`: 生成包含多种元胞自动机规则的 CoT (Chain-of-Thought) 解释数据集,用于训练能够识别和执行不同规则的自回归语言模型。这是对 `generate_ca_rule110_minimalist_trace.py` 的重要扩展,支持多规则环境下的推理能力测试。

`generate_ca_ood_hallucination_split.py`: 生成 CA OOD (Out-of-Distribution) 泛化测试数据集。将 256 个 CA 规则按比例分为训练集和 OOD 测试集,用于测试模型在未见过规则上的泛化能力。

`generate_hierarchical_ca_tree.py`: 层次化抽象 CA 树数据集生成器。用于更进一步验证中间表示的涌现现象,通过父节点的两个孙节点和另一个子节点的组合来检查涌现现象。

E. 物理模拟 (Physics Simulation)

`generate_projectile_motion_simulation.py`: 测试模型学习一个简单的动态物理过程的能力。这要求模型从初始条件(位置和速度向量)推断出整个时空轨迹。

`generate_snell_refraction_simulation.py`: 测试模型学习基础物理定律(斯涅尔折射定律)的能力。

`generate_snell_refraction_with_contextual_index.py`: 测试模型学习基础物理定律(斯涅尔折射定律)的能力,并且要求模型能从图像的上下文信息(背景颜色)中推断出物理参数(折射率)。

`generate_reaction_diffusion_deprecated.py`: (探索性/已弃用) 该脚本用于模拟基于 Gray-Scott 模型的反应-扩散系统,以生成复杂的、类似分形的“雪花”图案。它本质上属于连续物理场的模拟(涉及偏微分方程的离散化和浮点运算),而非简单的离散状态元胞自动机。

`generate_cube_rotation_matplotlib_deprecated.py`: (早期探索版本) 旨在测试模型从抽象的姿态参数(旋转角度)推理并渲染出三维物体正确视图的能力。

`generate_cube_rotation_pillow_v1.py`: (技术升级版本) 旨在测试模型从抽象的姿态参数推理并渲染出三维物体正确视图的能力,采用了更底层的、渲染效果更精确的技术路线。

`generate_cube_rotation_pillow_with_anchor.py`: (论文中使用的最终版本) 测试模型从抽象的姿态参数推理并渲染出三维物体正确视图的能力,并通过引入“视觉锚点”来辅助模型学习。

`generate_cube_rotation_pillow_wireframe.py`: (变体实验版本) 测试模型在更稀疏的视觉输入下, 能否仅通过线框和锚点信息来学习 3D 旋转。

`generate_catenary_curve_simulation_deprecated.py`: 这是我早期探索悬链线问题的脚本, 旨在测试模型学习由物理定律确定的非线性曲线的能力。

`generate_catenary_curve_from_points.py`: 测试模型学习由物理定律 (最小势能原理) 唯一确定的非线性曲线 (悬链线) 的能力。

`generate_orbital_path_from_initial_state.py`: 测试模型学习更复杂物理定律 (开普勒定律/万有引力定律) 的能力。

F. ARC-AGI 探索 (ARC-AGI Exploration)

注: 这里忘记了脚本分别对应哪些任务 id 和对应网址, 可能会在之后的版本补上

`generate_arc_contextual_color_swap.py`: 测试模型从图像的局部“上下文”或“示例”中学习规则, 并将其应用到同一图像的全局数据的能力。这直接模仿了 ARC-AGI 测试的核心理念。

`generate_arc_find_cross_pattern.py`: 测试模型在包含大量噪音的情况下进行视觉模式识别 (或可称作“目标检测”) 的能力。

`generate_arc_find_odd_one_out.py`: 测试模型执行一个复杂的“异类发现” (Find the Odd One Out) 元推理任务。模型需要逐行进行模式比较, 找出特例, 并将其重新组合到输出中。

`generate_arc_connect_colored_pairs.py`: 测试模型在同一图像中识别多个独立“连接任务”并理解一种隐含的“图层”或“绘制优先级”规则的能力。

`generate_arc_conditional_perpendicular_lines.py`: 测试模型根据物体的属性 (颜色) 和全局参照物 (边界线、图像边缘) 来执行不同几何操作的能力。

`generate_arc_column_projection.py`: 测试模型识别复杂的上下文关系 (“在...下方且在...范围内”) 并执行条件性列操作的能力。

`generate_arc_procedural_spiral.py`: 测试模型执行一个迭代的、程序性的生成算法的能力。模型需要理解指令、跟踪状态 (当前位置、方向、长度) 并循环执行。

`generate_arc_fractal_stamping.py`: 测试模型理解和执行递归或分形生成规则的能力。模型需要将输入图案本身作为一个“笔刷”, 根据输入图案中的“指令”进行重复绘制。

`generate_arc_flood_fill.py`: 测试模型执行经典的“洪水填充” (Flood Fill) 或“油漆桶”算法的能力。[ARC Prize - Puzzle #36a08778](#)

`generate_arc_layered_fill.py`: 测试模型理解一个程序性极强的、依赖于拓扑距离和条件判断的复杂填充算法。

`generate_arc_fluid_simulation.py`: 测试模型在图像空间中学习和模拟一个具有特定规则的流体动态过程的能力。

`generate_arc_periodic_conditional_fill.py`: 这个实验旨在测试模型学习一个复杂的、带有周期性和特殊 case 的条件格式化规则的能力。

`generate_arc_fill_square_holes.py`: 这个实验用于测试模型进行多步视觉推理的能力: 首先需要识别出复杂的“背景中的前景” (即矩形中的空洞), 然后对识别出的对象进行几何属性判断 (是否为正方形), 最后根据判断结果进行着色。

`generate_arc_conditional_recoloring.py`: 测试模型理解视觉图层和进行条件性对象属性修改的能力。

`generate_arc_sort_by_length_remap_position.py`: 测试模型执行一个“属性-位置解耦与重映射”的复杂排序任务。

`generate_arc_jigsaw_puzzle_simple.py`: 测试模型解决一个视觉匹配与变换问题的能力（早期版本）。[ARC Prize - Puzzle #898e7135](#)

`generate_arc_jigsaw_puzzle_advanced.py`: 测试模型解决一个复杂的视觉匹配与变换问题的能力。[ARC Prize - Puzzle #898e7135](#)

`generate_arc_connect_path_by_sequence.py`: 测试模型解析外部指令序列，并据此在图像中执行多步、有状态的路径连接任务的能力。

`generate_arc_reflection_simulation_deprecated.py`: (已废弃) 旨在测试模型理解复杂的基于物理光学的规则，包括射线发射、碰撞检测、角度反射和颜色变换。